ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

# Dockerfile refactoring: detection and effects

**Diogo Pereira Vieira**

**Master in Computer Engineering, Specialization in Cybersecurity and Systems Administration**

**Supervisor: Isabel Azevedo (ISEP)**

Porto, June 29 2026

Página em branco [apagar este comentário]

# Statement of Integrity

I hereby declare having that I have conducted this academic work with integrity.

I have not plagiarised or applied any form of undue use of information or falsification of results along the process leading to its elaboration.

Therefore, the work presented in this document is original and authored by me, having not previously been used for any other end. The exceptions are explicitly recognised in the section “Ethical considerations” of the first chapter. This section also states how AI tools were used and for what purpose.

I further declare that I have fully acknowledged the Code of Ethical Conduct of P.PORTO.

**ISEP, Porto, 20 September, 2026**

Página em branco [apagar este comentário]

# Dedicatory

....

Página em branco [apagar este comentário]

# Resumo

A refatoração é uma prática consolidada no desenvolvimento de software tradicional, onde os programadores reestruturam código existente para melhorar a sua estrutura interna sem alterar o comportamento externo. No entanto, enquanto a refatoração tem sido extensivamente estudada no contexto de código-fonte convencional, a sua aplicação a Infraestrutura como Código (IaC), particularmente Dockerfiles, permanece relativamente pouco explorada.

Docker tornou-se popular porque permite aos programadores empacotar aplicações com dependências em contentores isolados e portáteis. Um Dockerfile define como cada imagem é construída. Quando um projeto cresce, a complexidade aumenta, o tamanho das imagens sobe e o tempo de construção fica mais lento. Isso reduz o desempenho e aumenta o risco de problemas de segurança.

A medida que o código evolui os desenvolvedores fazem alterações nos DockerFiles e essas alterações afetam simultaneamente múltiplas dimensões de qualidade de formas conflituantes e esta relação entre estas dimensões de qualidade cria um problema técnico com várias dimensões que exige decisões equilibradas.

Existe uma lacuna importante, porque ferramentas recentes como o DRMiner começaram a detetar refatorações em Dockerfiles, mas ainda não existem métodos sistemáticos que detetem essas refatorações e avaliem de forma empírica o impacto em métricas de qualidade chave. Os catálogos atuais de refatorações para Dockerfiles têm alcance curto e acabam por não organizar as refatorações em categorias práticas de desempenho, segurança e manutenibilidade.

Esta dissertação usa a metodologia Design Science Research e o seu objetivo é criar e avaliar artefactos que resolvem problemas reais da organização, a mesma foca em dois objetivos principais:

1. Estender o catálogo de refatorações de Dockerfiles podendo distinguir entre os que são considerados desempenho, segurança e manutenibilidade. Este ponto consiste em ler a literatura existente,examinar os “Docker Best Practices” e depois consolidar o conhecimento numa classificação bem estruturada.
2. Fornecer uma ferramenta protótipo que tanto detete refatorações passadas em históricos de versões de Dockerfiles como avalie empiricamente o impacto das refatorações detetadas em métricas de qualidade chave, como tempo de construção, tamanho da imagem e indicadores de segurança.

Para guiar a investigação, foram definidas três questões de investigação: (RQ1) Qual é o estado da arte relativamente à deteção, classificação e avaliação de impacto de refatorações em Dockerfiles? (RQ2) Como pode um catálogo de refatorações de Dockerfiles ser estendido para incluir classificações por dimensões de qualidade? (RQ3) Qual é a eficácia de uma ferramenta protótipo na deteção automática de refatorações e na avaliação do seu impacto?

O protótipo será avaliado através de experiências num conjunto de repositórios de teste controlados e estruturados para o efeito, medindo e reportando o impacto das refatorações nas métricas de qualidade. As contribuições esperadas contem um catálogo estendido de refatorações, a ferramenta protótipo, uma análise empírica e um conjunto de dados para utilização pela comunidade científica.

**Palavras-chave:** Dockerfile, Refatoração, Infraestrutura como Código, Contentorização, Qualidade de Software, DevOps

Página em branco [apagar este comentário]

# Abstract

Refactoring is common in software development, but it remains rarely applied to Infrastructure as Code, especially in Dockerfiles, which is the focus of this project. As containerization becomes increasingly prevalent, developers face challenges in maintaining Dockerfile quality across multiple dimensions: performance, security, and maintainability. Different refactorings can simultaneously affect these dimensions in conflicting ways, making it difficult to assess the impact of a change on quality metrics.

This dissertation follows a Design Science Research methodology to address two main objectives: First, the existing catalogue of Dockerfile refactorings is extended and after is classified according to quality dimensions such as performance, security and maintenance. Second, a prototype tool is proposed that is capable of automatically detecting refactorings from version history and empirically evaluating their impact on quality metrics.

The prototype is empirically validated against a curated set of controlled testbed repositories, with evaluation presented in Chapter 6. The expected contributions include an extended refactoring catalogue, the prototype tool, and an enriched dataset for future research.

**Keywords:** Dockerfile, Refactoring, Infrastructure as Code, Containerization, Software Quality, DevOps

Página em branco [apagar este comentário]

# Acknowledgement

I’d like to thank my supervisor Isabel Azevedo for all her guidance and for always being available to help during the course of this work.
I would also like to thank my family, who provided me with all the materials I needed to succeed in life, my friends for their support
And finally, I would like to thank all my colleagues and teachers who have passed through my academic life at ISEP.

Página em branco [apagar este comentário

# Table of Contents

**List of Figures** ..... **xix**

**List of Tables** ..... **xxi**

**List of Listings** ..... **xxiv**

**List of Abbreviations** ..... **xxvii**

**1 Introduction** ..... **1**

1.1 Context and Problem ..... 1

1.2 Objectives, Research Questions and Methodology ..... 2

1.3 Ethical Considerations ..... 3

1.3.1 Use of AI Tools ..... 5

1.4 Contributions ..... 5

1.5 Document Structure ..... 6

**2 Background** ..... **8**

2.1 Docker and Infrastructure as Code ..... 8

2.2 The Build System, Caching and Layers ..... 9

2.3 Refactoring Theory ..... 10

2.4 Quality Dimensions in Dockerfile Context ..... 10

2.5 Technical Foundations ..... 10

2.5.1 Abstract Syntax Trees ..... 11

2.5.2 Static Analysis ..... 11

2.5.3 Mining Software Repositories ..... 11

2.5.4 Common Vulnerabilities and Exposures ..... 11

**3 Literature Review** ..... **12**

3.1 Methodology ..... 12

3.1.1 Data Sources ..... 12

3.1.2 Search Strategy ..... 13

3.1.3 Selection Criteria ..... 13

3.1.4 Selection Flow (PRISMA Diagram) ..... 14

3.2 State of the Art Analysis ..... 20

3.2.1 Large-Scale Analysis of the Docker Ecosystem ..... 20

3.2.2 Characterization of Dockerfile Smells ..... 20

3.2.3 Temporal Persistence of Dockerfile Smells ..... 21

3.3 Limitations of Existing Dockerfile Smell Catalogues ..... 21

3.3.1 Syntactic Focus Without Quality Dimension Mapping ..... 22

3.3.2 Lack of Empirical Impact Assessment on Quality Dimensions ..... 22

3.3.3 Limited Coverage of Security and Reproducibility Metrics ..... 22

3.4 Existing Tools for Dockerfile Quality Analysis and Refactoring ..... 23

3.4.1 DRMiner: Dockerfile Refactoring Detection ..... 23

3.4.2 Hadolint: Static Analysis for Dockerfiles ..... 26
3.4.3 Docker-Parfum: Automated Dockerfile Smell Detection and Repair ..... 27
3.4.4 Comparative Analysis and Research Gap ..... 28
3.5 Positioning of This Dissertation ..... 29
**4 Design ..... 30**
4.1 Prototype Tool ..... 34
4.1.1 Technical Description ..... 34
4.1.2 Planned Changes in the System ..... 35
4.1.3 Metric Selection and Scope ..... 37
4.1.4 Challenges ..... 39
4.1.5 Functional and Non-Functional Requirements ..... 40
4.1.6 Architecture Design ..... 42
4.1.7 Verification and Validation Strategy ..... 48
4.2 Extended Refactoring Catalogue ..... 49
4.2.1 Classification Criteria ..... 58
4.2.2 Maintainability Established by the Nature of the Operation ..... 58
4.2.3 Distribution of the Catalogue ..... 59
4.3 VCS Connector ..... 59
4.3.1 Technical Description ..... 59
4.3.2 Design and Requirements ..... 60
4.4 Detection Engine ..... 61
4.4.1 Technical Description ..... 61
4.4.2 Design and Requirements ..... 62
4.4.3 Supported Refactoring Types ..... 62
4.5 Performance Analyzer ..... 65
4.5.1 Technical Description ..... 65
4.5.2 Design and Requirements ..... 66
4.6 Data Extractor ..... 67
4.6.1 Technical Description ..... 67
4.6.2 Security Metrics — Trivy ..... 68
4.6.3 Maintainability Metrics — Hadolint ..... 69
4.6.4 Maintainability Metrics — Logical Instruction Count ..... 69
4.7 Report Generator ..... 69
4.7.1 Technical Description ..... 70
4.8 System Data Flow ..... 71
4.9 Design Limitations and Mitigations ..... 71
4.10 Summary ..... 74
**5 Implementation ..... 75**
5.1 Implementation Overview and Prototype Orchestration ..... 75
5.1.1 Stage structure ..... 76
5.1.2 Historical commit contexts ..... 78
5.1.3 Failure handling and exit codes ..... 79
5.1.4 Technologies and execution ..... 80
5.1.5 Scope of the implementation ..... 83
5.2 VCS Connector ..... 83
5.2.1 Implementation ..... 84
5.2.2 Line endings on Windows ..... 85
5.2.3 Testing and conformance ..... 86

5.3 Detection Engine ..... 86
5.3.1 Parsing ..... 86
5.3.2 Rule registry and the three architectural laws ..... 87
5.3.3 Rule families ..... 88
5.3.4 Composite detections ..... 91
5.3.5 RNF3 exercised ..... 91
5.3.6 Testing ..... 92
5.4 Performance Analyzer and Data Extractor ..... 93
5.4.1 Performance Analyzer ..... 93
5.4.2 Data Extractor ..... 97
5.5 Report Generator ..... 102
5.5.1 Output artifacts ..... 102
5.5.2 Aggregation, not interpretation ..... 103
5.5.3 Provenance ..... 103
5.5.4 A naming correction ..... 104
5.5.5 Testing and end-to-end verification ..... 104
5.6 Conformance Verification and Results ..... 105
5.6.1 Testing strategy ..... 105
5.6.2 Test inventory ..... 106
5.6.3 Coverage audit ..... 107
5.6.4 System-wide conformance verification ..... 109
5.6.5 Summary of results ..... 111
**6 Experimentation and Evaluation ..... 112**
6.1 Experimental Setup ..... 114
6.1.1 Hardware and operating system ..... 114
6.1.2 Technology stack ..... 114
6.2 Detection Accuracy ..... 115
6.2.1 Evaluation corpus ..... 115
6.2.2 External specification ..... 117
6.2.3 Tool execution ..... 118
6.2.4 Automated audit ..... 118
6.2.5 Results ..... 118
6.3 Evaluation of Metric Impact (Impact Measurement Reliability) ..... 119
6.3.1 Performance Impact (ΔSize) ..... 119
6.3.2 Security Impact (ΔCVEs) ..... 121
6.3.3 Maintainability Impact (ΔWarnings and ΔInstr) ..... 124
6.4 Discussion of Results ..... 127
6.4.1 Detetion Engine ..... 127
6.4.2 Performance Impact (ΔSize) ..... 128
6.4.3 Security Impact (ΔCVEs) ..... 130
6.4.4 Maintainability Impact (ΔWarnings and ΔInstr) ..... 133
6.5 Threats to Validity ..... 137
6.5.1 Detection ..... 137
6.5.2 Performance ..... 138
6.5.3 Security ..... 138
6.5.4 Maintainability ..... 138
**7 Conclusion ..... 139**
7.1 Fulfilment of the research objectives ..... 139
7.2 Principal findings ..... 140

7.3 Limitations ..... 141
7.4 Future work ..... 141
7.4.1 Multi-Dockerfile support ..... 141
7.4.2 Input mapping mechanisms ..... 142

**Reference ..... 143**

**Appendix A — Skills Management ..... 1**

Required Skills and Current Proficiency ..... 1

Development Strategy for Identified Gaps ..... 4

Monitoring and Impact on Success ..... 5

**Appendix B — Project Planning ..... 6**

Scope and Objectives ..... 6

Work Breakdown Structure ..... 8

Integrated Project Schedule ..... 17

Risk Management ..... 19

**Appendix C — Maintainability Screen Rules ..... 25**

**Inserir página em branco apenas se necessário de modo a que a próxima secção comece numa página à direita.**

# List of Figures

Figure 3.1 - PRISMA flow diagram of the systematic literature selection process. .....17
Figure 4.1 - Sequential architecture of the prototype analysis pipeline, showing the execution order of the components. The data dependencies between components are detailed in Figure 4.2. .....44
Figure 4.2 - Component diagram of the prototype: modules, provided and required interfaces, dependencies, and external tools.....45
Figure 4.3 - Use case diagram of the Dockerfile refactoring analysis tool. .....46
Figure 4.4 - Deployment diagram of the prototype, showing the Python application, the Docker daemon, and the tool and image containers. .....47
Figure B.1 - Work Breakdown Structure (WBS) for the Docker refactorings project .....10
Figure B.2 - Tasks of the Project Planning and Management phase .....11
Figure B.3 - Tasks of the Literature Review phase.....12
Figure B.4 - Tasks of the Extended Catalogue Construction phase .....13
Figure B.5 - Tasks of the Prototype Design phase .....14
Figure B.6 - Tasks of the Prototype Implementation and Verification phase .....15
Figure B.7 - Tasks of the Evaluation Corpus and Validation phase.....16
Figure B.8 - Tasks of the Closure phase.....17
Figure B.9 - Project schedule showing the seven dissertation phases, their durations, dependencies, and key milestones from October 2025 to September 2026 .....17

Inserir página em branco apenas se necessário de modo a que a próxima secção comece numa página à direita

# List of Tables

Table 3.1 - Summary of electronic databases and academic search engines utilized for the systematic literature search ..... 12
Table 3.2 - Academic databases and specific search queries used in the systematic literature review ..... 13
Table 3.3 - Inclusion and exclusion criteria, separated by selection arm (systematic database search and targeted handsearching), following the two-arm PRISMA 2020 model. ..... 14
Table 3.4 - Quantitative summary of the systematic literature selection process, detailing the number of records retained across the Identification, Screening, Eligibility, and Included phases of the PRISMA methodology ..... 15
Table 3.5 - Full set of references used in this chapter, the studies retained from the systematic review together with the supplementary sources (documentation, tools, books, and reports), with their type and specific role. ..... 20
Table 3.6 - The fourteen Dockerfile refactoring types identified by Ksontini et al. [2] ..... 25
Table 3.7 - Capability comparison of existing Dockerfile analysis tools and the proposed prototype ..... 28
Table 4.1 - Summary of the five rapid experiments, their objectives, methods, results, and the system components whose design they ground. ..... 33
Table 4.2 - Prototype components, their responsibilities, technologies, and satisfied requirements. ..... 48
Table 4.3 - Extended Dockerfile refactoring catalogue mapping fourteen types across three quality dimensions ..... 50
Table 4.4 - Version control system access methods considered for the VCS Connector ..... 60
Table 4.5 - Dockerfile parsing strategies considered for the Detection Engine ..... 61
Table 4.6 - Relationship between the fourteen catalogue refactorings and the reviewed tools (Hadolint reports smells; DRMiner detects refactorings), with the implementation decision for the prototype ..... 65
Table 4.7 - Image size retrieval methods considered for the Performance Analyzer ..... 66
Table 4.8 - Tool invocation strategies considered for the Data Extractor. ..... 68
Table 4.9 - Output formats considered for the Report Generator. ..... 70
Table 4.10 - Data transformations at each pipeline stage. ..... 71
Table 5.1 - The five components of the prototype and the stage in which each executes ..... 77
Table 5.2 - Mapping of failures to process exit codes ..... 80

Table 5.3 - External technologies used by each component and by the preparation phase, with the role each one performs ..... 81
Table 5.4 - The four rule families of the Detection Engine and the criterion each applies ..... 88
Table 5.5 - Distribution of the 111 rule tests across the fourteen catalogue types ..... 92
Table 5.6 - Four consecutive CLI builds of the R09 pair, against the values recorded in the experiment ..... 96
Table 5.7 - Values produced by the Data Extractor against those recorded in the four catalogue experiments carrying a metrics file ..... 100
Table 5.8 - Output of the first full end-to-end run, against the R09 catalogue pair ..... 105
Table 5.9 - Distribution of the test suite across the modules of the prototype ..... 107
Table 5.10 - Statement coverage per module across the complete prototype package ..... 108
Table 5.11 - Conformance of each component against the requirements of the design chapter ..... 110
Table 6.1 - Distribution of the 49 commit pairs across the fourteen catalogue rules and the four case classes. A pair performing several refactorings is counted once under each rule it exercises, which is why the row totals exceed the number of pairs. ..... 117
Table 6.2 - Conformance of the detection engine with the reference specification, by case class ..... 119
Table 6.3 - Image size measured over the isolated R10 commit pair, as reported by the prototype ..... 121
Table 6.4 - Vulnerability counts by severity over the isolated R02 commit pair, as reported by the prototype. ..... 123
Table 6.5 - Filtered warning counts over the maintainability commit pair, as recorded in the metrics.warnings block of the structured report. ..... 126
Table 6.6 - Logical instruction counts over the same commit pair, recorded under metrics.logical_instructions.delta_logical_instructions ..... 126
Table A.1 - Assessment of current skills ..... 4
Table B.1 - Mapping of DSR Activities to WBS Phases and Deliverables ..... 9
Table C.1 - Complete list of the 47 rules comprising the maintainability screen, grouped by their respective sub-categories, including their identifier, source tool, and official description. ..... 30

# List of Listings

Listing 5.1 - Structure of the prototype repository....76
Listing 5.2 - Command-line invocation of the prototype....82
Listing 5.3 - Main function signature and entry point of the VCS Connector....84
Listing 5.4 - Retrieval of the Dockerfile content at a single commit....85
Listing 5.5 - Shape of a detection rule, shown for R01....88
Listing 5.6 - Construction of the image pair, with removal guaranteed by the finally clause....94
Listing 5.7 - Computation of the size delta from the two prepared images....95
Listing 5.8 - Concurrent dispatch of the four external tool invocations....99
Listing 5.9 - Retention of the raw tool output at the component boundary....100
Listing 6.1 - Initialization of the isolated local repository for performance validation under /var/tmp....120
Listing 6.2 - Creation of the initial commit and capture of the before-state commit hash for the performance validation scenario....120
Listing 6.3 - Creation of the after-state commit by switching the base image to Alpine and capturing its commit hash for performance validation....120
Listing 6.4 - Invocation of the prototype over the performance validation commit pair, specifying the repository path, commit hashes, and the output directory....120
Listing 6.5 - Inspection of the human-readable summary report generated by the prototype for the performance validation scenario....121
Listing 6.6 - Initialization of the isolated local repository and Git configuration for the security validation scenario....121
Listing 6.7 - Creation of the initial state application file (app.js) and Dockerfile using an older Node.js base image for the security validation scenario....122
Listing 6.8 - Creation of the after-state Dockerfile shifting to the Alpine-based image variant, committing the change, and capturing the commit hash for security validation....122
Listing 6.9 - Invocation ofței prototype over the security validation commit pair, specifying the repository path, commit hashes, and the output report directory....122
Listing 6.10 - Initialization and configuration of the scratch repository for maintainability validation under /var/tmp....124
Listing 6.11 - Creation of the before-state Dockerfile with a mutable base image tag and separate RUN instructions for maintainability validation, along with commit hash capture....125
Listing 6.12 - Creation of the after-state Dockerfile pinning the image tag and inlining the RUN instructions for maintainability validation, along with the subsequent commit and hash capture....125
Listing 6.13 - Invocation of the prototype over the maintainability validation commit pair, specifying the output directory and capturing both output streams to a log file....125
Listing 6.14 - Abbreviated excerpt from src/performance_analyzer.py (lines 95–135); exception handling blocks omitted....129
Listing 6.15 - Invocation of the Trivy scanner over the built image and raw byte retention (_scan_image and raw_trivy....131
Listing 6.16 - Environment provenance block within the structured report (impact_report.json ....133
Listing 6.17 - Static declaration of the severity hierarchy (SEVERITY_TIERS) and categorical vulnerability aggregation logic....133
Listing 6.18- Declarative 47-rule maintainability screen definition and screening logic....135

Listing 6.19 - Invocation of Hadolint via stdin streaming and raw output capture .....136
Listing 6.20 - Structured report block (impact_report.json) recording warning counts and delta .....136
Listing 6.21 - Structured report block (impact_report.json) recording logical instruction counts and delta .....137

**Inserir página em branco apenas se necessário de modo a que a próxima secção comece numa página à direita**

# List of Abbreviations

| | |
|---|---|
| ACM | *Association for Computing Machinery* |
| **AI** | *Artificial Intelligence* |
| **API** | *Application Programming Interface* |
| **AST** | *Abstract Syntax Tree* |
| **CI/CD** | *Continuous Integration / Continuous Deployment* |
| **DSR** | *Design Science Research* |
| **IaC** | *Infrastructure as Code* |
| **IEEE** | *Institute of Electrical and Electronics Engineers* |
| **IPP** | *Instituto Politécnico do Porto* |
| **ISEP** | *Instituto Superior de Engenharia do Porto* |
| **MSR** | *Mining Software Repositories* |
| **PREPD** | *Preparação para Dissertação* |
| **RQ** | *Research Question* |
| **WBS** | *Work Breakdown Structure* |

**Inserir página em branco apenas se necessário de modo a que a próxima secção comece numa página à direita.**

# 1 Introduction

This chapter provides an overview of the context and problem addressed in this dissertation, the objectives defined, the research methodology adopted, ethical considerations, contributions, previous work, and the structure of the document.

## 1.1 Context and Problem

Refactoring is a disciplined technique for restructuring existing code through behavior-preserving transformations that improve internal structure without altering external behavior [1]. Although it is a well-established practice in traditional software engineering, its application to Dockerfile refactoring remains largely unexplored [2]. This gap is significant given the dominance of Docker: more than 71% of developers use Docker as their leading cloud technology [3] and 91% of organizations use or evaluate container technologies [4].

Dockerfiles tend to accumulate quality issues over time. Cito et al. [5] analyzed more than 70,000 Dockerfiles and found that they evolve more slowly than source code, with an average of only 3.11 revisions per year, but accumulate quality issues that affect image size and build performance. In software projects, technical debt can survive on average more than 1,000 commits before being resolved [6]. Similar patterns of technical debt accumulation are observed in Docker projects, where Ksontini et al. [2] identified multiple categories of technical debt including understandability and maintainability issues.

A critical feature of Dockerfile refactoring is that different optimizations affect multiple dimensions of quality in conflicting ways. For example, consolidating RUN instructions reduces the number of image layers and improves performance [7] but decreases readability [2]. Wu et al. [8] characterized the occurrence of Dockerfile smells in open-source projects, demonstrating the prevalence of these problematic patterns.

DRMiner was developed by Ksontini et al. [9] as the first tool to identify and analyze refactorings in Dockerfiles. However, the tool focuses on detection and does not evaluate the impact of refactorings on quality metrics. Static analysis tools such as Hadolint [10] identify smells but are not geared toward refactoring detection. Currently, refactoring catalogs for Dockerfiles remain limited [2], without a structured classification by quality dimensions.

This problem affects several stakeholders: developers who write and maintain Dockerfiles; DevOps teams that manage CI/CD pipelines where the quality of Dockerfiles impacts deployment efficiency; and organizations that rely on Docker environments to ensure efficiency and security. Solving this problem requires knowledge of Software Engineering, Performance Engineering, and Software Security.

## 1.2 Objectives, Research Questions and Methodology

This dissertation is structured around two main objectives and three research questions, which are directly mapped as follows:

(O1) Extend the catalogue of Dockerfile refactoring types by classifying them according to quality dimensions — performance, security, and maintainability. → Addressed by **RQ2**.

(O2) Develop a prototype tool that detects past refactorings in Dockerfile version histories and empirically evaluates their impact on key quality metrics. → Addressed by **RQ3.**

Both objectives are informed by the state of the art characterised in response to **RQ1.**

To achieve these objectives, this dissertation follows a Design Science Research (DSR) [11] methodology, which is appropriate for creating and evaluating artifacts intended to solve identified real-world problems. According to vom Brocke et al., DSR is structured around six activities:

**Problem identification and motivation:** Addressed in Section 1.1, identifying the lack of structured refactoring catalogues and the absence of tools to assess the practical impact of refactorings on quality metrics.

**Definition of objectives for a solution:** Defining the requirements for an extended catalogue (comprehensiveness and multi-dimensional classification) and a prototype tool (accuracy in detection and reliability in impact measurement).

**Design and development:** Involves the conceptual structuring of the **extended refactoring catalogue** and the technical architecture of the prototype tool, including repository mining, detection algorithms (AST and regex-based), and metrics collection modules.

**Demonstration:** Application of the artifacts to a **curated set of controlled testbed repositories** from GitHub to prove their functional feasibility in detecting refactoring instances. In the project planning (WBS), this activity is integrated into the Evaluation Corpus and Validation for operational efficiency.

**Evaluation: Systematic analysis of the indicators** of the behavioral effects of refactorings. This involves assessing detection precision and correlating refactoring events with empirical changes in performance, maintainability, and security indicators (CVE counts). Similarly to the demonstration, this activity is integrated into the "Evaluation Corpus and Validation" phase of the project planning (WBS) for operational efficiency

Communication: Dissemination of findings through this dissertation and the public availability of the artifacts, prototype source code, and enriched datasets in a public GitHub repository (available at

https://github.com/DiogoVieira123/diogov_Dockerfile_refactorings_detection_and_effects ).

The research produces two main artifacts that address the problems identified in Section 1.1. The extended refactoring catalogue addresses the problem that existing catalogues lack a structured classification by quality dimensions, it categorizes Dockerfile refactorings according to performance, security, and maintainability, providing developers with empirical evidence of the quality impact of each refactoring type, enabling informed decision-making. The prototype tool addresses the problem that current tools like DRMiner focus on detection but do not evaluate impact, it automatically identifies refactorings to evaluate their effects on three key quality attributes: maintainability, performance and security, enabling evidence-based refactoring decisions.

To guide the research, three research questions are defined. The first question addresses the current state of knowledge regarding refactoring practices in Dockerfiles. The second question focuses on the extension and organization of a refactoring catalogue according to quality dimensions. The third question evaluates the practical effectiveness of an automated detection and assessment tool.

- **RQ1:** What is the state of the art regarding the detection, classification, and impact assessment of refactorings in Dockerfiles?
- **RQ2:** How can a catalogue of Dockerfile refactorings be extended to include classifications by quality dimensions (performance, security, and maintainability)?
- **RQ3:** What is the effectiveness of a prototype tool in automatically detecting refactorings and assessing their impact on performance, security, and maintainability?

To ensure a rigorous evaluation, effectiveness in the context of RQ3 will be measured through three complementary dimensions:

- Detection Accuracy: evaluated via precision and recall metrics computed against a ground-truth set of controlled experiments and a dedicated test suite, in which each of the fourteen refactoring types is applied in isolation and the tool's output is rigorously verified against the known transformation.
- Impact Measurement Reliability: assessed by validating computed size deltas (ΔSize) against controlled baseline experiments using byte-precise Docker SDK measurements, by verifying tool integration fidelity for security metrics directly against raw Trivy scanner outputs under reproducible execution environments, and by ensuring the strict application of the 47-rule maintainability screen alongside manual cross-validation of the internal logical instruction counter (ΔInstr).

A systematic literature review will identify how refactorings affect quality dimensions, compiling findings into the extended catalogue. The prototype implements repository mining, detection algorithms based on syntactic analysis, and empirical impact evaluation across four metrics, reporting the detected refactoring type alongside the measured deltas. Evaluation will use controlled testbed repositories with analysis to determine the impact of refactorings on quality metrics (Chapter 6)

## 1.3 Ethical Considerations

The ethical considerations guiding this dissertation follow the standards established by the Instituto Politécnico do Porto (IPP) Code of Conduct [12] and the principles of the IEEE Code of Ethics [13]. These structures ensure that work is conducted with integrity, transparency, and accountability in a technical context, including software artifacts, automated analysis tools, and evaluation methods.

A fundamental ethical principle concerns the possession of adequate competence and knowledge, and according to the IEEE Code of Ethics [13] individuals should only perform tasks for which they are adequately qualified, this principle directly influenced the project preparation phase which involved acquiring and consolidating knowledge in areas such as containerization, Infrastructure as Code, Dockerfile semantics, static analysis, quality metrics evaluation, and empirical software engineering. These skills were developed through courses and collaborative work with the supervisor throughout the project preparation phase, to ensure that all choices made during the project are scientifically sound and responsibly executed.

The research methodology also has its ethical considerations related to the use of software tools, and all the tools used in this work are applied transparently and with an awareness of their limitations, ensuring that they do not hinder critical human interpretation. These tools support development in both theoretical and practical aspects but do not produce conclusions on their own. The results they generate are then analyzed and validated by the author to ensure scientific rigor.

The study analyzes Dockerfiles retrieved from public repositories, always ensuring they have appropriate licenses for research use. The entire process is conducted exclusively with these technical files, and no personal data is collected or processed at any stage. The analysis focuses on technical components, build instructions, dependency configurations, layer structures, and refactoring patterns.

Findings are reported in aggregate form to avoid exposing specific contributors or projects unnecessarily, respecting the norms of open-source communities and avoiding any misrepresentation or harm.

Another relevant ethical dimension concerns accuracy, transparency, and reproducibility, with the IEEE Code of Ethics [13] emphasizing honesty and clarity in the presentation of technical results, in this project all the procedures used will be documented to allow their replication by other researchers. The limitations related to the methodology will be explicitly acknowledged to avoid overstating the reliability or generality of the conclusions.

Finally, the project respects the principle of responsible planning mentioned in the IEEE guidelines, and the work follows a realistic schedule with milestones and deliverables aligned with the academic calendar and institutional expectations. Trade-offs involving time, complexity, and scope are made explicit in project planning to ensure balanced decision-making and avoid compromising the quality of the final work.

### 1.3.1 Use of AI Tools

Artificial intelligence tools were used in this dissertation in a controlled and transparent manner, following institutional guidelines and ensuring that academic integrity was never compromised. AI-based language models were used selectively for clarifying technical concepts, improving readability and organizational support.

- Clarifying technical concepts: assisting in understanding complex ideas related to build systems, refactoring patterns, and Docker internals.
- Improving readability: reviewing grammar, flow, and clarity of text when needed.
- Organizational support: Proposing different ways to organize the sections or present the methodology without interfering with the author's final decisions.

No content produced by any AI tools was incorporated without human revision and adaptation and all research tasks such as literature review, experimental design, data analysis, and interpretation were carried out by the author himself. AI tools were not used to generate code, analyze repositories, conduct experiments or interpret empirical results. Transparency regarding the use of AI is applied throughout the entire project and the final document meets the requirements by ensuring that all content generated by AI remains below the limit. All conclusions, discussions and technical results are based on the author’s reasoning, with this reasoning being complemented by academic sources and by his practical experience.

## 1.4 Contributions

This research dissertation contributes at both academic and practical levels within its application area of Dockerfile refactoring analysis. The main contributions are the extended catalogue of Dockerfile refactorings, the prototype tool, the empirical analysis, and the enriched dataset.

The extended catalogue of Dockerfile refactorings provides a comprehensive and structured classification of Dockerfile-specific refactorings into performance-oriented, security-oriented, and maintainability-oriented categories. This catalogue serves as a basis for future research and practical application, enabling developers and researchers to understand the empirical effects of each refactoring type on image size, CVE count, Hadolint violations, and logical instruction count.

The prototype tool automates the detection of Dockerfile refactorings from version control history and evaluates their impact on quality metrics. This tool allows developers and researchers to analyze the effects of refactoring decisions on image size, security indicators (CVE counts), and maintainability. Build time is identified as a relevant performance metric but is excluded from the prototype due to its high sensitivity to environmental factors, as discussed in Section 4.5.1.

The empirical analysis consists of an evaluation on a controlled testbed of Dockerfile repositories that validates the proposed classification and provides evidence on the measured impact of different refactoring types on performance, maintainability, and security indicators.

The enriched dataset contains annotated Dockerfile refactorings and their measured impacts. This dataset will be made available to the research community to support future studies in this domain.

## 1.5 Document Structure

This section provides an overview of the document structure. This dissertation is divided into several chapters:

- Chapter 1 - Introduction: Presents the context and problem being addressed, the objectives defined, the research methodology, ethical considerations, contributions, previous work, and the document structure.
- Chapter 2 - Background: Provides the foundational context and theoretical basis necessary to understand the technologies, concepts, and methodologies employed in this thesis, including Docker, containerization, Infrastructure as Code, and refactoring theory.
- Chapter 3 - Literature Review: Presents a systematic analysis of existing research on Dockerfile refactorings, code smells, and quality metrics, establishing the theoretical foundation for this dissertation and identifying the gaps that this work addresses.
- Chapter 4 - Design: Presents the design phase of the tool, thus describing the planned architecture, its design objectives, and the justifications for the decisions made.
- Chapter 5 - Implementation: Describes how the tool was built in detail and how the decisions made in the design chapter were applied in practice using current development tools and methods.
- Chapter 6 - Experimentation & Evaluation: Presents the results of the experimentation and the evaluation of the prototype on controlled testbed repositories, including the analysis of results.
- Chapter 7 - Conclusion: Reflects on the achievements of this dissertation, acknowledges limitations, and outlines directions for future research.

The document also includes three appendices. Appendix A — Skills Management identifies the technical and interpersonal skills relevant to the project objectives, assesses the current proficiency level for each one, and sets out the development strategy for the gaps found. Appendix B — Project Planning describes the work breakdown structure, the project schedule, and the risk management strategy that support the execution of the work. Appendix C — Maintainability Screen Rules lists the 47 Hadolint and ShellCheck rules that make up the maintainability screen applied to the extended-catalogue experiments.

# 2 Background

This chapter establishes the technical foundations required to understand the remainder of this dissertation. It begins by introducing Docker and Infrastructure as Code, followed by the build system, caching, and layering mechanisms that determine image size and reproducibility. It then presents refactoring theory and the quality dimensions relevant to the Dockerfile context, and closes with the technical foundations, abstract syntax trees, static analysis, mining software repositories, and the CVE system, that underpin the detection and measurement approach adopted in this work.

## 2.1 Docker and Infrastructure as Code

Infrastructure as Code (IaC) refers to the practice of managing and provisioning computing infrastructure through machine-readable definition files rather than through manual configuration or interactive tools [5,14]. IaC promotes managing infrastructure knowledge inside reusable scripts, replacing manual and error-prone system administration practices [14]. By encoding infrastructure decisions in text files, organizations gain the ability to track changes to infrastructure over time using version control systems, apply code review processes to infrastructure modifications, automate infrastructure deployment through CI/CD pipelines, and reproduce environments consistently across development, testing, and production stages.

Docker is an open-source platform that enables developers to package applications and their dependencies into lightweight, portable containers [5,15]. Unlike traditional virtual machines that require a complete operating system, Docker containers share the kernel of the host system while maintaining isolated execution environments [5,15]. This strategy reduces resource consumption and improves startup times, making containers particularly well suited for modern cloud-native and microservices architectures. The container includes the runtime environment, applications, libraries, and configuration files, thus behaving the same way across different environments [15]. This portability addresses the classic environment inconsistency problem, as the containerized application carries its entire execution context with it [15,16].

A Dockerfile is a text document that contains all the commands a user could call on the command line to assemble an image [7,8]. Each instruction represents a discrete step in the image-building process, executed in order by the Docker daemon during the build operation [5]. Common instructions include FROM to specify the base image upon which the new image is built, RUN to execute commands inside the container during the build process, COPY and ADD to transfer files from the host system into the container filesystem, WORKDIR to set the working directory for subsequent instructions, CMD and ENTRYPOINT to define the default command that runs when a container is started, ENV to set environment variables, and EXPOSE to document which network ports the container listens on at runtime [5,7].

Dockerfiles are considered a form of IaC because they declaratively specify the construction of container images, following the notion of Infrastructure-as-Code [5,8]. The Dockerfile serves as the authoritative definition of the container construction process, making it a critical artifact in the software development lifecycle. Source code repositories containing Dockerfiles potentially enable the execution of program code in an isolated and fast environment [5]. Dockerfiles are versioned alongside application source code, reviewed in code reviews, and executed automatically in CI/CD pipelines [5,16].

## 2.2 The Build System, Caching and Layers

The Docker build process transforms a Dockerfile into an executable image through a sequence of operations managed by the Docker daemon [7]. When a user executes a build command, the daemon reads the Dockerfile, executes each instruction in order, and progressively constructs the image by creating and committing filesystem layers [5,7].

A critical performance optimization in this process is the build cache [7]. Docker caches the result of each instruction as an intermediate image layer. During subsequent builds, if Docker determines that an instruction and its context have not changed, it reuses the cached layer instead of re-executing the instruction [7]. This mechanism can dramatically reduce build times, particularly when installing large dependency sets or performing expensive compilation steps. Docker's strategy of splitting images into multiple layers speeds up the process by only downloading those resources which are not yet available locally [5].

The build cache operates on the principle of instruction fingerprinting. Docker computes a hash of the instruction text and its inputs such as files being copied [7]. If an identical instruction with identical inputs was executed previously, Docker retrieves the cached layer. If any part of the instruction or its context changes, Docker invalidates the cache from that point forward and executes all subsequent instructions. Cache invalidation is sequential: when a step's cache is invalidated by changing files or modifying lines in the Dockerfile, subsequent steps of their cache will break [7,16]. This property makes the ordering of instructions in a Dockerfile critical for build performance [7].

Docker images are composed of layers, where each layer represents the filesystem changes produced by a single Dockerfile instruction [7]. Layers are stacked to form a unified filesystem view, and they are shared between images that have common instruction sequences [5]. This layering architecture enables efficient storage and distribution, as multiple images can share base layers, and only differential layers need to be transferred when pulling or pushing images to registries [7].

However, layers also introduce complexities. Image size accumulates across layers, meaning that even if a file is deleted in a subsequent instruction, that file still occupies space in the

layer where it was originally added [7]. This phenomenon leads to layer bloat, where intermediate build artifacts or temporary files unnecessarily increase the final image size [5]. Additionally, each RUN instruction creates a new layer, so separate package update and installation instructions create two layers, whereas consolidating them into a single instruction creates one layer, reducing both the layer count and potential size overhead [7,8].

## 2.3 Refactoring Theory

Refactoring is a disciplined technique for restructuring existing code through behavior-preserving transformations that improve internal structure without altering external behavior [1]. Martin Fowler defines refactoring as “a change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behavior” [1]. The fundamental principle is that code can be modified to improve its quality attributes (such as readability, maintainability, or performance) while ensuring that the observable functionality remains unchanged [1].

In the context of Dockerfiles, refactoring involves modifying instructions and their organization to optimize non-functional quality attributes such as build performance, maintainability, and image size [2], with the latter also contributing to an improved security posture. Unlike traditional source code refactoring, where behavior preservation is verified through test suites [1] Dockerfile refactoring must ensure the functional equivalence of the resulting container image.

## 2.4 Quality Dimensions in Dockerfile Context

Three quality dimensions are relevant to Dockerfile refactoring, as identified in empirical studies on Docker quality [2,8]:

**Performance:** This dimension encompasses image size (the final artifact size) [2,5]. Performance is affected by instruction ordering, cache utilization, layer count, and base image selection [2,7].

**Security:** This dimension relates to the attack surface and vulnerability exposure of the resulting container [7,17]. Missing adherence to best practices can lead to security issues [17]. Security considerations include base image version pinning, removal of unnecessary packages, user privilege configuration, and the presence of known vulnerabilities (CVEs) in installed components [7,18].

**Maintainability:** This dimension addresses code clarity and developer experience [2]. Maintainability was identified as a major technical debt target for applied refactorings in Docker projects [2]. Maintainability concerns include instruction organization, naming conventions, documentation through comments, and structural consistency across projects [2,8].

## 2.5 Technical Foundations

### 2.5.1 Abstract Syntax Trees

An Abstract Syntax Tree (AST) is a tree representation of the syntactic structure of source code, where each node represents a construct occurring in the code. Unlike concrete syntax trees that preserve all syntactic details including punctuation and whitespace, ASTs abstract away these elements to focus on the structural relationships between code components. ASTs underpin a range of source code analysis techniques, including static analysis, code transformation, and fine-grained source code differencing [19]. In Dockerfile analysis, AST-based approaches parse instructions into hierarchical structures that capture the relationships between commands, arguments, and parameters. DRMiner introduces an Enhanced AST (E-AST) that captures structural complexities of instructions and stages specific to Dockerfiles [9].

### 2.5.2 Static Analysis

Static analysis refers to the examination of source code without executing the program, enabling the detection of potential issues, code patterns, and quality metrics through automated inspection [8,10]. Tools such as Hadolint parse the Dockerfile into an AST and perform rules on top of the AST to detect smells [8,17]. In contrast to dynamic analysis, which requires running the software, static analysis operates on the code artifact itself, making it suitable for integration into development workflows and continuous integration pipelines [10].

### 2.5.3 Mining Software Repositories

Mining Software Repositories (MSR) is a field of Software Engineering research that analyzes and cross-links data from version control systems, issue trackers, and other development artifacts to uncover information about software systems [20]. Vidoni presents a systematic process for MSR that includes repository selection, data extraction, and analysis [20]. MSR methodologies enable the extraction of Dockerfile version histories from Git repositories, allowing the identification of changes between commits, as demonstrated by Cito et al. who mined the entire population of Dockerfiles on GitHub [5].

### 2.5.4 Common Vulnerabilities and Exposures

Common Vulnerabilities and Exposures (CVE) is a standardized system for identifying and cataloging publicly known cybersecurity vulnerabilities, maintained by the MITRE Corporation [18]. Each CVE entry provides a unique identifier, description, and references to related security advisories [18]. In the context of Docker security assessment, CVE databases enable the identification of known vulnerabilities in base images and installed packages, providing quantifiable security indicators that can be measured before and after refactoring operations.

# 3 Literature Review

This chapter reviews the current state of knowledge on Dockerfile refactoring detection, classification, and impact assessment. It first describes the methodology followed for the systematic literature review, then analyses the state of the art in empirical Dockerfile quality research. It subsequently examines the limitations of existing Dockerfile smell catalogues and the tools available for Dockerfile quality analysis and refactoring, and concludes by positioning this dissertation with respect to the identified research gap.

## 3.1 Methodology

This chapter presents a systematic literature review examining the current state of knowledge regarding Dockerfile refactoring detection, classification, and impact assessment. The review follows the PRISMA methodology to ensure reproducibility and minimize selection bias.

### 3.1.1 Data Sources

The systematic search was conducted across the following academic databases, selected for their comprehensive coverage of software engineering and computer science literature:

| Database | URL | Coverage |
|---|---|---|
| IEEE Xplore | https://ieeexplore.ieee.org | Primary source for software engineering conferences and journals |
| ACM Digital Library | https://dl.acm.org | Comprehensive computing research repository |
| Springer Link | https://link.springer.com | Academic books and journals in computer science and engineering |

Table 3.1 - Summary of electronic databases and academic search engines utilized for the systematic literature search

In addition to the peer-reviewed studies sourced exclusively from the electronic databases in Table 3.1, the review incorporated a set of authoritative non-academic sources: official technical documentation from Docker [7,21], the established static-analysis tool Hadolint [10], the Docker-Parfum tool [22], the foundational refactoring work of Fowler [1], and an industry report from the Cloud Native Computing Foundation [4]. These sources did not originate from the automated database queries. They were obtained through targeted handsearching, a directed manual search aimed at specific, known artefacts required to characterise current practice and to ground the tool comparison, following the "identification of studies via other methods" arm of the PRISMA 2020 guidelines. This parallel arm ensures the review reflects practical, industry-current implementations alongside the empirical academic evidence, without compromising the rigour of the database search.

### 3.1.2 Search Strategy

Searches were conducted on July 11, 2026, and organized systematically. Search queries were formulated using Boolean operators and keywords derived from the research questions. The search terms were organized into three groups corresponding to each research question:

Keywords for RQ1 (State of the Art): Infrastructure as Code, Dockerfile refactoring, Software Evolution, Static Analysis, Empirical Software Engineering

Keywords for RQ2 (Catalogue Extension): Dockerfile refactoring, Performance Optimization, Security in Containers, Maintainability, DevOps, Containerization

Keywords for RQ3 (Prototype Effectiveness): Git Mining, Automation Tools, Build Efficiency, Software Quality Metrics, Security Analysis, Container Build Automation

Table 3.2 presents the exact Boolean queries executed in each database:

| **Database** | **Query Used** |
|---|---|
| **IEEE Xplore** | ("Dockerfile" AND ("refactoring" OR "smells")) OR ("Infrastructure as Code" AND "quality") OR ("Docker" AND "mining") OR ("Docker" AND "empirical study") |
| **ACM Digital Library** | ("Dockerfile" AND ("refactoring" OR "smells")) OR ("container" AND "technical debt") OR ("Docker" AND "reproducible research") |
| **Springer Link** | title:"Dockerfile" |

Table 3.2 - Academic databases and specific search queries used in the systematic literature review

### 3.1.3 Selection Criteria

Database Arm (Peer-Reviewed Studies):

| **Braço / Fonte** | **Tipo de Critério** | **Critério** |
|---|---|---|
| | **Inclusion** | Published from 2015 onwards. |
| | **Inclusion** | Written in English. |

| Braço / Fonte | Tipo de Critério | Critério |
|---|---|---|
| | **Inclusion** | Peer-reviewed articles on containerization, IaC, refactoring, or software quality context. |
| | **Exclusion** | Duplicate entries identified across multiple databases. |
| | **Exclusion** | Publications not available in full text. |
| | **Exclusion** | Sources lacking verifiable peer-review status or academic provenance. |
| | **Exclusion** | Non-English publications. |
| | **Exclusion** | Position papers, tutorials, or non-empirical content. |

Other-methods arm (targeted handsearching - supplementary authoritative sources):

| Criterion Type | Criterion |
|---|---|
| **Inclusion** | Official technical documentation from authoritative technology providers. |
| **Inclusion** | Established open-source tools with documented functionality. |
| **Inclusion** | Industry surveys or reports from recognized organizations. |
| **Inclusion** | Foundational academic books from recognized publishers. |
| **Exclusion** | Sources lacking recognized institutional or industry authority. |

Table 3.3 - Inclusion and exclusion criteria, separated by selection arm (systematic database search and targeted handsearching), following the two-arm PRISMA 2020 model.

### 3.1.4 Selection Flow (PRISMA Diagram)

The selection process followed the PRISMA flow, as summarized in Table 3.4:

| Phase / Stage | Description of Records | Quantity |
|---|---|---|
| **Identification** | Raw records identified through automated search in databases (IEEE: 241 \| ACM: 20 \| Springer: 12) | 273 |
| **Deduplication** | Duplicate records removed via cross-validation of DOIs and titles (R246 corrected) | -1 |
| **Screening (Phase 1)** | Records excluded by title/metadata analysis (155 without container/IaC context and 20 not peer-reviewed) | -175 |
| **Eligibility (Phase 2)** | Records excluded by metadata thematic eligibility (34 No Catalog, 25 IaC, 12 Security, 5 Orchestration, 5 Performance) | -81 |
| **Included (Databases)** | Studies included from the database flow (8 Technical Development + 8 Contextual Support) | 16 |
| **Additional Methods** | Industry authoritative sources and grey literature integrated via targeted manual search (Handsearching) | 5 |
| **FINAL TOTAL** | Absolute total of documents integrated into the qualitative synthesis and practical support of the review | 21 |

Table 3.4 - Quantitative summary of the systematic literature selection process, detailing the number of records retained across the Identification, Screening, Eligibility, and Included phases of the PRISMA methodology

The selection followed the two-arm PRISMA 2020 model: a database arm, which applies the systematic queries and academic exclusion criteria, and an "other methods" arm, which admits authoritative industry sources through targeted handsearching.

Database arm. Of the 273 records identified through database searching (241 from IEEE Xplore, 20 from the ACM Digital Library, and 12 from Springer Link), one was removed as a cross-database duplicate, leaving 272 unique records. A title- and metadata-level screening (Phase 1) excluded 175 records: 155 whose titles indicated no containerization, Infrastructure-as-Code, or Docker-quality context; 19 non-peer-reviewed items (commercial e-books, book chapters, and one standard); and one proceedings front-matter entry. The remaining 97 records underwent a topical eligibility assessment (Phase 2), which excluded a further 81: 34 carried no source-level quality, refactoring, smell, or foundational ecosystem-related term; 25

addressed competing Infrastructure-as-Code technologies such as Ansible, Terraform, Puppet, or Chef; 12 concerned image vulnerability scanning rather than source-level quality; 5 targeted cluster orchestration or CI/CD pipelines; and 5 measured runtime performance or resource consumption. This arm concluded with 16 included studies. The screening was conducted strictly at the title and metadata level, and the complete execution logs are available in the literature-review/ folder of the replication package.

Other-methods arm. In parallel, targeted handsearching identified five authoritative non-academic sources: the foundational refactoring reference (Fowler), an industry state-of-practice report (CNCF), the official Docker best-practices and multi-stage build documentation, and the Hadolint linter. As authoritative but non-database sources, they bypass the database screening funnel and enter the synthesis directly under the corresponding inclusion criteria of Table 3.3.

The database arm concluded with 16 included studies and the other-methods arm with 5 authoritative sources, giving a total of 21 documents carried into the synthesis, as recorded in Table 3.4. Of the 16 database studies, eight core studies, together with the five handsearched sources, directly ground the analysis presented in this chapter and are summarised in Table 3.5. The remaining eight database studies are retained in the replication package as eligible studies identified by the search; two of them inform specific design decisions in later chapters, while the others document smell categories outside the quantitative scope of the proposed engine. Listing the full set preserves the completeness and reproducibility of the review.

Figure 3.1 presents the complete PRISMA flow diagram summarising the selection process:

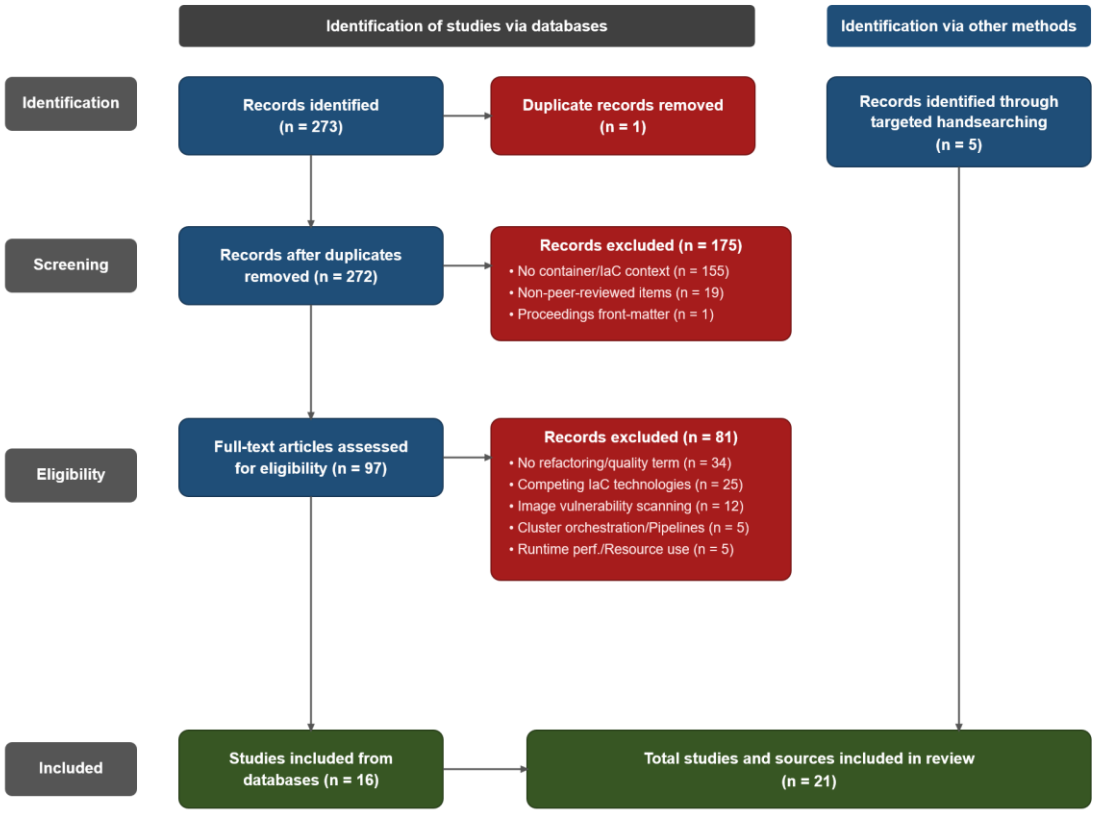


Figure 3.1 - PRISMA flow diagram of the systematic literature selection process.

The temporal boundary of 2015 was selected because Docker achieved widespread adoption around this period, and empirical studies on Dockerfile quality began to emerge. Boettiger [15] published foundational work on Docker for reproducible research in 2015, marking a significant milestone in academic recognition of container technologies.

The two-arm selection process retained 21 documents in total: 16 peer-reviewed studies obtained through the database search, together with 5 authoritative sources obtained through targeted handsearching (official documentation, established tools, a foundational reference, and an industry report).

Table 3.5 identifies the subset that directly grounds this chapter, the eight database studies drawn on in the analysis, together with the five handsearched sources, and summarises the specific role of each one; the remaining eight database studies are listed in the replication package. General methodological and framing references cited elsewhere in the dissertation fall outside this systematic scope and are therefore not listed here.

| **Ref.** | **Citation / Author** | **Type** | **Role in the Literature Review** |
|---|---|---|---|
| [1] | Fowler (2018) — *Refactoring: Improving the Design of Existing Code* | Foundational book | Establishes the canonical definition of refactoring as a behavior-preserving transformation. Obtained through targeted handsearching as a foundational reference, it provides the theoretical anchor for extending refactoring concepts to Dockerfiles across the chapter. |
| [2] | Ksontini et al. (2021) — *Refactorings and Technical Debt in Docker Projects: An Empirical Study* | Empirical study | Identifies the fourteen Dockerfile-specific refactoring types that form the catalogue extended in this dissertation. Used as the principal reference in Sections 4.3.2, 4.4, and 4.6 to ground the gap analysis and to position the proposed contribution against existing taxonomies. |
| [4] | CNCF — *Cloud Native 2024: Approaching a Decade of Code, Cloud, and Change* | Industry report | Industry-scale evidence on container adoption (more than 90% of organizations use or evaluate container technologies). Used in Section 4.1.1 to ground |

| Ref. | Citation / Author | Type | Role in the Literature Review |
|---|---|---|---|
| | | | the practical relevance of the review and to contextualise Docker adoption trends. |
| [5] | Cito et al. (2017) — *An Empirical Analysis of the Docker Container Ecosystem on GitHub* | Empirical study | Seminal large-scale empirical study analysing more than 70,000 Dockerfiles. Section 4.2.1 builds directly on this work to characterise the prevalence of quality issues across the Docker ecosystem. |
| [7] | Docker — *Best practices* | Official documentation | Reference source for Dockerfile authoring guidelines. Used throughout the chapter, particularly in Sections 4.1.1 and 4.5, to ground the discussion of refactoring patterns and their quality outcomes. |
| [8] | Wu et al. (2020) — *Characterizing the Occurrence of Dockerfile Smells in Open-Source Software* | Empirical study | Quantitative characterisation of smells in real-world Dockerfiles. Provides the basis for Section 4.2.2 and supplies the explicit link between Dockerfile smells and the three quality dimensions (maintainability, security, reproducibility) that frame the rest of the dissertation. |
| [9] | Ksontini et al. (2024) — *DRMiner: A Tool For Identifying And Analyzing Refactorings In Dockerfile* | Tool and empirical study | First tool dedicated to refactoring detection in Dockerfiles. Section 3.4.1 analyzes its technical approach and limitations, establishing the detection baseline that this dissertation extends. |
| [10] | Hadolint — *Haskell Dockerfile Linter* | Open-source tool | Widely-adopted static analysis tool for Dockerfiles. Section 3.4.2 examines its rule-based approach and limitations, positioning it as the second tool against which the |

| Ref. | Citation / Author | Type | Role in the Literature Review |
|---|---|---|---|
| | | | proposed prototype is compared in Table 4.6. |
| [15] | Boettiger (2015) — *An introduction to Docker for reproducible research* | Empirical study | Foundational academic work on Docker adoption for reproducible research. Used in Section 4.1.4 to justify the 2015 temporal boundary applied during study selection. |
| [17] | Rosa et al. (2024) — *Fixing Dockerfile smells: an empirical study* | Empirical study | Investigates the temporal persistence of Dockerfile smells across 53,456 unique Dockerfiles. Provides the empirical evidence for Section 4.2.3 on smell survivability and the observation that performance-related fixes are typically undocumented. |
| [21] | Multi-stage, Docker Documentation | Official documentation | Reference source for multi-stage build guidance. Obtained through targeted handsearching and used in Section 4.1.1 alongside [7] to ground the discussion of stage-based refactorings (Extract Stage / Inline Stage) |
| [22] | Durieux — *Docker-Parfum (Empirical Study of the Docker Smells Impact on the Image Size)* | Tool and Empirical study | Automated Dockerfile smell detection and repair tool, analyzed in Section 3.4.3 for its detect-and-repair approach based on AST rule matching. |
| [23] | Henkel et al. (2020) — *Learning from, Understanding, and Supporting DevOps Artifacts for Docker* | Empirical study | Empirical study of DevOps artifacts for Docker; referenced in Section 3.4.3 as the original source of several of the smell rules implemented by Docker-Parfum. |

Table 3.5 - Full set of references used in this chapter, the studies retained from the systematic review together with the supplementary sources (documentation, tools, books, and reports), with their type and specific role.

## 3.2 State of the Art Analysis

The empirical study of Dockerfile quality has revealed systematic patterns of problematic practices that accumulate over project lifecycles. Understanding these foundational studies is essential for contextualizing the contributions proposed in this dissertation.

### 3.2.1 Large-Scale Analysis of the Docker Ecosystem

Cito et al. [5] conducted a seminal empirical study analyzing the Docker container ecosystem on GitHub, examining 70,197 unique Dockerfiles originating from 38,079 GitHub projects. Their research revealed that Dockerfiles evolve more slowly than traditional source code, with an average of only 3.11 revisions per year. However, more popular projects demonstrated higher revision rates, averaging 5.81 revisions per year with approximately 5 lines changed per revision [5].

A critical finding from this study concerns build success rates: only 66% of Dockerfiles could be built successfully, with an average build time of 145.9 seconds for successful builds [5]. The study identified that most quality issues (28.6%) arise from missing version pinning, where developers fail to specify concrete versions for dependencies. This finding demonstrates that quality issues are not isolated incidents but systematic problems requiring structured solutions.

Cito et al. [5] also observed significant variation in base image sizes, noting that "alpine is only about 4 MB small, debian and ubuntu are both around 125 MB, and centos and fedora are around 195 MB." Despite these differences, they found that most projects use heavier base images, "probably due to the convenience of working with a known, established system," even though "that defeats the initial purpose of using containers to lower the footprint of virtualization" [5]. This observation has direct implications for performance-oriented refactorings.

### 3.2.2 Characterization of Dockerfile Smells

Wu et al. [8] extended the empirical foundation by characterizing the occurrence of Dockerfile smells across 6,334 open-source projects. Their research provides the most comprehensive categorization of Dockerfile quality issues to date, distinguishing between DL-smells (violations of official Dockerfile best practices) and SC-smells (violations of basic shell script practices).

Critically, Wu et al. [8] established the theoretical link between Dockerfile smells and multiple quality dimensions, stating that "those smells can be typical design issues that cause a

Dockerfile to fail when building images, or typical implementation issues that cause the building latency of a Dockerfile too long, even some architectural issues that may bring a Dockerfile the problem of maintainability, security, and reproducibility" [8]. This explicit connection between smells and the three quality dimensions of maintainability, security, and reproducibility provides the theoretical foundation for the quality-oriented classification proposed in this dissertation.

The quantitative findings are striking: nearly 84% of GitHub projects in their dataset have smells in their Dockerfile code, and on average, 62% of Dockerfile instructions are infected with smells [8]. Furthermore, they found that DL-smells appear more frequently than SC-smells, and that "a project that shows presence of large number of DL-smells moderately indicates presence of large number of SC-smells" [8], suggesting co-occurrence patterns that compound quality degradation.

Regarding specific problematic patterns, Wu et al. [8] highlighted that "using latest is prone to errors if the image will ever update," as this introduces non-deterministic behavior that affects reproducibility. They also found correlations between project characteristics and smell occurrence: "popular and young projects, or projects with large contributors team, tend to have fewer Dockerfile smells" [8].

### 3.2.3 Temporal Persistence of Dockerfile Smells

Rosa et al. [17] investigated the temporal dimension of smell persistence through an empirical study of 53,456 unique Dockerfiles. Their research revealed a critical pattern: most fixes are performed within 1 day (203 instances), but if a smell survives the first day, it is less likely to be corrected [17]. The median fix time for certain smell types reaches 597 days, demonstrating that quality issues can persist for extended periods without intervention.

This finding has significant implications for the research problem addressed in this dissertation. When developers perform refactorings to eliminate smells during project evolution, they rarely document these changes. Rosa et al. [17] found that only 18 out of 336 manually validated fixes were "informed" (explicitly documented). The most common undocumented fixes relate to performance improvements, with developers "paying more attention to changes aimed at improving the performance of Dockerfiles (image size and build time)" [17] without explicit acknowledgment.

## 3.3 Limitations of Existing Dockerfile Smell Catalogues

While the empirical studies reviewed above demonstrate the prevalence of quality issues in Dockerfiles, the catalogues used to classify these issues present significant limitations that hinder their practical applicability.

### 3.3.1 Syntactic Focus Without Quality Dimension Mapping

Current Dockerfile smell catalogues, including those operationalized in tools like Hadolint [10], primarily describe smells in syntactic or operational terms without systematic mapping to quality dimensions. For example, the rule DL3008 ("Pin versions in apt-get") is described procedurally but does not explicitly classify the smell according to whether it primarily affects security (through unpredictable dependency versions), reproducibility (through non-deterministic builds), or maintainability (through difficulty tracking changes).

As Wu et al. [8] established, Dockerfile smells can bring problems of "maintainability, security, and reproducibility." However, existing catalogues do not provide practitioners with guidance on which dimension a particular smell primarily affects, making it difficult to prioritize remediation efforts based on quality goals. A development team prioritizing security may need to address different smells than one prioritizing build performance, but current catalogues do not support such differentiated decision-making.

### 3.3.2 Lack of Empirical Impact Assessment on Quality Dimensions

A critical gap in existing catalogues is the absence of a systematic classification of refactorings based on their impact on specific quality dimensions. While Docker Best Practices provide general recommendations and Ksontini et al. [2] proposed 14 Dockerfile-related refactoring types , these resources focus primarily on the mechanical detection of changes.

Ksontini et al. [2] identified technical debt categories such as Image Size and Maintainability , but their taxonomy does not quantitatively link each refactoring type to measurable quality improvements (deltas) across the dimensions of performance, security, and maintainability. Consequently, practitioners lack empirical evidence to determine if a specific refactoring actually results in a smaller image or a more secure container

### 3.3.3 Limited Coverage of Security and Reproducibility Metrics

Existing catalogues demonstrate particularly weak coverage of security and reproducibility dimensions. While performance-related smells (affecting build time and image size) are well-documented, security-related issues such as running containers with elevated privileges or including unnecessary attack surface are less systematically catalogued.

Wu et al. [8] noted that smells can bring "the problem of maintainability, security, and reproducibility," yet their catalogue, like others, does not quantify the expected security impact of each smell type. For instance, while DL3007 ("avoid using latest tag") clearly affects reproducibility and potentially security, current catalogues do not provide guidance on the relative severity of this smell compared to others in terms of CVE exposure or build determinism.

## 3.4 Existing Tools for Dockerfile Quality Analysis and Refactoring

This section examines the three existing tools most relevant to the goals of this dissertation: DRMiner [9], the first dedicated Dockerfile refactoring detection tool; Hadolint [10], a widely adopted static analysis tool for Dockerfile smell identification; and Docker-Parfum [22], an automated refactoring application tool that rewrites Dockerfiles by applying a predefined set of transformations. The three tools address different aspects of Dockerfile quality. DRMiner identifies refactoring operations across version history, Hadolint flags smells in individual Dockerfile snapshots, and Docker-Parfum repairs smells automatically to produce optimised versions. All three serve as baselines for the comparative analysis in Section 3.4.4, each occupying a different point in the space the proposed prototype addresses: detection across revisions, smell identification within a single snapshot, and automated repair.

### 3.4.1 DRMiner: Dockerfile Refactoring Detection

DRMiner, introduced by Ksontini et al. [9], represents the first tool specifically designed to identify and analyze refactorings in Dockerfiles. The tool addresses a fundamental challenge: "developers usually avoid documenting their refactoring efforts, often combining them with other changes" [9]. This lack of documentation makes it difficult to understand how Dockerfiles evolve and whether changes constitute intentional quality improvements.

#### Technical Approach

DRMiner employs an Enhanced Abstract Syntax Tree (E-AST) based component-matching algorithm to detect refactoring operations. The E-AST extends traditional AST representations to capture Dockerfile-specific semantics, including "comprehensive flag representation, multi-stage build differentiation, and order preservation" [9]. The tool constructs E-AST models for two Dockerfile revisions linked to a commit, then matches components within these models using a bottom-up approach that starts from matching instructions and proceeds to composite instructions forming a stage.

The detection process applies preprocessing techniques including Shell-Stripping (simplifying shell scripts within RUN instructions) and Unification (replacing variable names with default values) to handle syntactic variations. The tool then applies detection rules derived from the 14 refactoring definitions established by Ksontini et al. [2], achieving a precision of 95.1% and recall of 97% in detecting 12 types of refactoring operations [9].

#### Detected Refactoring Types

Ksontini et al. [2] identified fourteen Dockerfile-specific refactoring types through manual analysis of open-source Docker projects. Building on these definitions, DRMiner [9] subsequently implemented automated detection rules for twelve of these types: Extract

Stage, Inline Stage, Move Stage, Sort Instructions, Replace ADD with COPY, Extract RUN Instructions, Inline RUN Instructions, Update Base Image, Update Base Image TAG, Rename Image, Add ENV Variable, and Add ARG Instruction. These types cover different components (instructions and stages) and control flows within Dockerfiles.

The fourteen refactoring types identified by Ksontini et al. [2] are summarised in Table 3.6. Each type describes a structural transformation applied to a Dockerfile while preserving the functional equivalence of the resulting image. These definitions form the basis on which this dissertation builds its contribution in Chapter 4.

| ID | Refactoring Type | Description |
|---|---|---|
| **R01** | **Inline RUN Instructions** | Consolidates multiple RUN instructions into one, reducing layer count and image size. |
| **R02** | **Update Base Image TAG** | Replaces a mutable, outdated, or unspecified base image tag with a specific or updated version tag, ensuring reproducibility and alignment with security patches. |
| **R03** | **Add ENV Variable** | Stores system-wide values as environment variables, avoiding hardcoded values and improving configurability. |
| **R04** | **Add ARG Instruction** | Uses ARG to define dynamic build-time identifiers (e.g., version, paths), avoiding duplication. |
| **R05** | **Extract Stage** | Introduces multi-stage builds to separate build dependencies from the runtime environment. |
| **R06** | **Inline Stage** | Removes an unnecessary build stage when multi-staging is not justified. |
| **R07** | **Sort Instructions** | Reorders instructions from least to most frequently changing to optimise layer cache reuse. |
| **R08** | **Replace ADD with COPY** | Replaces ADD with COPY where the implicit behaviors of ADD (URL fetching, tar extraction) are not needed. |

| ID | Refactoring Type | Description |
|---|---|---|
| **R09** | **Extract RUN Instructions** | Moves complex commands from a RUN instruction into a script file, reducing Dockerfile clutter. |
| **R10** | **Update Base Image** | Replaces a base image with a different base image (e.g., Alpine). |
| **R11** | **Move Stage** | Extracts a stage from a multi-stage context into a separate Dockerfile. |
| **R12** | **Remove RUN (mv command)** | Removes RUN instructions containing mv and uses COPY/ADD to set the correct path directly, reducing layers. |
| **R13** | **Update RUN Instruction** | Reformats RUN commands (splitting lines, ordering arguments) to improve readability and structure. |
| **R14** | **Rename Image** | Replaces or assigns a stage alias in a multi-stage build to replace numeric or legacy references with readable identifiers, improving traceability and understandability. |

Table 3.6 - The fourteen Dockerfile refactoring types identified by Ksontini et al. [2]

**Critical Limitations**

Despite its detection capabilities, DRMiner presents three significant limitations that motivate this dissertation:

First, DRMiner focuses exclusively on detection without impact assessment. The tool identifies refactorings but does not evaluate their effects on quality metrics. Ksontini et al. [9] acknowledge that "evaluating the impact of refactorings on these metrics is key to deriving actionable insights that can optimize current configurations and guide future refactoring strategies," but this functionality is not implemented. Practitioners using DRMiner can identify that a refactoring occurred, but cannot determine whether it improved or degraded image size, code warnings, or security posture

Second, DRMiner lacks quality dimension classification. The detected refactorings are not categorized according to their impact on performance, security, or maintainability. While

Ksontini et al. [2] identified technical debt categories, DRMiner does not map detected refactorings to these categories, limiting its utility for quality-oriented decision-making.

Third, DRMiner has limited public availability. While a replication package is mentioned [9], the tool's accessibility for the broader research community remains constrained, limiting opportunities for validation and extension.

### 3.4.2 Hadolint: Static Analysis for Dockerfiles

Hadolint [10] is a widely-adopted static analysis tool for Dockerfiles that validates inline bash commands and enforces best practices. As described by Wu et al. [8], "this tool parses the Dockerfile into an AST and performs rules on top of the AST" to detect violations of predefined best practice rules. Hadolint also integrates ShellCheck to lint bash code within RUN instructions, detecting both DL-smells and SC-smells.

#### Technical Approach

Hadolint operates on individual Dockerfile snapshots, parsing each file into an AST and applying a predefined set of rules that encode Docker best practices. Rules are categorized with prefixes: "DL" for Dockerfile-specific violations and "SC" for shell-script violations inherited from ShellCheck. Each rule violation produces a warning with severity level and remediation guidance.

#### Critical Limitations

Hadolint presents distinct limitations from DRMiner:

First, Hadolint performs single-snapshot analysis. The tool operates on individual Dockerfile versions rather than analyzing evolutionary changes across version histories. This prevents detection of refactoring operations that occur between commits, as refactorings are inherently comparative operations requiring analysis of before-and-after states.

Second, Hadolint lacks refactoring detection capabilities. While the tool effectively identifies smell violations against predefined rules, it does not detect when developers have performed refactoring operations to address these smells. A developer who consolidates RUN instructions to reduce layers would not have this refactoring identified by Hadolint.

Third, Hadolint provides no impact measurement. The tool does not correlate identified smells with measurable quality metrics. While it may flag that a Dockerfile uses the "latest" tag (DL3007), it cannot quantify the security or reproducibility impact of this violation in terms of CVE exposure or build determinism.

### 3.4.3 Docker-Parfum: Automated Dockerfile Smell Detection and Repair

Docker-Parfum [22] is an automated Dockerfile smell detection and repair tool developed by Durieux. Unlike DRMiner [9], which identifies refactorings across version histories, and Hadolint [10], which only detects smells without resolving them, Docker-Parfum performs both detection and automatic repair: given an input Dockerfile, it identifies smell violations and produces a transformed output Dockerfile in which the detected violations have been resolved through targeted rewrites of the offending instructions.

The tool operates on the Abstract Syntax Tree of the Dockerfile, which allows each detected smell to be linked to the node that produced it and each repair to be applied as a transformation of that node.

**Technical Approach**

Docker-Parfum is built on top of the Dinghy AST parser and operates through a rule-matching architecture. Each rule encapsulates two components: a detection predicate that traverses the parsed AST to locate violations of a given smell, and a repair function that rewrites the matched AST nodes to produce a compliant version of the instruction. Once all applicable rules have been executed, the modified AST is serialised back into a valid Dockerfile.

The tool natively supports the detection and automatic repair of 32 smell types, several of which were originally identified by Henkel et al. [23] in their empirical study of DevOps artefacts for Docker. The published study, however, restricted its evaluation to the 14 smells related to space waste, on the grounds that these are the ones with a measurable effect on image size; smells concerning security or build reliability were deliberately excluded from that analysis [22]. The fourteen evaluated rules concern package management and cache handling, among them aptGetInstallUseNoRec, which enforces the --no-install-recommends flag on apt-get install; aptGetInstallRmAptLists, which removes /var/lib/apt/lists/* after package installation; and the cache-cleaning rules pipUseCacheDir, npmCacheCleanAfterInstall and yarnCacheCleanAfterInstall. Its effectiveness was evaluated at scale, with Docker-Parfum repairing smells in the 338,211 Dockerfiles drawn from its own dataset and from Binnacle's, of which 164,597 contained at least one smell, and being compared against Binnacle on a manually annotated ground-truth set of 384 Dockerfiles. The detection-plus-repair pipeline is deterministic: the same input Dockerfile always produces the same output Dockerfile, since each rule applies a fixed AST transformation when its detection predicate matches.

**Critical Limitations**

Docker-Parfum presents three limitations that motivate the need for the prototype proposed in this dissertation.

First, Docker-Parfum quantifies the impact of its repairs only in terms of image size. It does not measure the maintainability or security dimensions, such as Hadolint warning counts or vulnerability counts, so it cannot indicate whether a transformation improves or degrades those dimensions. The proposed prototype instead reports all three quality dimensions for each detected refactoring, with maintainability covered by two complementary indicators.

Second, Docker-Parfum does not classify the transformations it applies according to the quality dimensions they primarily target. Its 32 rules encode a fixed mapping of input patterns

to output patterns, but the rationale of each transformation, whether it primarily addresses performance, security, or maintainability, is not explicit in its output.

Third, the rule set is fixed and tied to specific smell patterns. Docker-Parfum does not detect refactorings that fall outside its predefined rule catalogue, and it does not analyze evolutionary refactoring patterns across commit histories, which restricts its applicability for empirical studies of how Dockerfiles evolve over time.

These gaps are precisely what the extended catalogue and the impact assessment pipeline of the proposed prototype seek to address: the catalogue supplies the missing quality-dimension organization, and the pipeline supplies the missing quantification.

### 3.4.4 Comparative Analysis and Research Gap

Table 3.7 presents a comparative analysis of existing tools and the proposed prototype, highlighting the capabilities and gaps that motivate this research.

| **Capability** | **Hadolint** | **DRMiner** | **Docker-Parfum** | **Proposed Tool** |
|---|---|---|---|---|
| **Change Detection (Commit A → B)** | No | Yes | No | Yes |
| **History Analysis (Git)** | No | Yes | No | Yes |
| Multi-Dimension Impact Assessment | No | No | No | Yes (3 Dimensions) |
| **Empirical Assessment (Metrics)** | No | No | No | Yes (Real Impact) |
| **Automatic Repair** | No | No | Yes | No |
| **Primary Focus** | Static Smells | Mechanical Action | Smell Repair | Impact of Evolution |

Table 3.7 - Capability comparison of existing Dockerfile analysis tools and the proposed prototype.

Docker-Parfum is included in Table 3.7 because it possesses a genuine detection engine, thirty-two rules that locate smell violations in the parsed AST before any repair is attempted [22], which places it alongside the other analysis tools rather than outside them. Its position in the table is nonetheless distinct in two respects. Its detection operates on individual Dockerfile snapshots and not across a commit history, so it shares Hadolint's inability to detect change between two revisions rather than DRMiner's capability. And its empirical assessment is confined to image size, the single dimension it reports, whereas the proposed tool reports three. What sets Docker-Parfum apart from every other tool in the comparison is the Automatic Repair row: it is the only one that rewrites the Dockerfile to resolve the smells it detects. Its primary focus is therefore repair, which is what distinguishes it from the detection-oriented tools alongside which it appears in the comparison.

The comparative analysis reveals two fundamental gaps that collectively motivate this dissertation:

**Gap 1:** Absence of a Structured Refactoring Catalogue with Quality Dimension Classification. While Docker Best Practices [7] provide guidelines and smell characterizations, no comprehensive catalogue exists that systematically organizes Dockerfile refactorings according to their impact on key quality dimensions such as performance, security, and maintainability [7,8,17]. Existing resources describe smells syntactically without explicit mapping to quality outcomes, leaving practitioners without evidence-based guidance for prioritizing refactoring efforts.

**Gap 2:** Detection and Observation Without Quantified Impact Assessment. Existing tools and studies identify or observe Dockerfile refactorings and smells, but none quantifies their impact on measurable quality metrics. On the tooling side, DRMiner [9] detects refactorings with high precision (95.1% precision, 97% recall) but focuses exclusively on identification, while Hadolint [10] detects smells without evolutionary analysis; neither determines whether a refactoring improved or degraded quality. On the empirical side, Wu et al. [8] limited their analysis to the latest version of each Dockerfile rather than tracking how metrics evolve across refactoring events, and Rosa et al. [17] studied whether smells get fixed but not the quantified effect of those fixes on image size, CVE count. As Ksontini et al. [2] acknowledge, "evaluating the impact of refactorings on these metrics is key to deriving actionable insights," yet this functionality remains unimplemented. As a result, neither tools nor prior studies allow practitioners to make evidence-based refactoring decisions grounded in measured impact.

## 3.5 Positioning of This Dissertation

This dissertation addresses the identified gaps through two complementary contributions that extend beyond the capabilities of existing tools:

**Contribution 1:** The complete catalogue classifying all fourteen types is presented in Section 4.2. All fourteen types are adopted for automated detection in the prototype tool, as

described in Section 4.4.3. This dissertation extends that catalogue by classifying each type across three independent quality dimensions, performance, security, and maintainability, each one evaluated on its own terms, enabling practitioners to make evidence-based refactoring decisions.

**Contribution 2:** Prototype Tool with Empirical Impact Assessment. Unlike DRMiner [9], which focuses solely on detection, the proposed tool combines automated refactoring detection with empirical impact assessment. The tool measures quality metrics, specifically image size and CVE count, before and after detected refactoring operations, addressing the limitation acknowledged by Ksontini et al. [9] that impact evaluation "is key to deriving actionable insights." Build time was considered but excluded from the prototype due to its high sensitivity to environmental factors, as discussed in Section 4.5.1. By correlating detected refactorings with measurable changes in quality metrics, the tool enables evidence-based refactoring decisions that are not possible with current approaches.

Together, these contributions provide practitioners and researchers with the structured knowledge and automated tooling necessary to understand, detect, and evaluate Dockerfile refactorings in a systematic and empirically validated manner. The work directly addresses the observation by Ksontini et al. [2] that "there is a gap of knowledge in how container-based projects actually evolve and are maintained" by providing mechanisms to track and quantify that evolution.

The two gaps identified in this chapter, the absence of a quality-dimension classification for existing refactoring catalogues, and the lack of quantified impact assessment in existing tools and studies, directly motivate the design decisions presented in Chapter 4.

# 4 Design

This chapter presents the design of the prototype tool proposed in this dissertation. Throughout this chapter, the term impact is used to refer to the measurable effects of refactorings on quality metrics, specifically ΔSize, ΔCVEs, ΔWarnings, and ΔInstr, consistent with the title of this dissertation, Dockerfile Refactorings: Detection and Effects. It describes the planned architecture, its design objectives, and the justification for each decision made.

The chapter is grounded in the five rapid experiments conducted prior to this phase, which empirically validated the feasibility of each system component before its inclusion in the design. This approach is consistent with the Design and Development activity of the Design Science Research (DSR) methodology [11] adopted in this dissertation.

Table 4.1 summarises the five rapid experiments, mapping each one to its objective, the method used, the observed result, and the system component whose design it grounds. The Dockerfile pairs, measurement scripts, and raw outputs of each experiment are kept in a dedicated /experiments/ folder of the project's public GitHub repository (https://github.com/DiogoVieira123/diogov_Dockerfile_refactorings_detection_and_effects/tree/main/experiments), so that every result reported below can be inspected and reproduced.

| # | Objective | Method | Result | Validated Component |
|---|---|---|---|---|
| **Exp. 1** | Validate the three-axis measurement principle by applying four supported refactorings in isolation and confirming that the four deltas (Delta Size, Delta CVEs, Delta Warnings, Delta Instr) capture distinct impact profiles, where some refactorings affect more than one dimension and others | Apply each of four supported refactorings, Inline RUN Instructions (R01), Update Base Image TAG (R02), Update Base Image (R10), and Replace ADD with COPY (R08) , in isolation to a minimal Dockerfile pair, build both images, and measure the four deltas for each pair. | The four refactorings produced distinct impact profiles, measured with Hadolint unfiltered. Inline RUN Instructions (R01) reduced image size by 194,991 bytes and the logical instruction count by three, while the raw warning total fell from 10 to 3: DL3059 (3 → 0), the smell the refactoring targets, together with DL3008 (3 → 1) and DL3015 (3 → 1), which fall because four apt-get invocations became one. Update Base Image TAG (R02) eliminated the DL3007 latest-tag smell, taking the raw total from 1 to 0. Update Base Image | The four-metric measurement principle. |

| # | Objective | Method | Result | Validated Component |
|---|---|---|---|---|
| | affect only one. | | (R10, ubuntu:22.04 to alpine:3.19) reduced image size by approximately 26.3 MB and removed 15 CVEs, improving both performance and security, with no warning in either state. Replace ADD with COPY (R08) eliminated a single DL3020 warning with a minor size increase of 3 bytes and no security impact. No refactoring degraded a dimension it was not intended to affect, so the trade-off flag was not triggered by this set. | |
| **Exp. 2** | Validate the JSON output schemas of Hadolint and Trivy as parseable inputs for the Data Extractor. | Run both tools with --format json against PoC images and inspect their structures programmatically. | Hadolint returns a flat array (code, level, line, message). Trivy returns a nested Results[].Vulnerabilities[] with VulnerabilityID and Severity. Both schemas confirmed stable and programmatically parseable. | Data Extractor |
| **Exp. 3** | Validate programmati | Use GitPython against the | Two distinct Dockerfile versions retrieved as | VCS Connector |

| # | Objective | Method | Result | Validated Component |
|---|---|---|---|---|
| | c access to Dockerfiles directly from the Git object model, without a working tree checkout. | public repository docker/getting-started, retrieving the Dockerfile at commits 2bca273 and 2981665. | byte streams in fewer than ten lines of Python code. | |
| **Exp. 4** | Validate precise image size retrieval through the Docker daemon API rather than CLI text parsing. | Build a Dockerfile image and read image.attrs['Size'] via the Docker SDK for Python. | Returned 29,748,045 bytes (28.37 MB) as a raw integer, eliminating CLI rounding imprecision. | Performanc e Analyzer |
| **Exp. 5** | Validate logical-instruction parsing of Dockerfiles, including multi-line RUN reconstructio n. | Parse a Dockerfile pair with dockerfile-parse and compare the resulting logical instruction lists. | Four logical RUN instructions correctly identified in version A and one in version B; the 4-to-1 consolidation detected automatically. | Detection Engine |

Table 4.1 - Summary of the five rapid experiments, their objectives, methods, results, and the system components whose design they ground.

The chapter is organized as follows. Section 4.1 provides the technical description of the system, the paradigm shift it introduces, the engineering challenges it must resolve, the

functional and non-functional requirements derived from the experimental findings, the architecture design, and the verification and validation strategy. Section 4.2 presents the extended refactoring catalogue. Sections 4.3 through 4.7 describe each of the five system components in detail, justifying the technical decisions made. Section 4.8 presents the complete system data flow. Section 4.9 documents the known design limitations and planned mitigations. Section 4.10 closes with a chapter summary.

## 4.1 Prototype Tool

### 4.1.1 Technical Description

The prototype tool is a Python-based analysis system that receives as input three mandatory positional arguments (a Git repository path, a before commit identifier, and an after commit identifier) alongside optional command-line options for the output directory (-o), the evaluated Dockerfile path (-d), and an explicit build context directory (-c). It extracts the Dockerfile at each commit from the version history, detects whether a structural refactoring occurred between them, identifies which of the fourteen catalogued types it corresponds to, and produces an impact report quantifying the change across four metrics: ΔSize, ΔCVEs, ΔWarnings, and ΔInstr. The extended catalogue of Section 4.2 is a conceptual taxonomy: it names and organizes the refactoring types and records, from the literature and from controlled experiments, the quality directions each one is expected to produce. The prototype does not consult that taxonomy at run time. It detects the transformation, reports its formal identifier, and measures what the two Dockerfile states actually produce, without any expectation of the outcome being encoded in the software. The catalogue is therefore the reference against which results are read, not an input to the computation.

The selection of Python 3 as the implementation language was determined by the rapid experimentation phase: all five experiments were conducted in Python, and the libraries validated in those experiments, GitPython, dockerfile-parse, and the Docker SDK, are all Python-native. This eliminates any interoperability risk that would arise from selecting a different language and ensures that the prototype can be built directly on top of the validated experimental scripts.

The interaction with the Docker daemon is handled exclusively through the Docker SDK for Python [Exp. 4], which communicates directly with the daemon via the Unix socket and returns image sizes as raw byte integers. This is architecturally preferable to parsing the output of the Docker CLI, which rounds values and introduces imprecision. Experiment 4 confirmed that this call returns an exact byte count, 29,748,045 bytes (28.37 MB) for the documented public test image (ubuntu:22.04) used in that experiment, a precision level that

would be rounded away by the CLI. This precision is essential because the Performance Analyzer computes the size delta by subtracting two such byte counts, and CLI rounding would distort small deltas.

Version history access is handled by GitPython [Exp. 3], which navigates the Git object model natively, retrieving the Dockerfile content at any commit as a byte stream without requiring a working tree checkout or filesystem write operations. Experiment 3 validated this approach against the public repository docker/getting-started (commits 2bca273 and 2981665), confirming that two distinct Dockerfile versions can be retrieved correctly with fewer than ten lines of Python code.

Dockerfile parsing is handled by the dockerfile-parse library [Exp. 5], which reconstructs multi-line RUN instructions, those split across physical lines using the backslash continuation character, into their logical single-unit representation. This reconstruction is critical for the Detection Engine: without it, a single logical command spread over eight lines would be miscounted as eight instructions, producing false detection results. Experiment 5 confirmed that the library correctly identifies four logical RUN instructions in a Dockerfile where they span multiple physical lines, and one logical instruction in the consolidated version.

Security and Maintainability metrics are collected using Trivy and Hadolint [10] respectively, both of which were validated in Experiment 2 with JSON-formatted output. Trivy scans the built Docker image for known CVEs, producing a structured results array with vulnerability identifiers and severity levels. Hadolint performs static analysis on the Dockerfile text, producing a flat array of rule violation objects. Experiment 2 confirmed that both schemas are stable and programmatically parseable, providing the foundation for the Data Extractor component.

### 4.1.2 Planned Changes in the System

The current state of Dockerfile maintenance is characterised by the absence of systematic feedback. When a developer applies a refactoring, whether consolidating RUN instructions, updating a base image, or adding cache cleanup, the impact of that change on image size, vulnerability exposure, and structural quality is never actually measured, since decisions are typically based on general knowledge of best practices rather than on empirical evidence. This lack of systematic verification means that improvements are assumed rather than confirmed, leaving the real effects of the applied refactoring unknown.

The prototype tool addresses this gap by introducing a systematic measurement cycle, replacing this opacity with quantifiable evidence of the actual impact produced. For each pair of commits where a refactoring is detected, the system computes four delta values across the three quality dimensions: ΔSize (the change in image size in bytes and as a percentage), ΔCVEs (the change in vulnerability count by severity level), ΔWarnings (the change in Hadolint rule violations classified by quality dimension), and ΔInstr (the change in the number of logical instructions). Maintainability is the only dimension measured by two indicators, for the

reasons set out in Section 4.1.3: ΔWarnings reports the change at the smell level, while ΔInstr reports it at the structural level. Together, these values constitute the empirical evidence base for what a refactoring actually achieved.

Four isolated proof-of-concept experiments were conducted prior to this design phase to establish empirical baselines for each delta metric. Each experiment applied a single refactoring to a minimal Dockerfile pair, ensuring that observed deltas are attributable to the refactoring under test and not to confounding changes. All four were executed in a single run on 17 July 2026, using Docker 29.1.3, Hadolint 2.14.0, Trivy 0.72.0, and dockerfile-parse 2.0.1.

The purpose of this set is to show that a refactoring moves the quality dimensions it is expected to move, in a real build rather than in principle. The absolute values reported below are the measurements obtained in that run, and they are given so that each delta can be traced back to the pair it came from. They are not catalogue reference values: registries republish tags and Trivy updates its vulnerability database daily, so a later run may return different absolute figures without invalidating anything reported here. What the proof-of-concept set establishes is the practical viability of the measurement approach and the direction of each effect, and both hold independently of the run. Producing values that are comparable across refactoring types is the separate purpose of the controlled catalogue experiments of Section 4.2, which are conducted under their own conditions and are not derived from this set.

PoC-1 - Inline RUN Instructions (R01): The consolidation of four distinct RUN commands into a single chained command resulted in a Delta Size of −194,991 bytes (117,254,883 → 117,059,892), a Delta CVEs of 0 (26 → 26, unchanged at 12 MEDIUM and 14 LOW), and a Delta Instr of −3 (5 → 2). For the proof-of-concept set Hadolint runs without any rule filter, so the reported figure is ΔWarnings (total raw): it fell from 10 to 3, a difference of −7. The reduction decomposes into DL3059 (3 → 0), the consecutive-RUN smell the refactoring directly targets, together with DL3008 (3 → 1) and DL3015 (3 → 1), which fall because the four apt-get invocations that each violated those rules became one; DL3009 was unchanged at 1. This total aggregates rules across quality dimensions and is reported to demonstrate detection sensitivity, not as the filtered maintainability indicator defined in Section 4.1.3.

PoC-2 - Update Base Image TAG (R02): Replacing the mutable ubuntu:latest tag with the explicit ubuntu:22.04 tag produced a Delta Size of −11,845,577 bytes (41,593,622 → 29,748,045), a Delta CVEs of −26 (45 → 19), a Delta Instr of 0 (1 → 1), and a ΔWarnings (total raw) of −1, falling from 1 to 0 and accounted for entirely by the elimination of DL3007, effectively removing the "latest-tag" anti-pattern. The severity distribution improves alongside the total: the before state carries 5 HIGH, 34 MEDIUM, 4 LOW and 2 UNKNOWN, while the after state carries 9 MEDIUM and 10 LOW, so every HIGH severity vulnerability is removed. At execution time the ubuntu:latest tag was resolved to the explicit manifest it pointed to, and the before figures describe that state; because the tag is mutable, a later run may resolve to a different manifest and return different absolute values. The elimination of DL3007 is invariant across runs, and the direction of the size and CVE deltas follows from the substitution itself rather than from the particular version the tag happened to resolve to.

PoC-3 - Update Base Image (R10): Replacing the ubuntu:22.04 base image with the minimal alpine:3.19 image produced a Delta Size of −26,318,550 bytes (29,748,045 → 3,429,495), a Delta CVEs of −15 (19 → 4), and a Delta Instr of 0 (1 → 1). Neither state produced any Hadolint warning under the unfiltered count, so ΔWarnings (total raw) is 0: this refactoring acts on the base image rather than on any pattern Hadolint inspects. This is the most impactful refactoring in the set, showing that the choice of base image moves performance and security at the same time. One qualification applies to the security figure. The total falls from 19 to 4, but the before state carries no HIGH severity vulnerability (9 MEDIUM, 10 LOW) while the after state carries one (1 HIGH, 2 MEDIUM, 1 LOW), so the severity distribution changes as well as the total, and a smaller image is not automatically a safer one. Because Trivy updates its vulnerability database daily, the absolute counts belong to the run reported here; what the experiment establishes is the direction and the scale of the change.

PoC-4 — Replace ADD with COPY (R08): Working on the same alpine:3.19 base used in PoC-3, replacing an ADD instruction that copies a regular file with the equivalent COPY instruction produced a Delta Size of +3 bytes (3,421,509 → 3,421,512), a Delta CVEs of 0 (4 → 4, unchanged at 1 HIGH, 2 MEDIUM and 1 LOW, the profile inherited from that base), a Delta Instr of 0 (2 → 2), and a ΔWarnings (total raw) of −1, falling from 1 to 0 and accounted for entirely by the elimination of DL3020. The change is purely structural, eliminating the implicit-extraction risk associated with ADD without affecting image size or vulnerability exposure.

The prototype automates the entire measurement process, transforming what required manual invocation of three separate tools into a single programmatic call.

### 4.1.3 Metric Selection and Scope

The three quality dimensions are operationalised through metrics that can be computed automatic ally for any Dockerfile pair. Performance is measured by image size (ΔSize), retrieved in bytes through the Docker SDK. Security is measured by the CVE count reported by Trivy (ΔCVEs). Maintainability is measured by two indicators rather than one: the Hadolint warning count (ΔWarnings) and the logical instruction count (ΔInstr). Throughout, ΔWarnings refers to this filtered count. Where a broader count of all Hadolint rules is reported, it is used as context to show that a smell was eliminated, and never as the metric itself.

Two warning-counting regimes are used in this work, and the distinction is deliberate rather than incidental. The extended catalogue experiments apply a maintainability screen of 47 rules: those enabled by default in Hadolint and ShellCheck whose effect is on reproducibility, structural correctness, instruction usage and notation, metadata, shell correctness, or build logs. Rules whose effect is on image size, such as cache cleanup or RUN consolidation, are excluded because that effect is already measured by ΔSize; rules whose effect is on attack surface are excluded for the same reason with respect to ΔCVEs. Without this screen, a single refactoring that reduces image size would be counted twice, once in ΔSize and again in ΔWarnings, and the dimensions would cease to be independent.

The four proof-of-concept experiments of Section 4.1.2 are the single exception. There Hadolint runs unfiltered and the raw total is reported, because their purpose is different: they are exploratory sensitivity checks establishing that the pipeline detects warning changes at all

and attributes them to the correct rules, before any dimensional interpretation is placed on the count. Proof-of-concept warning totals are labelled "total raw" wherever they appear and are never used as the maintainability indicator. The consequence is visible in PoC-1, where the raw total falls by seven but only two of those eliminations, both of DL3008, belong to the maintainability screen; the remaining five belong to rules whose effect is already captured by ΔSize. Reporting the raw total there measures the reactivity of the instrument, not the maintainability of the Dockerfile.

The second maintainability indicator exists because the correspondence between dimensions and metrics is not symmetric. ΔSize and ΔCVEs are direct measurements of the built image: each reads a property of the artifact the Dockerfile produces. ΔWarnings works differently. It counts violations of a predefined rule set, so it only reacts when a refactoring happens to match a rule that Hadolint implements. However, many structural improvements, such as simplifying the Dockerfile logic, do not necessarily trigger a reduction in Hadolint warnings. For these cases, ΔWarnings stays at zero, not because the refactoring failed to improve maintainability, but because the tool's static analysis is not designed to measure structural complexity. Since maintainability is the dimension most often affected across the catalogue (Table 4.3), relying on ΔWarnings alone would leave the most frequently affected axis largely unmeasured.

ΔInstr addresses this at a different level. Hadolint reports smells; the instruction count reports structure. Wu et al. [8] use the number of instructions in a Dockerfile as a proxy for its complexity, treating it as an independent variable in their regression models of smell occurrence. The same reasoning applies here: a refactoring that removes a redundant stage, or that eliminates instructions whose only purpose was to move files, leaves a structurally simpler Dockerfile, and that simplification shows up in the instruction count even when no Hadolint rule fires.

Three properties made ΔInstr preferable to the alternative considered. It requires no external tool: the count is obtained by parsing each Dockerfile state with dockerfile-parse, a Python library already part of the prototype, so no container has to be built or scanned and no subprocess has to be invoked to produce it. It keeps the maintainability dimension from depending on a single instrument, since Hadolint is embedded in the prototype and cannot serve as its own reference. And it operates on logical instructions rather than physical lines of code (LOC): a single RUN split across eight lines with backslash continuations would be counted as eight lines by a LOC measure but represents one logical instruction, so the logical count stays insensitive to formatting. This follows the theoretical advantage that Park [24] recognises in logical statement counts, namely their relative independence from formatting style, which is the property that matters when the goal is to measure structural change rather than physical layout.

**Reading ΔInstr**

The sign convention applied to ΔSize, ΔCVEs, and ΔWarnings, where a negative delta means improvement, does not transfer to ΔInstr. Refactorings act on the instruction count in three distinct ways, and each has to be read against the operation that produced it:

- **Consolidation and removal reduce the count.** Inline RUN Instructions (R01), Inline Stage (R06), and Remove RUN (R12) leave fewer instructions, and the reduction is a structural gain.
- **Extraction and addition increase it:** Add ENV Variable (R03), Add ARG Instruction (R04), Extract Stage (R05), Extract RUN Instructions (R09), and Move Stage (R11)

introduce instructions on purpose. An ENV that centralises a version number adds one instruction and still makes the file easier to maintain, which is the point of the refactoring; extracting a RUN block into a script or a stage into its own file works the same way.

- **Some refactorings leave it unchanged:** Update Base Image TAG (R02), Sort Instructions (R07), Replace ADD with COPY (R08), Update Base Image (R10), Update RUN Instruction (R13), and Rename Image (R14) act on arguments, order, or formatting, so the count is identical before and after.

ΔInstr is therefore reported as a structural indicator interpreted per refactoring type, and not as a score to be minimised.

**Refactorings at the limit of quantitative reach**

Two refactorings sit at the edge of what the four metrics can register, and each does so for a different reason. Update RUN Instruction (R13) reformats a RUN by splitting it across lines and ordering its arguments; the controlled experiment measured a size increase of 16 bytes (11,100,915 → 11,100,931), reflecting the longer command string stored in the image configuration, a variation that falls well below the relevance threshold. Rename Image (R14) replaces numeric stage references with readable identifiers and measured no change at all (3,632,095 → 3,632,095), the only exact null size delta in the catalogue. On the remaining indicators both behave the same way: the vulnerability count is stable across states, 146 for R13 and 77 for R14, each measured on a non-null debian:12-slim baseline; neither matches a Hadolint rule, so ΔWarnings is zero; and neither adds or removes a logical instruction, so ΔInstr is zero. What the metrics do not capture is the benefit that motivates both refactorings, which is readability. This is reported as a genuine result rather than treated as a gap: inventing a readability metric for two refactorings would weaken the measurement approach instead of strengthening it. The maintainability classification of both rests on the nature of the operation, discussed in Section 4.2.3.

### 4.1.4 Challenges

Three engineering challenges shaped the design of the prototype and must be resolved for the system to produce reliable results.

The first challenge is multi-line RUN instruction parsing. Dockerfiles commonly express a single logical shell command across multiple physical lines using the backslash continuation character. A parser that operates at the line level will fragment a single logical RUN instruction into multiple tokens, causing incorrect instruction counts and false refactoring detections. The dockerfile-parse library resolves this by exposing logical instruction objects regardless of their physical line count, as validated in Experiment 5. The Detection Engine is designed to operate exclusively on this normalised representation.

The second challenge is JSON schema heterogeneity. The Data Extractor must aggregate metrics from two tools with incompatible output structures. Hadolint produces a flat array of warning objects with fields code, level, line, and message. Trivy produces a deeply nested structure with a Results array, each element of which contains a Vulnerabilities sub-array. A

delta computation that is arithmetically valid requires both schemas to be normalised into a common internal representation before any comparison is performed. Experiment 2 confirmed the structure of both schemas, providing the ground truth for this normalisation design.

The third challenge is consolidation-versus-deletion ambiguity. When the count of RUN instructions decreases between two commits, the change may represent a pure consolidation, a pure deletion, or a mixed commit combining instruction inlining with concurrent code cleanup. These operations have distinct structural implications. To accommodate real-world developer practices where refactorings and minor cleanups frequently overlap, the Detection Engine employs a fusion-proof criterion rather than a strict preservation check. It searches for positive evidence of instruction merging: if the primary command blocks are effectively combined into a surviving instruction, the change is classified as a consolidation refactoring and triggers the R01 rule even if minor adjacent commands or cleanups were simultaneously deleted. Conversely, if the reduction in instruction count stems from a pure deletion without any structural fusion, the change is classified as a deletion and is not reported as a refactoring.

### 4.1.5 Functional and Non-Functional Requirements

The functional requirements are derived directly from the experimental findings and are organized by system component.

RF1 - VCS Connector: The system shall extract the complete textual content of the Dockerfile at two user-specified commits from a local Git repository, using programmatic blob-level access without requiring a working tree checkout. [Validated in Experiment 3]. Acceptance criterion: Dockerfile content retrieved at two commits is byte-identical to the content stored in the Git object database

RF2 - Detection Engine: The system shall parse both Dockerfiles into their canonical logical instruction representations and determine whether the structural change between them constitutes a supported refactoring type. For each detection, the engine shall identify the specific instructions involved. [Validated in Experiment 5]. Acceptance criterion: the 4-to-1 RUN consolidation from Experiment 5 is detected automatically from the parsed instruction lists.

RF3 - Detection Engine: The system shall assign to each detected transformation the formal identifier of the corresponding catalogue type (R01–R14) and emit a typed DetectionResult carrying that identifier together with the instructions involved. The identifier is a label attached to the detection and carries no quality expectation: no impact value is read from the catalogue at this or any later stage, and the metric deltas are computed independently by the Performance Analyzer and the Data Extractor from the two Dockerfile states. Acceptance criterion: the Dockerfile pair for R06 (Inline Stage) produces a DetectionResult carrying the identifier R06 and the stage instructions involved, with no impact field populated.

RF4 - Performance Analyzer: The system shall build the Docker image for each Dockerfile state and retrieve the image size in bytes via the Docker SDK, returning a signed delta (ΔSize) expressed in bytes. [Validated in Experiment 4]. Acceptance criterion: PoC-3 test — Delta Size correctly computed between Dockerfile A (ubuntu:22.04) and Dockerfile B (alpine:3.19): the image size is reduced from 29.7 MB to 3.4 MB, a difference of 26.3 MB. At byte resolution the Docker SDK reports this as a signed Delta Size of −26,318,550 bytes (29,748,045 → 3,429,495 bytes).

RF5 - Data Extractor: The system shall invoke Hadolint and Trivy against each image state using JSON output mode, normalise their heterogeneous schemas into a common metric representation, and return the warning count delta (ΔWarnings) and CVE count delta (ΔCVEs) by severity tier. The CVE count shall be the number of distinct vulnerability identifiers reported by Trivy, not the number of occurrences, so that a single vulnerability affecting several packages is counted once. The warning count shall encompass all maintainability-related rule violations reported by Hadolint, capturing the global sum of warnings to reflect the overarching maintainability health of the Dockerfile, thereby ensuring a universal evaluation independent of specific rule constraints. [Validated in Experiment 2 and PoC tests PoC-2, PoC-3]. Acceptance criteria: PoC-2 test — correctly computes the warning delta for the test scenario, yielding −1 for DL3007; PoC-3 test — correctly computes the CVE delta from the Trivy JSON output, with the reference value of −15 reflecting the specific database state at the time of the test run rather than a fixed global constant.

RF6 - Data Extractor: The system shall count the logical instructions of each Dockerfile state, excluding COMMENT entries, and return the signed difference (ΔInstr) as a structural maintainability indicator reported alongside ΔWarnings. The count shall be derived from the textual content of the Dockerfile through dockerfile-parse, and not from the built image, so that multi-line instructions joined by backslash continuations resolve to a single logical unit. Acceptance criteria: R06 test — $\Delta\text{Instr} = -3$ correctly computed; R08 test — $\Delta\text{Instr} = 0$ for a refactoring that reorganizes or replaces instructions without changing the total logical instruction count.

RF7 - Report Generator: The system shall produce a structured output presenting the before and after values for each quality metric along with the computed metric deltas (delta_size, delta_warnings, delta_cves, delta_logical_instructions) associated with the detected refactoring rules for the target Dockerfile. Acceptance Criterion: The generated report correctly displays the initial metric values, final metric values, and calculated deltas for all four measured metrics alongside the corresponding detected rule IDs.

RNF1 - Metric Precision: The Performance dimension shall be quantifiable at byte-level resolution sufficient to detect significant size reductions. Acceptance criterion: PoC-3 test — the Docker SDK returns the exact byte size of each image, so the 26.3 MB reduction is preserved at full resolution as −26,318,550 bytes rather than being rounded away by CLI text parsing.

RNF2 - Diagnostic Sensitivity: The two maintainability instruments shall each be sensitive enough to detect a change in the property they measure. At the smell level, Hadolint shall detect the complete elimination of a relevant rule violation; at the structural level, the parser shall detect the addition or removal of a single logical instruction. Sensitivity is a property of the instruments and not a claim of coverage: as stated in Section 4.1.3, neither indicator spans the full set of maintainability sub-characteristics defined by ISO/IEC 25010 [25], and the requirement asserts only that each responds to the changes it is designed to register. Acceptance criteria: PoC-2 test — Delta Warnings(DL3007) = -1 correctly detected, representing complete elimination of the mutable-tag smell between Dockerfile A (ubuntu:latest) and Dockerfile B (ubuntu:22.04); R06 test — Delta Instr = -3 correctly detected on a refactoring that registers an improvement on the maintainability axis.

RNF3 - Extensibility: The Detection Engine rule set shall be extensible to new refactoring types without modification of existing component code, following the Open-Closed Principle [26]. Each rule is self-contained and registered in a declarative rule list, so that adding a type requires an addition to that list and no change to the execution logic of any component. The prototype supports fourteen types; the architecture must accommodate additional types in future iterations. Acceptance criterion: a new detection rule can be added to the registry and exercised end to end without editing any existing rule or any downstream component.

RNF4 - Reliability: All external invocations, Docker SDK, Hadolint, Trivy, and the parsing of Dockerfile content shall be wrapped in exception handlers that distinguish recoverable errors from unrecoverable failures and surface informative diagnostic messages. Acceptance criterion: build failure, daemon error, missing Dockerfile, and malformed Dockerfile content each produce a distinct informative exception.

RNF5 - Reproducibility: The system shall record, in every report, the analysis timestamp, the version of each external tool invoked (Docker Engine, Hadolint, Trivy, dockerfile-parse), and the Trivy vulnerability database version, enabling future re-runs to identify whether differences in CVE counts are attributable to Dockerfile changes or to database updates. Furthermore, reproducibility is ensured not only by these records but also through the capability to retain the source analysis artifacts generated by external tools during the extraction phase. Acceptance criterion: every generated report contains the analysis timestamp, the tool version block, the Trivy vulnerability database version, and the retained source analysis artifacts.

### 4.1.6 Architecture Design

The prototype tool is structured as a sequential data pipeline composed of four logical stages, where each stage transforms a well-defined input into an output consumed by the next. This architecture enforces the strict causal dependency chain required by the analysis: version retrieval (Stage 1) must precede refactoring detection (Stage 2), which in turn provides the context for the empirical evaluation (Stage 3) and the final report generation (Stage 4). The sequential pipeline ensures deterministic, reproducible execution consistent with RNF5.

The pipeline is expressed as the following transformation sequence, defining the execution order:

- **Stage 1 — Data Collection:** Retrieves the Dockerfile content from the version history via the VCS Connector.
- **Stage 2 — Refactoring Detection:** Parses the Dockerfiles and identifies the transformation using the Detection Engine.
- **Stage 3 — Empirical Evaluation and Measurement:** Performs the build and analysis. This stage relies on the Image Builder mechanism to reconstruct the environment, while the Performance Analyzer and Data Extractor execute in parallel to collect performance, security, and maintainability metrics.
- **Stage 4 — Report Generation:** Aggregates the results into the final ImpactReport.

Three architectural approaches were considered. An event-driven architecture was rejected because it cannot enforce the strict causal dependency chain required by this analysis: each stage depends on the typed output of the previous stage, version extraction must produce Dockerfile strings before detection can operate, and detection must produce a DetectionResult before the measured deltas can be paired with it. A component that consumes an event before its predecessor has completed would receive incomplete data, producing unreliable results. A service-oriented architecture was rejected because exposing each pipeline stage as an independent network service would introduce inter-service communication overhead and interoperability risks that contradict the Python-native design constraint established during the rapid experimentation phase. The sequential pipeline was selected because MSR-based analysis inherently follows a structured sequence of repository selection, data extraction, and analysis steps [20], and this architecture maps directly onto the validated experimental scripts, enforces the causal ordering of operations, and ensures that identical inputs always produce identical outputs.

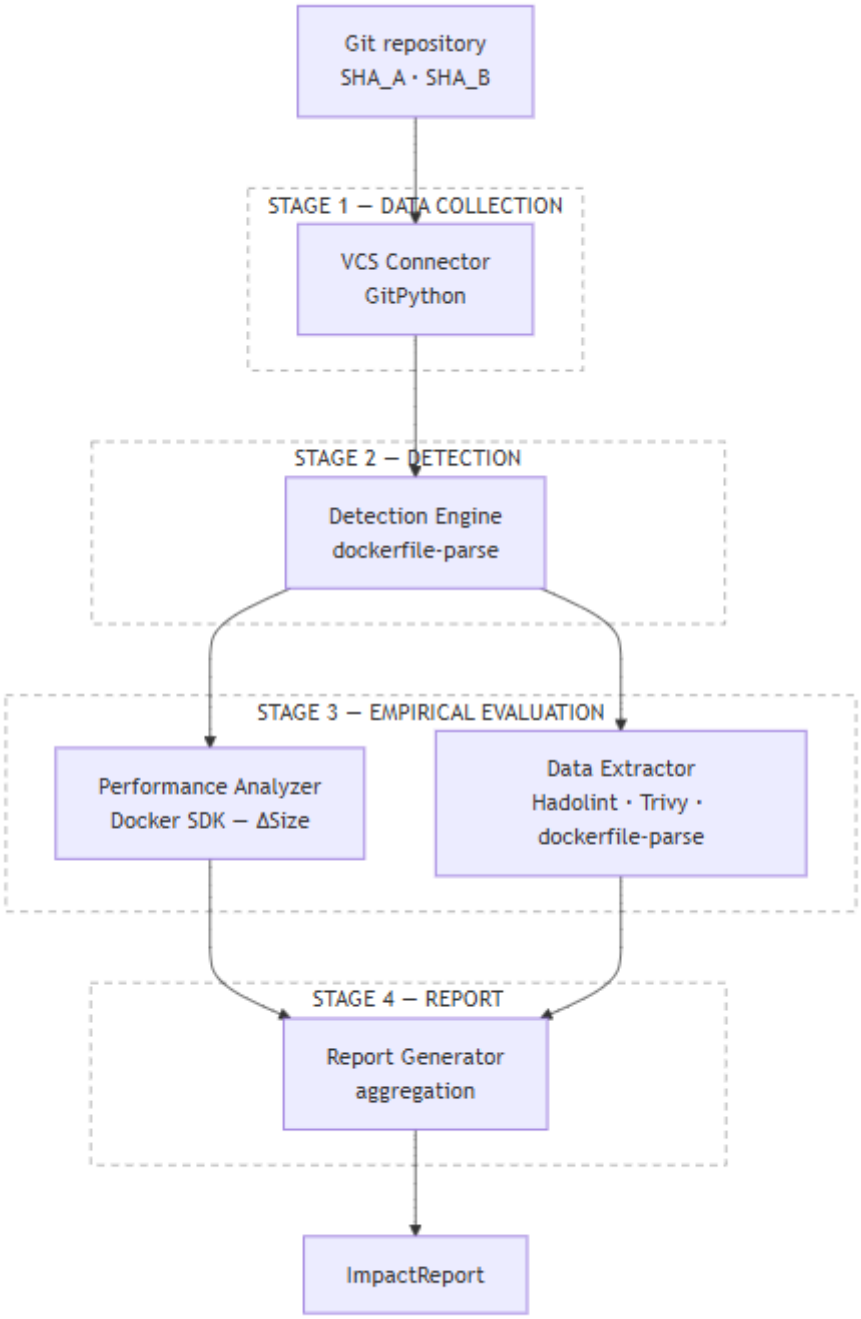



Figure 4.1 - Sequential architecture of the prototype analysis pipeline, showing the execution order of the components. The data dependencies between components are detailed in Figure 4.2.

At the metric collection stage, between the Detection Engine and the Report Generator, the three quality dimensions are measured in parallel. The Data Extractor dispatches Hadolint (Maintainability) and Trivy (Security) concurrently while the Performance Analyzer retrieves the image size via the Docker SDK (utilizing the internal Image Builder mechanism to construct

the container images). Since these three operations are mutually independent, none requires the result of another, parallel execution is the natural architectural choice, reducing the total metric collection time from the sum of three sequential operations to approximately the duration of the slowest one. Each branch produces a typed metric object that is aggregated by the Report Generator. The fourth delta, ΔInstr, is produced by the Data Extractor as well, but it does not occupy a branch of its own: it requires no external tool invocation, since it is a direct count over the instruction list the component parses from the Dockerfile text itself, and its cost is negligible against the build and scan operations that dominate this stage. Neither branch consumes the output of the Detection Engine: both operate on the Dockerfile pair produced by the VCS Connector, so the deltas are obtained without any knowledge of which refactoring was detected. The detected identifier and the measured deltas meet only at the Report Generator, where they are paired in the final report. To complement the execution sequence shown in Figure 4.1 with an explicit view of the components and the data exchanged between them, Figure 4.2 presents the component diagram of the prototype, showing each module, its provided and required interfaces, the dependencies between modules, and the external tools used.

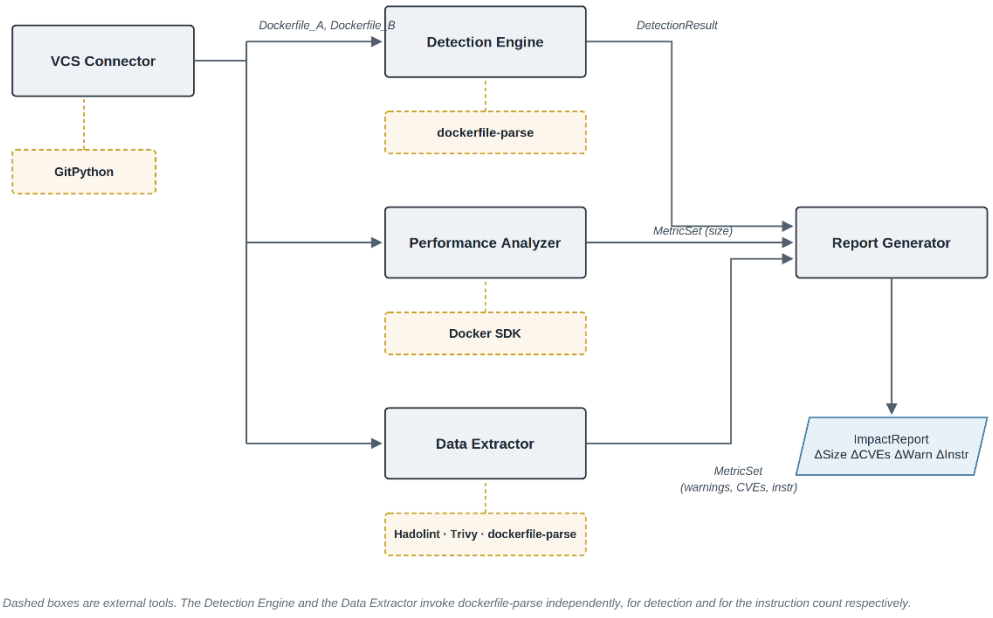



Figure 4.2 - Component diagram of the prototype: modules, provided and required interfaces, dependencies, and external tools

Figure 4.2 presents the component diagram of the prototype, showing the flow of data between components rather than their temporal execution order. Each component provides a typed object, shown as the label on its outgoing dependency, and requires the objects on its incoming dependencies, following the data transformations defined in Table 4.10. The VCS Connector provides the two Dockerfile versions (Dockerfile_A and Dockerfile_B), which are required by the Detection Engine, the Performance Analyzer, and the Data Extractor. The

Detection Engine provides a DetectionResult, carrying the catalogue identifier of the transformation and the instructions involved, which is required directly by the Report Generator. The Performance Analyzer provides the image sizes and the Data Extractor provides the metric sets covering warnings, vulnerabilities, and the logical instruction count. The Report Generator requires the DetectionResult, the sizes, and the metric sets, and produces the ImpactReport as its output artifact. The diagram makes the separation between detection and measurement explicit: the Performance Analyzer and the Data Extractor have no incoming dependency on the Detection Engine, so no expected impact is available to them while they measure. The dashed dependencies show the external tools each component uses: GitPython (VCS Connector), dockerfile-parse (Detection Engine and Data Extractor), the Docker SDK and the Docker daemon (Performance Analyzer), and Hadolint and Trivy (Data Extractor). The Report Generator relies only on internal rule-based logic and uses no external tool.

The interaction between the user and the prototype is intentionally simple. Figure 4.3 presents the use case diagram, capturing what the user can do with the tool.

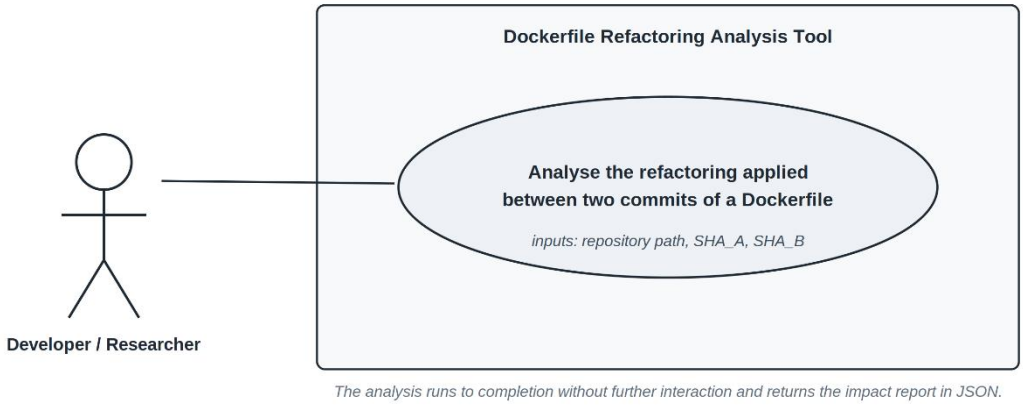



Figure 4.3 - Use case diagram of the Dockerfile refactoring analysis tool.

The tool exposes a single use case to its user, a developer or researcher: analysing the refactoring applied between two commits of a Dockerfile. The user provides the repository and the two commit identifiers, and the analysis then runs to completion without further interaction, producing the impact report in JSON format. The simplicity of this view reflects the design of the tool as a single-purpose, non-interactive analysis pipeline.

Figure 4.4 shows the deployment view of the prototype, identifying where each part of the system executes at runtime.

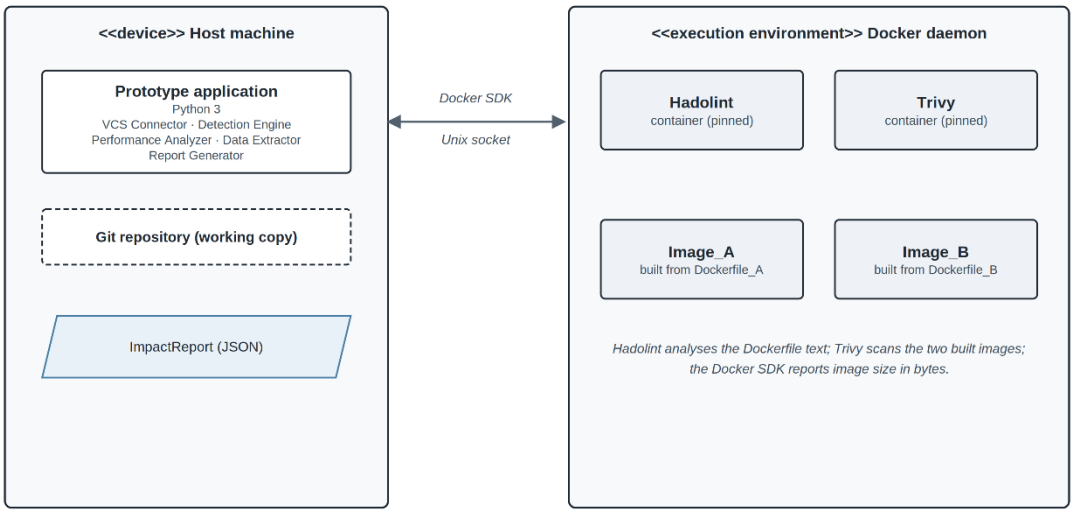



Figure 4.4 - Deployment diagram of the prototype, showing the Python application, the Docker daemon, and the tool and image containers.

The prototype runs as a Python application on the host machine and delegates all container operations to the Docker daemon, with which it communicates through the Docker SDK over the Unix socket. The Hadolint and Trivy tools execute as containers managed by this daemon, alongside the images built from Dockerfile_A and Dockerfile_B. Hadolint analyzes the Dockerfile text, while Trivy scans the two built images. Running the tools as containers keeps their versions pinned and removes the need for host-level installation, which supports the reproducibility of the analysis.

To provide a complete overview of the entire architecture, Table 4.2 summarizes the responsibilities of every system component, the technologies used for their implementation, and the requirements they address:

| **Component** | **Responsibility** | **Validated Technology** | **Satisfies Requirements** |
|---|---|---|---|
| **VCS Connector** | Extracts Dockerfile content from the Git object model | GitPython — Exp. 3 | RF1, RNF4, RNF5 |
| **Detection Engine** | Parses logical instructions, identifies the refactoring type, and emits its formal identifier | dockerfile-parse — Exp. 5 | RF2, RF3, RNF3 |

| Component | Responsibility | Validated Technology | Satisfies Requirements |
|---|---|---|---|
| **Performance Analyzer** | Builds image and retrieves size in bytes via daemon API | Docker SDK — Exp. 4 | RF4, RNF1, RNF4 |
| **Data Extractor** | Invokes static and security tools, counts logical instructions, and normalises metrics | Hadolint + Trivy — Exp. 2; dockerfile-parse | RF5, RF6, RNF2, RNF4, RNF5 |
| **Report Generator** | Aggregates the detected identifier with the metric deltas and produces the ImpactReport | Rule-based computation | RF7, RNF5 |

Table 4.2 - Prototype components, their responsibilities, technologies, and satisfied requirements.

### 4.1.7 Verification and Validation Strategy

The verification and validation strategy for the prototype is structured across three levels, consistent with standard software engineering practice.

At the unit level, each pipeline component is tested in isolation using the inputs and expected outputs established during the controlled multi-dimension experiments. For example, the Detection Engine unit test verifies that the Dockerfile pair for R06 (Inline Stage) is correctly identified and that the emitted DetectionResult carries the R06 identifier. The Report Generator unit test verifies that the four deltas are computed from the two MetricSet objects and paired with that identifier without alteration.

At the integration level, the interfaces between components are validated by running the complete pipeline against each of the four controlled multi-dimension experiment Dockerfile pairs (R06, R09, R11, and R12) and verifying that the typed data structures, DetectionResult, MetricSet, ImpactReport, are correctly produced and consumed at each boundary as specified in Table 4.2.

At the system level, end-to-end behavior is validated against the four controlled multi-dimension experiment scenarios (R06, R09, R11, and R12), all executed using an alpine:3.20 base image. The expected reference values are:

- **R06 (Inline Stage)** — $\Delta\text{Size} = +116$ bytes (3,632,710 → 3,632,826), $\Delta\text{CVEs} = 0$, $\Delta\text{Warnings} = 0$, $\Delta\text{Instr} = -3$ (8 → 5);
- **R09 (Extract RUN Instructions)** — $\Delta\text{Size} = +417$ bytes (3,633,775 → 3,634,192), $\Delta\text{CVEs} = 0$, $\Delta\text{Warnings} = 0$, $\Delta\text{Instr} = +1$ (5 → 6);
- **R11 (Move Stage)** — $\Delta\text{Size} = +1$ byte (3,632,724 → 3,632,725), $\Delta\text{CVEs} = 0$, $\Delta\text{Warnings} = 0$, $\Delta\text{Instr} = +1$ (8 → 9);
- **R12 (Remove RUN (mv))** — $\Delta\text{Size} = -1{,}331$ bytes (3,634,927 → 3,633,596), $\Delta\text{CVEs} = 0$, $\Delta\text{Warnings} = 0$, $\Delta\text{Instr} = -2$ (8 → 6).

Divergence from these reference values indicates a regression in the pipeline. These reference values serve as regression anchors for the implementation phase described in Chapter 5.

## 4.2 Extended Refactoring Catalogue

Each rule in the Extended Catalogue is evaluated across all three quality axes, Performance, Security, and Maintainability, forming a complete three-dimensional impact vector. This approach provides a transparent overview of what a transformation is intended to achieve and of any degradation it introduces on the remaining axes:

| **ID** | **Rule** | Perfomance | **Security** | Maintainability |
|---|---|---|---|---|
| **R01** | Inline RUN Instructions | ↑ | X | ↓ |
| **R02** | Update Base Image TAG | ↑* | ↓ | ↑ |
| **R03** | Add ENV Variable | X | ↓ | ↑ |
| **R04** | Add ARG Instruction | X | ↓ | ↑ |
| **R05** | Extract Stage | ↑ | ↑ | ↑ |
| **R06** | Inline Stage | X | X | ↑ |
| **R07** | Sort Instructions | ↑ | X | ↓ |

| ID | Rule | Perfomance | Security | Maintainability |
|---|---|---|---|---|
| **R08** | Replace ADD with COPY | X | ↑ | ↑ |
| **R09** | Extract RUN Instructions | X | X | ↑ |
| **R10** | Update Base Image | ↑ | ↑ | ↑ |
| **R11** | Move Stage | X | X | ↑ |
| **R12** | Remove RUN Instruction (mv) | X | X | ↑ |
| **R13** | Update RUN Instruction | X | X | ↑ |
| **R14** | Rename Image | X | X | ↑ |

Table 4.3 - Extended Dockerfile refactoring catalogue mapping fourteen types across three quality dimensions

Methodological note on the relevance threshold. For performance evaluation purposes in this research, a 1 MB threshold was established. This order of magnitude reflects the scale of analysis typically used in the specialized literature to size Docker images (where variations and reductions are inherently reported in Megabytes). Residual dimensional variations below 1 MB fall below the operational granularity of the artifact, presenting a null impact on storage, network, or startup time. Consequently, they were methodologically classified as neutral (X).

A crucial conceptual distinction must be established regarding the quality dimensions assigned to the refactoring types within the Extended Catalogue (Table 4.3). The directional impact values mapped across the three quality dimensions, Performance, Security, and Maintainability, for each rule represent a theoretical propensity or strategic intent derived from software engineering literature and documented best practices (which are presented, discussed, and individually cited for each refactoring rule detailed below, from R01 to R14), i.e., what a structural transformation is conceptually designed to achieve in an ideal, isolated setting. Rather than assuming a rigid primary focus, each refactoring is evaluated through a three-dimensional impact vector, where individual dimensions are classified as improved (↑), degraded (↓), or neutral (X). This multi-axial structure explicitly captures potential directional compromises and side effects of a transformation across independent quality axes without imposing a hierarchical ranking between them.

However, the evaluation pipeline is built to capture the empirical reality of software evolution, since what a refactoring is defined to do and what it produces in a given build are not the same claim. When implemented in a real-world DevOps environment, a refactoring operation behaves as a necessary but not sufficient condition for quality improvement:

applying it creates the opportunity for the expected effect, and whether that effect materialises depends on the state of the artifact it was applied to rather than on the transformation alone. While the detection engine successfully validates the structural and syntactic alignment of a change against the catalogue, the developer remains responsible for the semantic parameters supplied to that instruction. If a structurally correct refactoring is executed with obsolete, insecure, or poorly configured arguments, external tools will register a local degradation in performance or security metrics.

Consequently, an empirical divergence between the catalogue's classification and the tool's computed deltas does not invalidate the taxonomy. Instead, it exposes the operational friction between structural intent and semantic execution, reinforcing the utility of the prototype as an objective auditing mechanism capable of surfacing measurable quality impacts when developer decisions diverge from theoretical benefits.

This separation is what makes the tool an auditing instrument rather than a confirmation of its own taxonomy. The pipeline holds no table of expected impacts, no dictionary keyed by refactoring type, and no lookup performed while an analysis is running. It builds both images, scans both, parses both, and reports the four deltas it obtains. A result that contradicts the direction recorded in Table 4.3 is therefore a finding about the build environment or about the transformation as applied in that specific case, and never an inconsistency inside the software. Keeping the theoretical mapping outside the code is also what allows the catalogue to be revised without touching the prototype, and the prototype to be run against refactorings whose quality direction is still open.

The remainder of this section develops the technical justification for each classification in Table 4.3. For each refactoring, the dimensions are supported either by empirical evidence from the literature or, where no such evidence exists, by a controlled experiment designed for this dissertation (described in Section 4.2.1 and available in the project repository). Each justification also notes any relevant trade-off across quality dimensions.

**R01 — Inline RUN Instructions**

**Performance ↑.** Rosa et al. [17] identify DL3059 as the smell most frequently fixed by developers in open-source projects, with 168 manually validated fixes, approximately three times more frequent than any other fix. The study reports that these fixes were "explicitly performed to reduce the Docker image size and the number of layers". Ksontini et al. [2] independently classify Inline RUN Instructions under the Image Size category, observing it in four commits of their corpus.

**Security X.** The controlled experiment on debian:12-slim measured a stable vulnerability count of 146 in both states (ΔCVEs = 0). Because the baseline is non-null, this value is a measurement rather than an absence of data: consolidating consecutive RUN instructions alters the layer structure without changing which packages are installed or their versions, so the attack surface is unaffected.

**Maintainability ↓.** Nüst et al. [16] observe that a single lengthy RUN instruction chaining multiple commands "creates instructions that may be hard to read", and recommend that each RUN perform one scoped action, since reasonably grouped instructions increase the readability of the Dockerfile. The performance gain therefore carries a legibility cost.

**R02 — Update Base Image TAG**

**Performance ↑* (not attributable).** The controlled experiment measured a size reduction of 11,845,577 bytes (41,593,622 → 29,748,045), a numerically substantial improvement of approximately 28% and the only measured variation in the catalogue above the relevance threshold. It is nevertheless reported under a distinct symbol because it constitutes a threat to validity rather than a clean measurement of the refactoring. The before state uses the floating tag ubuntu:latest, whose resolution changes whenever the publisher rebuilds the image: the magnitude observed reflects the difference between whichever version latest pointed to at build time and the pinned ubuntu:22.04, not an effect produced by the transformation itself. A subsequent execution of the identical pair may yield a different magnitude, or a positive delta, if latest resolves to a smaller image than the pinned tag. The improvement is real for this run and not reproducible across runs, which is precisely the property that motivates the refactoring in the first place, and the reason the size delta cannot serve as a stable reference value.

**Security ↓.** The Docker Documentation [7] states that when pinning an explicit version "you're opting out of automated security fixes, which is likely something you want to get", and recommends the opposite practice for security: "To keep your images up-to-date and secure, rebuild your images regularly with updated dependencies." Building on that warning and on Zerouali et al. [27], who found across 7,380 Debian-based images that the number of outdated packages is strongly correlated with the number of resolved vulnerabilities ($\rho > 0.9$), and that the most vulnerable containers were those not updated for more than two years, pinning without a regular update procedure accumulates exposure over time. The effect is moderated rather than absolute: the same study notes that no release is devoid of vulnerabilities, so currency alone does not eliminate them.

**Maintainability ↑.** Rosa et al. [17] report a 64% acceptance rate for DL3007 fixes submitted as pull requests. The Docker Documentation [7] gives the underlying rationale: with a mutable tag "you're not guaranteed to get the same for every build" and "you don't have an audit trail of the exact image versions that you're using".

This experiment was conducted independently of the proof-of-concept run reported in Section 4.1.2, using the same before and after images. The two sets of figures coincide to the byte because ubuntu:latest still resolved to the same manifest on the second occasion and the measurement pipeline is deterministic. That agreement confirms the determinism of the measurement rather than the stability of the refactoring: it holds only for as long as the publisher leaves the tag pointing where it is.

#### R03 — Add ENV Variable

**Performance X.** The controlled experiment on alpine:3.20, pinned by digest in both states, measured an increase of 174 bytes (3,632,570 → 3,632,744), corresponding to the manifest entry for the ENV directive. The Docker Documentation [7] states that "each ENV line creates a new intermediate layer, just like RUN commands", and the measurement quantifies that layer as metadata carrying no filesystem content. Under the relevance threshold adopted in this work, 174 bytes represent 0.005% of the image and produce no measurable effect on storage, network transfer, or start-up time.

**Security ↓.** Both the Docker Documentation [7] and Bui et al. [28] warn that placing sensitive data in ENV instructions is a security risk. Bui et al. observe that such values are "transferred to the Docker images during the build and visible to every image user", and the documentation adds the mechanism: "even if you unset the environment variable in a future

layer, it still persists in this layer and its value can be dumped." This risk is strictly conditional on the nature of the centralised value. As Rahman and Williams [29] establish for infrastructure-as-code more generally, a security smell is a recurring coding pattern that implies a security weakness, but labelling that pattern an actual vulnerability "necessitates consideration of the application context": a hard-coded secret is relevant only if it is genuinely used to authenticate. Where the refactoring targets operational data such as software versions or directory paths, no security risk materialises.

**Maintainability ↑.** The Docker Documentation [7] recommends ENV "to set commonly used version numbers so that version bumps are easier to maintain", noting that this approach "lets you change a single ENV instruction to automatically bump the version of the software in your container". Nüst et al. [16] make the same point, observing that ENV is useful for setting software versions or paths and reusing them across multiple instructions to avoid mistakes.

#### R04 — Add ARG Instruction

**Performance X.** The controlled experiment on alpine:3.20 measured an increase of 193 bytes (3,632,570 → 3,632,763). Read against R03, the pair establishes an empirical contrast that the literature had asserted but not quantified: the ARG directive costs 19 bytes more than the equivalent ENV, consistent with the documentation's statement that build arguments "may persist in the image metadata, as provenance attestations and in the image history", the provenance record being the additional overhead. Both values are manifest metadata rather than filesystem content, and both fall below the relevance threshold.

**Security ↓.** The Docker Documentation [7] states that build arguments "are inappropriate for passing secrets to your build, because they're exposed in the final image", and that they "may persist in the image metadata, as provenance attestations and in the image history, which is why they're not suitable for holding secrets". As with R03, the risk is conditional on whether the centralised value is a credential or operational metadata.

**Maintainability ↑.** The Docker Documentation [7] states that specifying versions as build arguments "lets you build with different versions without having to manually update the Dockerfile" and that "it also makes it easier to maintain the Dockerfile, since it lets you declare versions at the top of the file", concluding that build arguments "make Dockerfiles more flexible, and easier to maintain".

#### R05 — Extract Stage

**Performance ↑.** Ksontini et al. [2] classify Extract Stage under the Image Size category, observing it in seven commits, the second most frequent of the five refactorings in that category. The Docker Documentation [7] gives the mechanism: multi-stage builds "let you reduce the size of your final image, by creating a cleaner separation between the building of your image and the final output", ensuring that "the resulting output only contains the files that are needed to run the application".

**Security ↑.** This is the most pronounced security effect measured in the catalogue. The controlled experiment moved a Rust application from a single-stage rust:1.78 build to a multi-stage build with an alpine:3.20 runtime, reducing the vulnerability count from 6,162 to 0 (ΔCVEs = −6,162). The result confirms empirically what the Docker Documentation [7] recommends: "A small image with minimal dependencies can considerably lower the attack

surface." Discarding the compiler toolchain and build dependencies from the final artifact eliminates the vulnerabilities they carry.

**Maintainability ↑.** Ksontini et al. [2] classify this refactoring under Modularity, describing it as a case where "refactorings are used to reduce image complexity by breaking image into various stages". The same source characterises the absence of modularity as a condition where "adding new/changing existing components is difficult and error-prone", which is the defining concern of maintainability.

**R06 — Inline Stage**

**Performance X.** The controlled experiment on alpine:3.20 measured an increase of 116 bytes (3,632,710 → 3,632,826), attributable to layer metadata. The transformation removes a stage boundary without altering the filesystem content of the final image, and the residual variation falls below the relevance threshold.

**Security X.** The vulnerability count was zero in both states (0 → 0). Because the alpine:3.20 baseline is itself null, this value does not distinguish an absence of effect from an absence of detectable vulnerabilities.

**Maintainability ↑.** The logical instruction count fell by three (8 → 5) as the redundant FROM and the COPY --from were removed, collapsing the build from two stages into one. No rule in the 47-rule maintainability screen changed state. The reduction is a direct, measured structural simplification.

**R07 — Sort Instructions**

**Performance ↑.** Zhu et al. [30] evaluated instruction reordering across 2,000 GitHub repositories, reporting that the technique improved 92.75% of the Dockerfiles analysed, with a mean rebuild-time reduction of 26.5% and reductions above 50% in 12.82% of the files. The mechanism is postponing frequently modified instructions and prioritising stable, resource-intensive steps, which preserves cache validity across rebuilds.

**Security X.** The controlled experiment on debian:12-slim measured a stable vulnerability count of 119 in both states (ΔCVEs = 0). With a non-null baseline, this is a measurement: reordering instructions changes when each step executes but not what the resulting image contains.

**Maintainability ↓.** The same study reports that restoring semantic grouping, the arrangement developers find readable, carries a cost that Zhu et al. [30] quantify at approximately 5.9% and describe as a "trade-off between efficiency and readability". Optimal cache ordering and human-readable ordering are therefore not the same arrangement.

**R08 — Replace ADD with COPY**

**Performance X.** The controlled experiment on alpine:3.20 measured an increase of 1 byte (3,632,098 → 3,632,099), a nominal difference in the instruction configuration. For a regular local file, COPY and ADD produce identical content in the destination layer, and the variation is far below the relevance threshold.

**Security ↑.** Bui et al. [28] identify the unnecessary use of ADD as a security smell, observing that "ADD instructions can download and unpack remote files from URLs without any checks,

which potentially introduces security risks", and recommending replacement by equivalent COPY instructions. Replacing ADD where its extraction and fetching capabilities are not required removes that implicit behaviour.

**Maintainability ↑.** Rosa et al. [17] report a 71% acceptance rate for DL3020 fixes submitted as pull requests, the second highest in their study. Ksontini et al. [2] list Replace ADD Instruction with COPY Instruction among the "three extra Dockerfile-related refactoring types performed to improve Maintainability", observing it in four commits, and attribute the benefit to ADD's unpredictability: "the result of such unreliable behavior often came down to copying when we want to extract and extracting when we want to copy."

**R09 — Extract RUN Instructions**

**Performance X.** The controlled experiment on alpine:3.20 measured an increase of 417 bytes (3,633,775 → 3,634,192), the largest residual variation in the catalogue, determined by the external script file added to the image and the manifest cost of the additional COPY directive. Although the most pronounced of the metadata-scale results, it remains three orders of magnitude below the operational threshold and is therefore classified as neutral.

**Security X.** The vulnerability count was zero in both states (0 → 0). Because the alpine:3.20 baseline is itself null, this value does not distinguish an absence of effect from an absence of detectable vulnerabilities.

**Maintainability ↑.** The controlled experiment measured a logical instruction count rising by one (5 → 6), reflecting the COPY added for the external script, with no rule in the 47-rule maintainability screen changing state. This positive delta is the structural cost of the extraction rather than a regression: the seven chained shell commands moved out of the Dockerfile were never counted, having lived inside the arguments of a single RUN, so the benefit of the operation falls outside the reach of the instruction count. The direction of that benefit is established by Ksontini et al. [2], who list Extract RUN Instructions among the "three extra Dockerfile-related refactoring types performed to improve Maintainability" and observe that the technique "does not only shrinks the image size but also groups commands in a much more cleaner, simpler, and portable format". The classification therefore combines the measured structural cost with the documented benefit that the metric cannot capture.

**R10 — Update Base Image**

**Performance ↑.** Haque and Babar [31] demonstrate the effect directly: replacing node:current-slim with node:alpine in a containerised application reduced the image from 183 MB to 127 MB — a variation well above the relevance threshold. Ksontini et al. [2] classify Update Base Image under the Image Size category, with twelve commits, the highest frequency of that category.

**Security ↑.** Haque and Babar [31] report that in the same application 94.2% of the vulnerabilities in the final image were inherited from the base image, and that the substitution reduced the vulnerability count from 69 to 4. In a second demonstration, migrating from debian:jessie to debian:buster reduced highly exploitable and high impact vulnerabilities by 88.2%. The choice of base image therefore governs the majority of an application's vulnerability exposure.

**Maintainability ↑.** Ksontini et al. [2] list Update Base Image among the "three extra Dockerfile-related refactoring types performed to improve Maintainability", explaining that using existing images avoids building and downloading dependencies on top of the base image, which "will reduce maintainability efforts as all required installations are done and best practices are probably applied especially when using official images".

**R11 — Move Stage**

Ksontini et al. [32] omitted this refactoring from their study because it appeared in only two Dockerfiles, so no measurement of its impact exists in the literature. Its effect was characterised exclusively through a controlled experiment on alpine:3.20, pinned by digest in both states.

**Performance X.** The controlled experiment on alpine:3.20 measured an increase of 1 byte on the final runtime image (3,632,724 → 3,632,725). This is the only refactoring in the catalogue whose after state spans two files, so the image metrics are taken on the final runtime image alone: the builder is built first and consumed by tag, and since the two images share layers, their sizes are never summed. The residual variation falls below the relevance threshold.

**Security X.** The vulnerability count was zero on the final runtime image in both states (0 → 0). Because the alpine:3.20 baseline is itself null, this value does not distinguish an absence of effect from an absence of detectable vulnerabilities.

**Maintainability ↑.** The logical instruction count admits two readings. Counted across both after files it rises by one (8 → 9), since the extracted Dockerfile requires its own FROM; counted in the main Dockerfile alone it falls by three. Both are true: the first expresses the cost of the modularisation, the second what it relieves in the principal artifact. The classification follows the measured reduction of three instructions in the principal artifact. No rule in the 47-rule maintainability screen changed state across either file.

**R12 — Remove RUN Instruction (mv command)**

In the absence of an empirical study isolating this refactoring, its impact was measured exclusively through a controlled experiment on alpine:3.20, pinned by digest in both states.

**Performance X.** The experiment measured a reduction of 1,331 bytes (3,634,927 → 3,633,596) through the elimination of two redundant layers. The direction of the effect is favourable, but the magnitude falls below the relevance threshold adopted in this work: at 1.3 KB the variation is three orders of magnitude below the operational scale of the artifact and produces no measurable effect on storage, network transfer, or start-up time. The dimension is therefore classified as neutral. The reduction is expected to scale with the size of the moved data, since that data would otherwise be duplicated across the temporary and final layers, but larger cases were not measured.

**Security X.** The vulnerability count was zero in both states (0 → 0). Because the alpine:3.20 baseline is itself null, this value does not distinguish an absence of effect from an absence of detectable vulnerabilities.

**Maintainability ↑.** The logical instruction count fell by two (8 → 6) as the two RUN instructions whose sole purpose was to execute a move were eliminated, with the destination path set directly in the preceding COPY instructions. No rule in the 47-rule maintainability screen changed state.

**R13 — Update RUN Instruction**

**Performance X.** The controlled experiment on alpine:3.20 measured an increase of 16 bytes (11,100,915 → 11,100,931), reflecting the longer command string stored in the image configuration after the instruction is split across lines. The package set and its versions are identical in both states, and the variation is negligible against an image of eleven megabytes, falling well below the relevance threshold.

**Security X.** The controlled experiment on debian:12-slim measured a stable vulnerability count of 146 in both states (ΔCVEs = 0). With a non-null baseline this is a measurement: reformatting an instruction changes its physical layout without altering what is installed.

**Maintainability ↑.** The Docker Documentation [7] recommends both components of this refactoring. On line splitting: "Split long or complex RUN statements on multiple lines separated with backslashes to make your Dockerfile more readable, understandable, and maintainable." On argument ordering: "Whenever possible, sort multi-line arguments alphanumerically to make maintenance easier. This helps to avoid duplication of packages and make the list much easier to update." Nüst et al. [16] concur, noting that "content spread across more and shorter lines also improves readability of changes in version control systems"

**R14 — Rename Image**

**Performance X.** The controlled experiment on alpine:3.20 measured no change whatsoever (3,632,095 → 3,632,095, ΔSize = 0). This is the only refactoring in the catalogue with an exact null size delta: assigning explicit names to build stages alters how they are referenced without adding any directive to the image manifest. The classification follows from the measurement itself and does not depend on the relevance threshold.

**Security X.** The controlled experiment on debian:12-slim measured a stable vulnerability count of 77 in both states (ΔCVEs = 0). With a non-null baseline this is a measurement rather than an absence of data.

**Maintainability ↑.** Ksontini et al. [2] classify renaming under Understandability, describing this category as refactorings "applied to reduce the effort to understand code, e.g., renaming elements". Replacing numeric stage indices with readable identifiers is the Dockerfile instance of that practice. The improvement is qualitative and outside the reach of the four quantitative indicators, which is reported as a genuine finding rather than a gap.

**Baseline asymmetry in the security dimension**

The security experiments for R06, R09, R11, and R12 were conducted on a clean alpine:3.20 baseline, yielding a constant zero count (0 → 0).

Because these refactorings are purely structural and syntactic (modifying build stages or layer arrangements without altering software packages or dependencies), their security impact is inherently neutral. Even if evaluated on a non-null baseline (such as Debian), the resulting delta would remain zero, as structural transformations do not introduce or remove package vulnerabilities. The Alpine baseline thus correctly demonstrates the security safety and neutrality of these refactorings.

The asymmetry runs the other way for the remaining security experiments. Inline RUN Instructions (R01), Sort Instructions (R07), Update RUN Instruction (R13), and Rename Image (R14) were measured on debian:12-slim rather than on alpine:3.20, and Extract Stage (R05) on the rust:1.78 toolchain image the refactoring itself requires. The reason is the one given above, applied in reverse: alpine:3.20 reports no vulnerabilities, so a ΔCVEs computed on it would be zero for every refactoring in the catalogue, and a zero obtained from a null baseline does not distinguish an absence of effect from an absence of anything to detect. On a baseline carrying a substantial and stable count, 146 for R01 and R13, 119 for R07, and 77 for R14, a null delta becomes a measurement in its own right, and the pronounced reduction recorded for R05 becomes attributable to the transformation rather than to the choice of image.

### 4.2.1 Classification Criteria

Each refactoring is evaluated independently across all three quality axes, Performance, Security, and Maintainability, forming a complete three-dimensional impact vector. This approach provides a transparent overview of positive effects, neutral behaviors, and trade-offs introduced by each transformation across all axes without relying on a primary-versus-secondary hierarchy.

Two rules govern how each axis is filled. First, a direction is only recorded when it is supported by evidence, whether measured in the literature or observed in a controlled experiment; no axis is marked to suggest importance where no supporting effect exists. Second, the value recorded on each axis carries a sign: an upward arrow marks an improvement, a downward arrow marks a regression, and a cross marks a neutral outcome, meaning that the refactoring leaves that axis unaffected or that the measured variation falls below the operational threshold defined for the dimension.

Recording regressions directly in the columns rather than only in the surrounding text is what makes the vector complete. For example, Add ENV Variable (R03) and Add ARG Instruction (R04) improve maintainability, but both open the possibility of exposing sensitive data through the build, so security is recorded as a regression on those rows and explained in the justification below. In the same way, the readability cost of aggressive consolidation in Inline RUN Instructions (R01) and of reordering in Sort Instructions (R07) is recorded as a maintainability regression rather than left implicit. For Update Base Image (R10), moving to a lighter and more current base image reduces inherited vulnerabilities and shrinks the final artifact at the same time, so security and performance are both recorded as improvements, with neither presented as the consequence of the other. Read row by row, the table therefore answers three separate questions, what does this refactoring do to performance, to security, and to maintainability, instead of a single question about which dimension matters most.

### 4.2.2 Maintainability Established by the Nature of the Operation

Refactorings such as R06 ($\Delta\text{Instr} = -3$), R09 ($\Delta\text{Instr} = +1$), R11 ($\Delta\text{Instr} = +1$), and R12 ($\Delta\text{Instr} = -2$) are classified under maintainability as they exhibit distinct adjustments in the logical instruction count. These structural shifts occur by design: operations like stage extraction and modularization naturally introduce an intentional structural overhead, a necessary operational cost to establish multi-stage boundaries and context isolation, whereas inlining or removal actions reduce instruction counts.

Maintainability, in terms of structural simplicity and readability, cannot be fully captured by standard artifact metrics. Through the analytical evaluation of the code's structure before and after the operation, we conclude that maintainability is enhanced, regardless of whether the instruction delta is positive or negative. For operations that increase instruction counts, this intentional addition is fully justified as the operational cost required to achieve direct gains in clarity, readability, and modular architecture, ultimately improving maintainability while preserving the integrity of the build process.

### 4.2.3 Distribution of the Catalogue

The high prevalence of positive Maintainability ($\uparrow$) impacts across the catalogue reflects the structural nature of these refactoring operations rather than a classification bias. Both empirical data and existing literature confirm that the catalogued transformations, such as stage naming, instruction formatting, variable extraction, and logic modularization, systematically target and enhance Dockerfile readability and organization. Where a refactoring produces direct benefits in Performance ($\uparrow$) or Security ($\uparrow$), the three-dimensional vector explicitly records it: Inline RUN Instructions (R01), Extract Stage (R05), and Sort Instructions (R07) deliver measurable Performance improvements, while Extract Stage (R05), Replace ADD with COPY (R08), and Update Base Image (R10) actively enhance Security. Consequently, the classification vector for each rule follows individual empirical and theoretical evidence, rather than an artificial target balance across the three dimensions.

## 4.3 VCS Connector

### 4.3.1 Technical Description

The VCS Connector is the entry point of the analysis pipeline. Its sole responsibility is to retrieve the complete textual content of the Dockerfile at each of the two specified commits, making it available to the Detection Engine as a UTF-8 string. The component interacts exclusively with the Git object model through GitPython, without modifying the repository state or writing any files to the filesystem.

Given a commit SHA, GitPython resolves it to a git. Commit object, from which the commit's tree is accessed. The tree is navigated to the blob corresponding to the Dockerfile path, and its byte content is read through the data_stream interface and decoded to UTF-8. This sequence of operations is idempotent: repeated calls with the same commit SHA always return identical strings, satisfying RNF5.

Table 4.4 compares the two access strategies considered for the VCS Connector:

**VCS Connector**

| Approach | Reproducible | Validated |
|---|---|---|
| **Shell git show** | ✗ Shell escaping artefacts | ✗ |
| **GitPython (selected)** | ✓ Programmatic blob access | ✓ Experiment 3 |

Table 4.4 - Version control system access methods considered for the VCS Connector

GitPython was selected over higher-level mining frameworks because the VCS Connector's task is narrow: retrieving the Dockerfile at two known commits. For this, GitPython's programmatic, blob-level access to file content at any commit, without a working-tree checkout, is sufficient and avoids the overhead of a full mining framework. Spadini et al. [33] characterise GitPython as the state-of-the-art Python framework for interacting with Git, and propose PyDriller as a more concise mining alternative; that alternative is unnecessary for this task. This access was validated in Experiment 3 against the docker/getting-started repository, satisfying RF1.

### 4.3.2 Design and Requirements

**Planned Changes**

In the current state of practice, accessing the Dockerfile at a historical commit requires manual invocation of git show <commit>:Dockerfile, followed by saving the output to a file for further processing. This workflow is error-prone, non-reproducible across shell environments, and unsuitable for automation. The VCS Connector replaces this workflow with a programmatic API call that returns the file content as a Python string directly, eliminating shell escaping artefacts and ensuring byte-identical retrieval.

**Challenges**

The principal challenge of this component is the handling of repositories where the Dockerfile is not located at the root of the repository tree, but within a subdirectory. The design accommodates this by accepting the Dockerfile path as a parameter rather than assuming a fixed location. A secondary challenge is the handling of commits where the Dockerfile did not yet exist; the component raises an informative exception in this case, allowing the pipeline to abort cleanly rather than producing an empty string that downstream components would misinterpret.

**Functional and Non-Functional Requirements**

The VCS Connector must satisfy RF1 fully: it must extract the Dockerfile content at both commits without requiring a working tree checkout. Non-functionally, it must satisfy RNF5 (reproducibility) by ensuring that the same commit SHA always yields the same output, and RNF4 (reliability) by raising specific exceptions for non-existent commits and non-existent Dockerfile paths.

**Design**

The component is implemented as a stateless function that accepts the repository path, SHA_A, SHA_B, and an optional Dockerfile path (defaulting to the repository root). It returns a named tuple containing Dockerfile_A and Dockerfile_B as strings. The stateless design ensures that the component has no side effects and can be tested in isolation with any Git repository.

## 4.4 Detection Engine

### 4.4.1 Technical Description

The Detection Engine is the analytical core of the pipeline. It receives the two Dockerfile strings from the VCS Connector and determines whether the structural change between them constitutes a recognised refactoring type. The engine uses dockerfile-parse to decompose each Dockerfile into an ordered list of logical instruction objects, each containing the instruction type and its normalised argument string, and then applies a set of detection rules, one per supported refactoring type.

The prototype implements automated detection rules for all fourteen refactoring types of the extended catalogue presented in Section 4.2, since the catalogue is adopted in full rather than as a selected subset.

Table 4.5 compares the two parsing strategies considered for the Detection Engine:

**Detection Engine**

| Approach | Multi-line RUN support | Python-native | Validated |
|---|---|---|---|
| **Regex line matching** | ✗ | ✓ | ✗ |
| **dockerfile-parse (selected)** | ✓ | ✓ | ✓ Experiment 5 |

Table 4.5 - Dockerfile parsing strategies considered for the Detection Engine

Regex-based line matching was rejected because Dockerfile instructions can span multiple physical lines through the backslash continuation character [7], preventing a line-level parser from reconstructing them into the canonical logical representations required by RF2. The adoption of structural parsing is consistent with the approach taken by DRMiner [9], which employs an Enhanced AST to capture the structural complexities of Dockerfile instructions. dockerfile-parse was selected and its ability to correctly reconstruct multi-line instructions was confirmed in Experiment 5, where four logical RUN instructions spread across multiple physical lines were correctly identified as four units, satisfying RF2.

### 4.4.2 Design and Requirements

#### Planned Changes

In the current state, the existence of a refactoring in a commit can only be identified through manual inspection of the diff, a process that Ksontini et al. [9] note is rarely documented by developers. The Detection Engine replaces manual inspection with automatic structural comparison, producing a typed result that names the refactoring and identifies the specific instructions involved.

#### Challenges

Beyond the multi-line parsing challenge resolved by dockerfile-parse (Section 4.1.3), the Detection Engine must resolve the consolidation-versus-deletion ambiguity described in Section 4.1.3. It also faces the challenge of concurrent rule activation: when a commit modifies the Dockerfile in a way that matches multiple detection rules simultaneously, the engine operates under a non-exclusion principle. In the prototype, rather than enforcing an arbitrary hierarchical filter that privileges one quality dimension over another, the system dynamically records all identified patterns into a cumulative sequence. This ensures that every applied optimization, whether targeting Security, Performance, or Maintainability, is fully captured and quantified. If a deterministic execution sequence is required by the programmatic pipeline, the engine relies strictly on the chronological order of the rule-registry index, completely decoupling the processing flow from any perceived severity of the quality dimensions.

#### Functional and Non-Functional Requirements

The Detection Engine must satisfy RF2: parse both Dockerfiles into logical representations and identify the refactoring type and affected instructions. It must satisfy RNF3 (extensibility) by implementing each rule as an independent function registered in a central rule registry, so that new types can be added without modifying existing rules.

### 4.4.3 Supported Refactoring Types

Before listing the types the prototype supports, it helps to position each one against the tools reviewed in Section 3.4. Table 4.6 maps the fourteen refactorings to the Hadolint rule they address, where one exists, to the coverage provided by DRMiner, and to the implementation

decision made for the prototype. Hadolint and DRMiner operate at different levels, so each column is read on its own terms: Hadolint reports individual smells on a single Dockerfile snapshot, while DRMiner is the only tool that detects refactorings across two revisions. The Hadolint column therefore names the smell a refactoring fixes, where one exists, rather than claiming Hadolint detects the refactoring itself.

Only three refactorings map directly to specific Hadolint rules: Inline RUN Instructions (DL3059), Update Base Image TAG (DL3007), and Replace ADD with COPY (DL3020) [10]. This limited mapping is expected, as Hadolint validates best-practice compliance within static snapshots, whereas most refactoring operations involve deeper structural transformations that transcend standard linting.

The Docker-Parfum column narrows this further. Docker-Parfum implements 32 smell detections and reimplements a subset of Hadolint rules among them, of which only DL3020 corresponds to any of the three rules above [22]. Replace ADD with COPY (R08) is therefore the single catalogue refactoring that Docker-Parfum both detects and repairs; for the remaining thirteen, no corresponding rule exists in its catalogue. This near-absence of overlap stems from a difference in the level at which each tool operates: Docker-Parfum targets smells located inside the arguments of shell commands, while the extended catalogue operates on the structure of the Dockerfile itself (the tag of a base image, the presence and arrangement of stages, the declaration of ENV and ARG variables, and the order of instructions) [22].

Regarding detection capabilities, DRMiner identifies twelve of the fourteen refactoring types through its Enhanced AST matching, with the remaining two, Remove RUN (mv command) and Update RUN Instruction, currently lacking automated coverage [9]. Nevertheless, the prototype implements detection rules for all fourteen types as they constitute the extended catalogue of common Dockerfile refactorings identified in the literature [2]. By implementing the complete set rather than a restricted subset, the prototype ensures that the catalogue's classification is applied consistently to every detected refactoring, providing a comprehensive and standardized assessment framework.

The prototype resolves the research gap identified in Chapter 3 regarding existing tools such as DRMiner [9]: the lack of automated coverage for the complete set of common Dockerfile refactorings. By implementing detection rules for all fourteen types of the extended catalogue, the tool identifies these refactorings and provides an evaluation of the Dockerfile across quality dimensions such as performance, security, and maintainability.

| **ID** | **Refactoring Type** | **Hadolint (smell fixed)** [10] | **DRMiner** [9] | **Docker-Parfum** [22] | **Implemented in Prototype** |
|---|---|---|---|---|---|
| **R01** | Inline RUN Instructions | DL3059 | Yes | No | Yes |

| ID | Refactoring Type | Hadolint (smell fixed) [10] | DRMiner [9] | Docker-Parfum [22] | Implemented in Prototype |
|---|---|---|---|---|---|
| **R02** | Update Base Image TAG | DL3007 | Yes | No | Yes |
| **R03** | Add ENV Variable | — | Yes | No | Yes |
| **R04** | Add ARG Instruction | — | Yes | No | Yes |
| **R05** | Extract Stage | — | Yes | No | Yes |
| **R06** | Inline Stage | — | Yes | No | Yes |
| **R07** | Sort Instructions | — | Yes | No | Yes |
| **R08** | Replace ADD with COPY | DL3020 | Yes | Yes | Yes |
| **R09** | Extract RUN Instructions | — | Yes | No | Yes |
| **R10** | Update Base Image | — | Yes | No | Yes |
| **R11** | Move Stage | — | Yes | No | Yes |
| **R12** | Remove RUN (mv command) | — | No | No | Yes |

| ID | Refactoring Type | Hadolint (smell fixed) [10] | DRMiner [9] | Docker-Parfum [22] | Implemented in Prototype |
|---|---|---|---|---|---|
| **R13** | Update RUN Instruction | — | No | No | Yes |
| **R14** | Rename Image | — | Yes | No | Yes |

Table 4.6 - Relationship between the fourteen catalogue refactorings and the reviewed tools (Hadolint reports smells; DRMiner detects refactorings), with the implementation decision for the prototype

## 4.5 Performance Analyzer

### 4.5.1 Technical Description

To retrieve image metadata within the containerized environment, the two official interfaces provided by the ecosystem were considered: the command-line interface (Docker CLI) and the software development kit (Docker SDK). In strict respect to the official Docker Engine API architecture [34], these two approaches represent the standard interfaces available to send instructions and interact with the Docker daemon. However, during the architectural investigation of these methods, a critical limitation in the Docker CLI regarding image size precision was identified. As documented by the developer community in Docker Issue #31298 [35], the CLI introduces decimal rounding and human-readable truncation (e.g., rounding byte counts to the nearest megabyte) by design. Since the non-functional requirement RNF1 demands byte-level precision to detect micro-variations and small size deltas, any reliance on the CLI was rejected.

Consequently, the Performance Analyzer measures the impact of the detected refactoring on the Performance dimension by building the Docker image from each Dockerfile state and communicating with the Docker daemon exclusively through the Docker SDK. It utilizes the image.attrs["Size"] attribute, which returns the VirtualSize, the total uncompressed byte count of all image layers, as a 64-bit integer. Experiment 4 confirmed that this direct query to the underlying API endpoint returns an exact byte count, 29,748,045 bytes (28.37 MB) for the documented public test image (ubuntu:22.04) used in that experiment, a precision level that would be rounded away by the CLI visual interface.

Build time was considered as an additional performance metric but was excluded from the prototype design. Its high sensitivity to environmental factors, host hardware, Docker cache state, network latency, makes a single measurement per state insufficient for reliable delta computation. Image size, by contrast, is a near-deterministic function of the Dockerfile content and the base image state; in practice, byte-level variations of a few bytes may occur across repeated builds in the same environment, as documented in Section 5.4.1.

Table 4.7 compares the two size retrieval methods considered for the Performance Analyzer:

**Performance Analyzer:**

| **Approach** | **Precision** | **Validated** |
|---|---|---|
| **Docker CLI (--format "{{.Size}}")** | ✗ Rounded human-readable strings | ✗ Docker issue #31298 [35] |
| **Docker SDK (selected)** | ✓ Raw 64-bit integer bytes | ✓ Experiment 4 |

Table 4.7 - Image size retrieval methods considered for the Performance Analyzer

The Docker CLI was rejected because it returns human-readable strings that introduce rounding imprecision, a behavior documented in the Docker project's own issue tracker. This was confirmed in the context of this prototype in Experiment 4, where the Docker SDK returned an exact byte count for a test image (29,748,045 bytes) rather than a rounded string. This byte-level precision is what allows small deltas, such as the −194,991 bytes measured for the Inline RUN Instructions refactoring in PoC-1 — to be computed correctly, where CLI rounding would reduce them to zero and violate RNF1.

### 4.5.2 Design and Requirements

**Planned Changes**

While a standard pipeline design might measure image size using the Docker CLI command docker images --format "{{.Size}}", technical documentation and open issues (such as Docker issue #31298) indicate that this approach returns a pre-parsed, human-readable string subject to rounding imprecision. To bypass this limitation from the start, the Performance Analyzer

replaces any reliance on the CLI with a direct SDK call that returns the raw integer byte count, completely eliminating the human-readable truncation risk

**Challenges**

The Performance Analyzer requires the Docker daemon to be running on the host system and the image build to complete successfully for both states. Builds that fail, due to network errors during package installation or invalid Dockerfile syntax, must be handled gracefully, with the component raising an informative exception that allows the pipeline to abort cleanly and report the failure accurately.

**Functional and Non-Functional Requirements**

The component must satisfy RF4 (byte-precise size retrieval) and RNF1 (metric precision sufficient to detect significant size reductions), as demonstrated by Experiment 4, where the Docker SDK successfully retrieved the exact, unrounded count of 29,748,045 bytes for the ubuntu:22.04 test image, preventing the human-readable rounding and truncation errors inherent in the Docker CLI which would approximate this value to 29 MB.

## 4.6 Data Extractor

### 4.6.1 Technical Description

The Data Extractor is responsible for collecting the Security and Maintainability metrics for both Dockerfile states. It invokes Hadolint [10] and Trivy [36] in parallel against each image state and normalises their heterogeneous JSON outputs into a common internal metric schema. The parallel execution is architecturally justified by the mutual independence of the two operations: Hadolint analyzes the Dockerfile text while Trivy scans the built image, and neither requires the result of the other. Running them concurrently reduces the total metric collection time to approximately the duration of the slower operation.

**Table 4.8 compares the two invocation strategies considered for the Data Extractor:**

**Data Extractor**

| **Approach** | **Total time** | **Validated** |
|---|---|---|
| **Sequential invocation** | ✗ Sum of both execution times | ✗ |
| **Parallel invocation (selected)** | ✓ Duration of the slower operation | ✓ Bass et al. [37] |

Table 4.8 - Tool invocation strategies considered for the Data Extractor.

Sequential invocation was rejected because it introduces unnecessary blocked time when two mutually independent operations run one after the other, a suboptimal design that Bass et al. identify as the precondition for applying the 'Introduce Concurrency' performance tactic [37]. Hadolint and Trivy satisfy this precondition: Hadolint performs static analysis on the Dockerfile text while Trivy scans the built image, meaning neither tool requires the output of the other as input. Parallel invocation was therefore selected, reducing total metric collection time from the sum of both execution times to the duration of the slower operation [37].

Both tools are executed as Docker containers rather than as locally installed binaries, using their official images (hadolint/hadolint and aquasec/trivy). This keeps the tool versions pinned and reproducible across environments and avoids host-level installation dependencies. The prototype invokes each container through the Docker daemon: Hadolint receives the Dockerfile text via standard input, while Trivy is given access to the built image to be scanned.

During data extraction, the tool executes external static analysis and security tools, collecting their raw report outputs in JSON format. While these raw artifacts are processed internally for metric extraction, they are optionally preserved for auditability and reproducibility purposes, enabling full transparency of the underlying analysis

**Functional and Non-Functional Requirements**

The Data Extractor must satisfy RF5: invoke both tools in JSON mode, normalise their schemas, and return warning and CVE count deltas. It must satisfy RNF2 (sensitivity to the complete elimination of relevant Hadolint rule violations), RNF4 (reliability through exception handling), and RNF5 (reproducibility through Trivy database version recording).

### 4.6.2 Security Metrics — Trivy

Trivy is invoked in image-scanning mode with the --format json flag. The component navigates the resulting Results[].Vulnerabilities[] structure and counts occurrences by severity tier (CRITICAL, HIGH, MEDIUM, LOW, UNKNOWN), producing a dictionary that represents the security profile of each image state. Experiment 2 confirmed the stability of this schema. The PoC-3 proof-of-concept experiment confirmed Trivy's diagnostic sensitivity: it correctly identified the vulnerabilities present in the ubuntu:22.04 base image and the smaller set present in the minimal alpine:3.19 image, capturing a Delta CVEs of -15 and demonstrating that the choice of base image is a primary driver of an image's security profile.

Trivy was selected as the security instrument because it is the most widely adopted open-source container vulnerability scanner, used as the default in projects such as Harbor and certified by Red Hat [36]. Alternatives such as Grype exist, but Trivy was chosen for its broader coverage across OS and language packages.

### 4.6.3 Maintainability Metrics — Hadolint

Hadolint [10] is invoked via standard input pipe with the --format json flag. The component parses the resulting flat array of warning objects and applies the maintainability screen defined in Section 4.1.3, retaining the 47 default-enabled Hadolint and ShellCheck rules that target maintainability and discarding those whose effect belongs to the performance or security dimensions, since those effects are already measured by ΔSize and ΔCVEs. The screen is held in a single declarative rule list, so it can be inspected and revised without touching the parsing logic. For the proof-of-concept runs of Section 4.1.2 the list is bypassed and the raw total is recorded instead, in line with the two-regime distinction stated in Section 4.1.3.

It is explicitly acknowledged that Hadolint functions as a proxy for maintainability rather than a direct measurement thereof, as its rules encode conformance to documented best practices [7] rather than the full range of maintainability characteristics. This operationalisation is consistent with Wu et al. [8], who used Hadolint-detected smells as empirical indicators of Dockerfile quality issues. It is also why Hadolint is not the only maintainability indicator: as described in Section 4.1.3, the warning count is complemented by the logical instruction count (ΔInstr), which reports structural change at a level the rule set does not reach. The two are collected by the same component, the Data Extractor, and are reported side by side: Hadolint through an external invocation, ΔInstr through a direct count over the parsed instruction list.. This limitation is declared in Section 4.9.

Hadolint was selected as the maintainability instrument because it is the reference tool used in the literature for detecting Dockerfile smells, adopted both in empirical studies and in enterprise code quality tooling [17].

### 4.6.4 Maintainability Metrics — Logical Instruction Count

The second maintainability indicator requires no external tool. The component parses each Dockerfile state with dockerfile-parse, the same library used by the Detection Engine, and counts the resulting logical instructions, excluding COMMENT entries, which the parser reports alongside instructions but which are not instructions themselves. Because the parser reconstructs multi-line instructions into a single logical unit, a RUN split across several physical lines with backslash continuations counts as one, which is what makes the count insensitive to formatting.

The count is returned for both states as part of the MetricSet, and the Report Generator computes the signed difference (ΔInstr) in the same way it computes the other deltas. The rationale for this indicator, and the reading its non-monotonic behavior requires, are set out in Section 4.1.3.

## 4.7 Report Generator

### 4.7.1 Technical Description

The Report Generator is the terminal stage of the pipeline. It consumes the DetectionResult produced by the Detection Engine and the two MetricSet objects produced by the Performance Analyzer and the Data Extractor, computes the signed delta for each quality metric, and produces the ImpactReport. Its role is aggregation, not interpretation: it pairs the formal identifier of the detected refactoring with the four measured deltas exactly as they were obtained, and adds no expected direction, no dimensional label, and no judgement about whether the result matches the catalogue.

The sign convention is uniform for the three metrics that measure the built image and the smell count: a negative delta indicates improvement (smaller image, fewer CVEs, fewer warnings). For Security, the report includes a net-change per-severity delta vector rather than a single aggregate, enabling detection of cases where the total CVE count improves but the severity distribution worsens

ΔInstr is reported under a contextual reading rather than as a scalar regression indicator. As set out in Section 4.1.3, refactorings act on the instruction count in opposite directions by design: consolidation and removal reduce it, whereas extraction and addition increase it. Consequently, a positive ΔInstr is not inherently interpreted as a quality degradation. The report presents ΔInstr alongside the specific refactoring type that produced it, ensuring the metric is evaluated against the nature of the applied operation rather than against an arbitrary fixed target.

Table 4.9 compares the two output formats considered for the Report Generator:

**Report Generator**

| **Approach** | **Structured output** | **Validated** |
|---|---|---|
| **Plain text** | ✗ | ✗ |
| **JSON (selected)** | ☑ | ☑ RFC 8259 [38] |

Table 4.9 - Output formats considered for the Report Generator.

JSON was selected as the primary output format instead of unstructured alternatives (e.g., plain text), because it is a standardised lightweight format designed specifically for the portable representation of structured data [38], satisfying RF7, which requires structured output for programmatic processing, and RNF5, which requires identifiable fields for the analysis timestamp.

**Functional and Non-Functional Requirements**

The Report Generator must satisfy RF7: produce a structured output presenting before-and-after values along with the computed metric deltas. The report is serialised in JSON format for downstream processing and in a human-readable text summary for direct review by practitioners, alongside the execution console logs. Both formats include the analysis timestamp and the commit SHAs to satisfy RNF5 (reproducibility).

## 4.8 System Data Flow

This section traces the complete data flow of the system from user inputs to the final ImpactReport, identifying the typed data structures exchanged at each component boundary.

Table 4.10 summarises the typed data transformations at each pipeline stage:

| Stage | Components | Data Transformation |
|---|---|---|
| **1** | VCS Connector | Input: Repository/SHA. Output: Dockerfile text. |
| **2** | Detection Engine | Input: Dockerfiles. Output: DetectionResult. |
| **3** | Performance Analyzer, Data Extractor | Input: Dockerfiles. Output: Metrics (MetricSet) / Deltas. |
| **4** | Report Generator | Input: DetectionResult + MetricSet. Output: ImpactReport (JSON). |

Table 4.10 - Data transformations at each pipeline stage.

## 4.9 Design Limitations and Mitigations

The prototype implements all fourteen refactoring types catalogued by Ksontini et al. [2]. Four of them were exercised in isolated proof-of-concept experiments to establish the measurement baselines; the empirical evaluation of the full set is addressed in Chapter 6.

The four refactoring types exercised in the proof-of-concept experiments, Inline RUN Instructions, Update Base Image TAG, Update Base Image, and Replace ADD with COPY, were

selected as representative cases of the quality dimension profiles supported by the catalogue. Together they exercise the performance, security, and maintainability dimensions, providing empirical evidence that the three-axis measurement principle holds across distinct refactoring profiles, including refactorings that affect a single dimension and refactorings that affect more than one.

The empirical feasibility of each selected type was validated through four isolated proof-of-concept experiments, one per type, producing concrete baseline values for each delta metric. These baselines are documented in Section 4.1.2 and serve as acceptance criteria for the verification strategy described in Section 4.1.6.

The consolidation-versus-deletion ambiguity described in Section 4.1.3 is mitigated by the content preservation check but not fully resolved. Commits where both consolidation and deletion occur simultaneously produce a mixed-change result that is explicitly flagged in the report as requiring human validation. This limitation is shared with DRMiner [9], which also cannot deterministically resolve this class of ambiguity.

Build time is excluded from the Performance dimension due to its high environmental sensitivity, as discussed in Section 4.5.1. The Performance dimension is therefore operationalised exclusively through image size. Incorporating build time in a statistically reliable way would require multiple measurements per state and a controlled environment; this is identified as a direction for future work.

The use of Hadolint [10] as a proxy for maintainability is a methodological simplification consistent with Wu et al. [8] and Rosa et al. [17], and declared as a threat to validity in Chapter 6. The addition of ΔInstr reduces, but does not remove, the exposure of the maintainability dimension to that simplification. Three limitations of the structural indicator are acknowledged. First, it measures structure and not maintainability itself: fewer instructions indicate a simpler Dockerfile, following the use of instruction count as a complexity proxy by Wu et al. [8], but simplicity and maintainability are related rather than equivalent. Second, it is not monotonic, so its sign cannot be interpreted without knowing which refactoring produced it. Third, it is blind to refactorings that leave the instruction count unchanged, which is the case for Sort Instructions (R07), Update RUN Instruction (R13), and Rename Image (R14); for the last two, no metric adopted here registers their effect, and the maintainability classification rests on the nature of the operation as discussed in Section 4.2.3.

The prototype requires a locally running Docker daemon and a pre-cloned repository as operational pre-conditions. It does not include a graphical interface, batch processing, or CI/CD integration. These capabilities are identified as future work in Chapter 7.

The prototype's measurement scope is strictly bound to the Dockerfile path specified for the tracked artifact (RF1). Restricting the tracking scope to a single file path constitutes a deliberate design decision aimed at ensuring a strictly deterministic measurement pipeline, avoiding speculative heuristics that would be necessary for the automatic aggregation of satellite files (e.g., Dockerfile.builder). While this approach covers the vast majority of the case studies (14 out of 15 scenarios), structural multi-file refactorings fall outside the current

functional scope. As observed in the experiments, this design is transparent. Refactorings like R09 (Extract RUN) show no divergence, since they offload logic to shell scripts and a shell script does not count as a Dockerfile instruction, so the count never had it to lose. Refactorings like R11 (Move Stage) diverge from the global catalog accounting in instruction count (ΔInstr), since they create a new, separate Dockerfile that falls outside what the prototype reads. This limitation is consciously assumed to safeguard the internal validity of the impact analysis, because the alternative is a heuristic that aggregates satellite files, and such a heuristic would trade determinism for coverage the prototype cannot verify. It represents a structural property of the prototype's scope, which audits the primary artifact's evolution rather than calculating total project state changes, and relegates support for multi-file repositories to future research directions.

A further key methodological disclaimer acknowledged in this design framework concerns the contextual and volatile nature of external container environments during local dynamic assessment. So, the extended catalogue (Section 4.2) maps out those theoretical quality directions expected from each refactoring type based on the empirical literature, but the experimental deltas coming from a real-world execution still depend heavily on the operational context. Transient network latency during RUN instructions, untracked updates within remote package manager mirrors such as unexpected software upgrades during an apt-get execution, and silent baseline adjustments in external images maintained by public registries are all external variables entirely decoupled from the Dockerfile's syntax, and all three introduce confounding factors that temporarily perturb the collected performance (ΔSize) and security (ΔCVEs) indicators.

The primary validation criterion for the prototype's effectiveness is its structural detection accuracy, which remains unaffected by external environmental variables. Localized deviations or anomalies within the empirical quality deltas do not compromise system precision, as measurements are exact for the artifact built; consequently, observed variations reflect changes in the artifact rather than instrument error. Rather than systemic flaws, these instances serve as useful indicators of Infrastructure-as-Code ecosystem volatility, highlighting real-world edge cases where theoretically sound refactoring patterns encounter conflicting practical outcomes.

A critical operational consideration during the automated execution of the prototype concerns the availability of base images within the Docker daemon. While public images (such as node:12.0 or official distribution tags) are automatically fetched from remote container registries if absent from the host, custom or project-specific base images must be pre-existent in the local Docker cache.

If a target Dockerfile references a custom local image that has not been previously built, the preparation build phase will fail with an image-not-found error. In real-world multi-component repositories, target projects typically provide build orchestration files, such as Makefile instructions, outlining the sequence required to compile local dependencies. Consequently, users must ensure that any specialized or custom base images are successfully

built and cached locally prior to launching the analysis pipeline, serving as an operational prerequisite for the tool's successful execution.

Vulnerability Database Volatility: It is important to note that security metrics ($\Delta$CVEs) are inherently subject to external temporal variations. Because vulnerability scanners such as Trivy update their Common Vulnerabilities and Exposures (CVE) databases daily, scanning the exact same image state at different dates may yield differing absolute vulnerability counts as new CVEs are continuously discovered and catalogued. To ensure full transparency and auditability, the prototype's reporting mechanism (RNF5) systematically records the precise timestamp of the Trivy vulnerability database version used during each execution. Consequently, any variation in CVE counts across runs performed on different dates is attributable to external database updates rather than inconsistencies in the prototype's detection or measurement pipeline.

Operational Pre-conditions and Scope Nuance: The execution environment requires a locally running Docker daemon supporting the Docker Buildx plugin, alongside active network connectivity to pull the external Hadolint and Trivy tool images if they are absent from the local daemon cache. However, a critical structural nuance must be highlighted: if the analysis is restricted strictly to refactoring detection without empirical impact measurement, the architecture relies exclusively on Python 3.12+, git, and the dependencies listed in requirements.txt. The pure detection engine operates entirely in native Python logic, allowing core code structures and test suites to execute without any Docker dependency. Docker, Buildx, and containerized tools are engaged exclusively during the impact measurement stages.

## 4.10 Summary

This chapter presented the complete design of the prototype tool for automated Dockerfile refactoring detection and multi-dimensional quality impact assessment. The architecture is a sequential data pipeline of five components, VCS Connector, Detection Engine, Performance Analyzer, Data Extractor, and Report Generator, each with a clearly bounded responsibility, a typed interface, and a directly traceable validation experiment.

The VCS Connector design is grounded in Experiment 3 (GitPython, commits 2bca273 and 2981665, docker/getting-started). The Detection Engine design is grounded in Experiment 5 (dockerfile-parse, 4-to-1 RUN consolidation detected automatically). The Performance Analyzer design is grounded in Experiment 4 (Docker SDK, 29,748,045 bytes retrieved via daemon API). The Data Extractor design is grounded in Experiment 2 (Hadolint and Trivy JSON schemas validated).

The central contribution of the design is the introduction of the delta (Delta) as the analytical unit of quality communication. The four isolated proof-of-concept experiments demonstrated that the supported refactoring types produce measurable and distinct values across the four deltas and the three quality dimensions they operationalise, with different refactorings

affecting different combinations of those dimensions. Inline RUN Instructions produced Delta Size = −194,991 bytes, ΔWarnings (total raw) = −7 and Delta Instr = −3, confirming both the complete elimination of the consecutive-RUN smell and the structural consolidation that produced it. Update Base Image TAG produced Delta Warnings(DL3007) = −1, eliminating the latest-tag smell. Update Base Image produced Delta Size = −26,318,550 bytes and Delta CVEs = −15, showing that the choice of base image moves performance and security simultaneously. Replace ADD with COPY produced Delta Warnings(DL3020) = −1 with negligible size and no security impact. Across this set, the evaluated refactorings produced clear, measurable changes in their respective dimensions without introducing unexpected cross-dimensional conflicts. The prototype automates the measurement of these deltas between commits, addressing the inability of existing tools to objectively quantify the actual impact of detected refactorings. The following chapter describes how this design was realised in code.

# 5 Implementation

This chapter describes how the prototype designed in Chapter 4 was built, and how each design decision was carried into code.

## 5.1 Implementation Overview and Prototype Orchestration

The prototype was implemented in Python 3.14.4. The five components defined in Section 4.1.6 are realised as five modules under src/, accompanied by two modules that belong to the orchestration rather than to the architecture: pipeline.py, which chains the components and manages the resources they consume, and image_builder.py, which builds the Docker images the measurement components read. Listing 5.1 shows the structure of the repository.

```
.
├── src/
│   ├── vcs_connector.py        # component
│   ├── detection_engine.py     # component
│   ├── performance_analyzer.py  # component
│   ├── data_extractor.py       # component
│   ├── report_generator.py     # component
│   ├── image_builder.py        # preparation phase
│   └── pipeline.py             # orchestration
├── tests/
├── experiments/
├── main.py
├── conftest.py
└── requirements.txt
```

Listing 5.1 - Structure of the prototype repository

The experiments/ directory holds the Dockerfile pairs, measurement scripts and raw outputs of the catalogue experiments and the four proofs of concept, so that every value reported in Chapter 4 can be inspected and reproduced. The empty conftest.py at the root marks the project root for pytest, making src/ importable from the test suite without installation.

### 5.1.1 Stage structure

Each component was built first as an independent library with a single public function, and none imports another. For a period the pipeline therefore existed as a design and as throwaway validation scripts, but not as code in the repository. src/pipeline.py closes that gap by holding the orchestration as an importable function, while main.py provides the command-line entry point. The split is deliberate: it keeps the sequencing testable without invoking a process.

Operational environment note: The prototype must be executed within a native Linux or WSL2 (Windows Subsystem for Linux) environment where the Docker daemon is locally accessible. Running the tool directly through a Windows Python interpreter will fail to communicate with the Unix socket hosted inside the Linux container daemon.

The five components execute across four sequential stages. The stages run strictly in order and each is separated from the next by a typed boundary, so that the output type of a stage is the input type of its successor and the components compose in exactly one way. Table 5.1 summarises the structure.

| Stage | Component | Responsibility |
|---|---|---|
| **1** | VCS Connector | Retrieval of the two Dockerfile states |
| **2** | Detection Engine | Identification of the applied refactoring |
| **3** | Performance Analyzer | Measurement of ΔSize |
| **3** | Data Extractor | Measurement of ΔCVEs, ΔWarnings and ΔInstr |
| **4** | Report Generator | Aggregation and writing of the output artifacts |

Table 5.1 - The five components of the prototype and the stage in which each executes

Stage 3 is preceded by a preparation phase that is not a component. Exporting the file trees of the two commits and building the corresponding images are operations the orchestrator performs on behalf of both measurement components, using pipeline.commit_context and image_builder.py. The decision to place them outside the component set follows from their relationship to the architecture: they produce no metric, satisfy no functional requirement of their own, and exist only because both measuring components need the same resource. Assigning the build to either component would make the other wait on it, turning a concurrent stage into a sequential one; giving each its own build would duplicate the most expensive operation in the pipeline. Lifting it into the orchestrator resolves both, and keeps the component set identical to the one specified in Section 4.1.6.

The orchestrator owns the sequence and the components remain ignorant of it. Ordering, resource lifetime and the concurrency internal to Stage 3 live in one place, which is what keeps the components independently testable and independently replaceable, and what allows the extensibility required by RNF3 to be exercised without touching an orchestration layer.

Stage 1 — Data collection. The VCS Connector accesses the Git object database in isolation and retrieves the Dockerfile content at each of the two specified commits at blob level. No checkout is performed on the local working tree and no file is written to the caller's filesystem, in accordance with RF1. The stage yields the two states as UTF-8 strings.

Stage 2 — Refactoring detection. The Detection Engine receives the two textual contents and uses dockerfile-parse to decompose each state into an ordered list of logical instructions, so that an instruction split across several physical lines by backslash continuation resolves to a single unit. The fourteen catalogue rules are then applied to the pair of lists, and the stage yields a DetectionResult carrying the identifier of each refactoring recognised and the instructions involved. Where no supported refactoring is recognised, the result is empty and the analysis continues: the absence of a detection is a finding, not a failure.

Stage 3 — Empirical evaluation and measurement. The orchestrator first exports the file tree of each commit into its own temporary directory and builds the corresponding pair of Docker images, holding both exports and the build in a single contextlib.ExitStack. The mechanism is described in Section 5.1.2.

The two measurement components then run over the images just built. The Performance Analyzer reads the size of each image through the Docker SDK and returns ΔSize; the Data Extractor invokes Trivy against the built images for ΔCVEs, Hadolint against the Dockerfile text for ΔWarnings, and dockerfile-parse over the two states for ΔInstr. Neither depends on the result of the other, so both are submitted to a two-worker ThreadPoolExecutor against the same image references. Both futures are resolved before either result is used, so a failure in one does not leave the other running past the exit of the preparation context. The images are built once, before the measurement begins, and are released only when the stage closes, whatever happened inside it.

Neither measuring component receives the DetectionResult produced in Stage 2. Both operate on the Dockerfile pair retrieved in Stage 1 and on the prepared images, so the deltas are obtained without any knowledge of which refactoring was detected. This preserves in code the separation between detection and measurement established in Section 4.1.1.

Stage 4 — Report generation. The Report Generator aggregates the metrics produced in Stage 3 with the identifier produced in Stage 2, compiles the technical provenance block, and writes the final artifacts: the structured JSON report and the human-readable summary, together with the retained tool evidence. Provenance is gathered after the measurements rather than before, because Trivy downloads its vulnerability database into the shared cache during the first scan, and a version query made beforehand would report it as absent. Collecting the tool versions inside the preparation context, once Stage 3 has completed, makes the recorded database version the one the scans actually used, which is what RNF5 requires.

### 5.1.2 Historical commit contexts

A Dockerfile carrying a COPY instruction cannot be built without the files that instruction reads, yet the VCS Connector retrieves Dockerfile content at blob level without a checkout (RF1) and no component reconstructs a full tree. The first orchestrator resolved this by passing the working tree as it stands to both builds, which introduced a soundness defect with two silent consequences: a file that changed alongside the refactoring reached both measurements, so the size delta carried that difference as well as the refactoring's; and the earlier state was built against files that may not have existed at its commit, which is not a measurement of that state at all.

The defect was removed by giving each build its own historical context. The context manager pipeline.commit_context(repository, sha) exports the file tree of one commit into a temporary directory, so that each Dockerfile state is built against the repository exactly as it stood when that state was committed. The export uses repo.archive(stream, treeish=sha, format="tar"), which writes the tree as an in-memory tar stream and touches neither the working tree nor the index. This keeps the mechanism consistent with the blob-level access performed in Stage 1: nothing is checked out, and the repository is left exactly as it was found.

Two safeguards accompany the export. The archive is unpacked with extractall(destination, filter="data"), a strict filter that rejects absolute paths, parent-directory traversal, links pointing outside the destination, and device nodes, so that a crafted repository cannot write beyond the temporary directory. The tempfile.TemporaryDirectory is created outside the try block and released in its finally, so the directory is removed whether the export fails, the caller raises, or the block ends normally.

The consequence for the build is that build_image_pair receives context_before and context_after rather than a single context path, since the whole point of the change is that the two builds no longer share one. The orchestrator opens both exports and the build inside a single contextlib.ExitStack, which holds the directories for as long as the builds need them and unwinds in reverse order on the way out: the images are removed first, then the trees that produced them. The images therefore remain alive for the whole of the measurement phase and are released only once Stage 3 closes.

The --context command-line flag and the corresponding build_context parameter were removed as part of this change. With per-commit contexts in place, the only remaining effect of that option would have been to reintroduce the defect the change corrects, and an option whose sole outcome is an unsound measurement is worse than no option at all.

The effect was verified end to end. A repository was built with two commits whose committed payload.txt is 6 bytes, after which the working-tree copy was replaced with a 405,264-byte file that was never committed. Analysed through the command line, the images measured 3,632,156 and 3,632,741 bytes, the size of the committed payload. Had the working tree been used, both would have carried the additional 400 KB and the measurement would have been meaningless. No temporary directory survived the run.

### 5.1.3 Failure handling and exit codes

RNF4 requires that external invocations raise distinct informative exceptions, and each component honours that requirement within its own boundary. At the process boundary, however, distinct exception types collapse into a single non-zero exit unless they are mapped, and the diagnostic value is lost. The entry point therefore assigns each failure its own exit code, as shown in Table 5.2.

| Exit code | Stage | Origin | Exception |
|---|---|---|---|
| **0** | — | Analysis completed | — |
| **1** | 1 | VCS Connector | VcsConnectorError |
| **2** | 2 | Detection Engine | DockerfileParseError |
| **3** | 3 | Preparation phase | CommitContextError, ImageBuildError |
| **4** | 3 | Performance Analyzer | PerformanceAnalyzerError |
| **5** | 3 | Data Extractor | DataExtractorError |

| Exit code | Stage | Origin | Exception |
|---|---|---|---|
| **6** | 4 | Report Generator | Report writing failure |

Table 5.2 - Mapping of failures to process exit codes

Three of the six codes fall within Stage 3, which reflects the fact that it is the only stage combining a preparation phase with two concurrent components. The three are genuinely different events and worth distinguishing at the process boundary: a build that does not complete aborts the analysis before any measurement is attempted, whereas a size that cannot be inspected or a scan that cannot complete fails one metric of a stage already running. Collapsing them into a single stage-level code would discard the diagnostic value that RNF4 exists to provide.

CommitContextError was introduced with the historical contexts and covers three conditions: a repository that cannot be opened, a treeish that Git does not recognise, and an archive that cannot be unpacked. It ensures that a raw GitPython or tarfile error never escapes the orchestrator, and it maps to exit code 3 alongside ImageBuildError, since both belong to the preparation phase and both abort the analysis before either component runs.

The mapping was exercised against a real fault rather than a simulated one. A transient Trivy database download failure during an end-to-end run surfaced as DataExtractorError naming the failing scan and exited 5, with the RNF4 contract and the exit-code mapping both behaving as specified.

### **5.1.4** Technologies and execution

Each component was implemented against the technology validated for it during the rapid experimentation phase, so the prototype is built directly on top of the experimental scripts rather than on a re-selected stack. Table 5.3 lists the technologies in use and the role each one performs.

To execute the analysis pipeline successfully, the underlying infrastructure relies on three independent operational tiers rather than a single package list. First, system-level prerequisites must be installed beforehand: Python 3.12+ (due to archive extraction security filters using extractall(..., filter="data")), the native git binary (as GitPython requires it to invoke git archive for tree exports without local checkouts), a running Docker Engine, and the docker-buildx-plugin (since builds are executed via subprocess invoking docker buildx build --load to support advanced BuildKit directives). Second, Python-layer dependencies are handled via pip install -r requirements.txt, covering modules such as GitPython, dockerfile-parse, and the docker SDK. Third, external tools like Hadolint and Trivy do not require local host installation; they are automatically managed and executed as pinned Docker containers (hadolint/hadolint and aquasec/trivy), depending only on network access to pull images if absent from the local cache

| Stage | Component or phase | External technology | Version | Role |
|---|---|---|---|---|
| **1** | VCS Connector | GitPython | 3.1.50 | Blob-level retrieval from the Git object database |
| **2** | Detection Engine | dockerfile-parse | 2.0.1 | Decomposition into logical instructions for rule matching |
| **3** | Preparation phase | GitPython | 3.1.50 | Export of each commit's file tree as a tar stream |
| **3** | Preparation phase | Docker Buildx (CLI subprocess) | v0.36.1 | Image construction with BuildKit support (--chmod, heredocs) |
| **3** | Preparation phase | Docker SDK for Python | 7.1.0 | SDK-based container management and client connection |
| **3** | Performance Analyzer | Docker SDK for Python | 7.1.0 | Byte-precise image size retrieval (ΔSize) |
| **3** | Data Extractor | Trivy | 0.72.0 | Vulnerability scanning of the built images (ΔCVEs) |
| **3** | Data Extractor | Hadolint | 2.14.0 | Static analysis of the Dockerfile text (ΔWarnings) |
| **3** | Data Extractor | dockerfile-parse | 2.0.1 | Logical instruction count (ΔInstr) |
| **4** | Report Generator | none | — | Internal aggregation logic only |
| — | Execution environment | Docker Engine | 29.1.3 | Daemon hosting builds, scans and tool containers |

Table 5.3 - External technologies used by each component and by the preparation phase, with the role each one performs

The dockerfile-parse library appears twice, and the duplication is intentional. The Detection Engine parses each Dockerfile state to obtain the ordered list of logical instructions its rules match against, while the Data Extractor parses the same states independently to count them for ΔInstr. Neither invocation depends on the other, and the cost of parsing the pair twice is negligible against the build and scan operations that dominate the pipeline. Paying it preserves the independence of the measurement from the detection, which is the property Section 4.1.6 requires.

Hadolint and Trivy are absent from the Python dependency set because they are not Python libraries: both execute as pinned Docker containers rather than as host binaries, as described in Section 5.4.2. The declared dependencies are therefore GitPython, dockerfile-parse, docker and pytest, held in requirements.txt at the repository root.

An analysis is run by specifying the repository, the two commits, and an output directory, alongside optional configuration flags for the Dockerfile path (-d) and the build context directory (-c), as shown in Listing 5.2.

```
python3 main.py [-h] [-o OUTPUT] [-d DOCKERFILE] [-c PATH] repository commit_before commit_after
```

Listing 5.2 - Command-line invocation of the prototype

The tool accepts a total of seven parameters, three mandatory positional arguments and four optional flags:

Positional Arguments

- **repository**: Path to a local, pre-cloned Git repository.
- **commit_before**: SHA of the preceding commit (abbreviated forms accepted).
- **commit_after**: SHA of the succeeding commit.

Optional Flags

- **-h, --help**: Shows the help message and exits.
- **-o, --output** (default: ./impact_report): Target directory for the generated impact report and raw artifacts.
- **-d, --dockerfile** (default: Dockerfile): Path of the evaluated Dockerfile, relative to the repository root.
- **-c, --context** (default: automatic / parent directory of the Dockerfile): Explicit build context directory relative to the repository root. When omitted, the system automatically adopts the parent directory where the Dockerfile is located, resolving paths strictly within each commit's exported tree to preserve historical isolation.

Run against a repository holding the R09 catalogue pair, this invocation reported R09 Extract RUN Instructions with ΔSize +410, ΔCVEs +0, ΔWarnings +0 and ΔInstr +1, exited 0, and wrote five files: impact_report.json, human_readable_summary.txt, terminal_output.log and under

raw_data/ the retained Trivy and Hadolint evidence. The environment block recorded Docker Engine 29.1.3, Docker Buildx v0.36.1, Hadolint 2.14.0, Trivy 0.72.0, the Trivy database timestamp, dockerfile-parse 2.0.1, GitPython 3.1.50 and Docker SDK 7.1.0, satisfying the provenance requirement of RNF5

While the invocation above illustrates a targeted spot-check against an individual refactoring case, the overall verification of the tool is established through a comprehensive test suite covering the entire application logic rather than a single scenario. The declared dependencies are therefore GitPython, dockerfile-parse, docker, pytest, and pytest-cov, held in requirements.txt at the repository root. To facilitate the reproduction of results bygetPackage/evaluation committee, the complete test suite and coverage audit can be executed from a clean repository clone using Python 3.12 or higher (owing to archive extraction security filters) via pip install -r requirements.txt followed by python -m pytest --cov=src. As detailed in Section 5.6.2, this execution will successfully validate all components of the tool, yielding 302 passing tests, with 0 skipped and 0 failures under a fully configured environment.

### 5.1.5 Scope of the implementation

The scope of the implementation is deliberately narrower than the repositories it analyses. The prototype and its fourteen rules operate on single-Dockerfile refactorings: detect_refactorings(dockerfile_a, dockerfile_b) takes two strings and nothing else, all fourteen rule functions receive exactly two instruction lists for one file, and the Detection Engine performs no filesystem access at all, so a second Dockerfile is unreachable rather than merely unused. This is a conscious limitation rather than an oversight. Most of the catalogue operates over the internal structure of one file, and cases involving the coordinated refactoring of several Dockerfiles in a single commit demand a relational analysis that exceeds the scope of a research prototype. Move Stage (R11), which extracts a build stage into a separate Dockerfile, marks the boundary of that decision, and its consequence for ΔInstr is discussed in Section 5.4.2.

Verification was structured on two levels. Dedicated unit tests validate the isolated behaviour of each component, and integration tests verify the interaction and data flow between them, so that the internal integrity of the pipeline is guaranteed and every functional and non-functional requirement is exercised rather than assumed. The orchestration itself is covered by 28 tests in tests/test_pipeline.py, in which the five components are replaced by fakes so that the wiring is exercised without a daemon and without repeating what each component's own suite already covers: the historical commit export, the stage sequencing, the concurrency internal to Stage 3, the lifetime of the images and the temporary trees, and the exit-code mapping. The components were additionally checked programmatically against the design chapter rather than by reading, comparing implemented interfaces and metric names with those specified. The full inventory and the results of that verification are presented in Section 5.6.

## 5.2 VCS Connector

The VCS Connector is the entry point of the pipeline and the only component that touches the repository under analysis. Given a repository path and two commit identifiers, it retrieves the textual content of the Dockerfile at each commit directly from the Git object database, performing no working-tree checkout and writing nothing to the filesystem, as RF1 requires. It constitutes Stage 1 of the four-stage execution described in Section 5.1.1.

The component was implemented with GitPython 3.1.50, the technology validated in Experiment 3 and selected in Section 4.3. Programmatic blob-level access is reproducible across environments and avoids the shell-escaping artefacts that invoking git show would introduce, while a full mining framework such as PyDriller is unnecessary for the narrow task of retrieving one file at two known commits.

### 5.2.1 Implementation

The component is realised as a single stateless function, matching the design specified in Section 4.3:

```
retrieve_dockerfile_pair(repo_path, sha_a, sha_b, dockerfile_path="Dockerfile")
```

Listing 5.3 - Main function signature and entry point of the VCS Connector

It returns a DockerfilePair named tuple holding the two states as UTF-8 strings. The function has no side effects and holds no state between calls, which is what allows it to be exercised in isolation against any Git repository.

Three decisions carried the specification into code. The blob is read through the data_stream byte-stream interface and decoded to UTF-8, which is the retrieval path validated in Experiment 3 and the reason the operation never materialises a file. The dockerfile_path argument is a parameter rather than a fixed location, defaulting to Dockerfile at the repository root but accepting any path, so that repositories keeping the Dockerfile in a subdirectory are supported, the principal challenge identified for this component in Section 4.3. Finally, failures raise specific exceptions instead of returning empty strings: RepositoryNotFoundError, CommitNotFoundError and DockerfileNotFoundError, all deriving from a common VcsConnectorError base, so that the orchestrator can abort cleanly and map the failure to exit code 1, as described in Section 5.1.3.

Listing 5.4 shows the retrieval of a single state, with the two failure modes made explicit at the point where each can occur.

```
def _read_dockerfile_at_commit(repo: git.Repo, sha: str, dockerfile_path: str) -> str:
    try:
        commit = repo.commit(sha)
    except (git.BadName, ValueError, git.GitCommandError) as exc:
```

```
        raise CommitNotFoundError(
            f"Commit '{sha}' does not exist in repository '{repo.working_dir}'."
        ) from exc
    try:
        blob = commit.tree[dockerfile_path]
    except KeyError as exc:
        raise DockerfileNotFoundError(
            f"'{dockerfile_path}' does not exist at commit '{sha}' ({commit.hexsha[:7]})."
        ) from exc
    return blob.data_stream.read().decode("utf-8")
```

Listing 5.4 - Retrieval of the Dockerfile content at a single commit

Two properties of this function are worth noting. The commit is resolved before the tree is navigated, so a commit that does not exist and a Dockerfile that does not exist at an otherwise valid commit produce distinct exceptions rather than a single generic error, which is the granularity RNF4 asks for. And every exception is chained with from exc, so the originating GitPython error remains available for diagnosis while the caller sees only the component's own exception type.

The retrieval mechanism is also what the preparation phase of Stage 3 was later built to match. When per-commit build contexts were introduced, the export of each commit's file tree was implemented through repo.archive precisely so that it would touch neither the working tree nor the index, keeping the whole pipeline consistent with the no-checkout guarantee this component establishes. That mechanism is described in Section 5.1.2.

### 5.2.2 Line endings on Windows

The first test run failed on Windows, and the cause is worth recording because it fixed an assumption for every downstream component.

The test fixture wrote its Dockerfiles with Path.write_text, which translates \n to \r\n on that platform. The committed blobs therefore contained CRLF, and the assertion of byte-identical retrieval failed. The connector itself was correct throughout: it returned exactly the bytes Git had stored. The defect was in the fixture, and it was resolved by writing the files with write_bytes, which keeps the blob content exact.

The observation generalises beyond the test. Blob content reflects what was committed, including CRLF line endings where a Windows contributor produced them, so no downstream component may assume LF-only input. The consequence is visible in the Data Extractor, where the Dockerfile text is normalised before being fed to Hadolint, as described in Section 5.4.2.

### 5.2.3 Testing and conformance

The component is covered by nine tests across two files, one offline suite and one integration test.

tests/test_vcs_connector.py holds eight unit tests running against a temporary Git repository built in the fixture, so the suite needs no network and no external clone. They cover byte-identical retrieval of the two versions, acceptance of abbreviated SHAs, a Dockerfile located in a subdirectory, the working tree left untouched after retrieval, which is the no-checkout guarantee of RF1, the idempotence of repeated calls with the same SHA required by RNF5, and the three error cases required by RNF4.

tests/test_vcs_connector_getting_started.py holds the integration test, which replicates Experiment 3 against a local clone of docker/getting-started at commits 2bca273 and 2981665. It asserts that the retrieved pair differs exactly by the --platform=$BUILDPLATFORM and $TARGETPLATFORM additions, so the assertion checks the content of the retrieval and not merely its success. The test is guarded by a skipif and runs only when the GETTING_STARTED_REPO environment variable is set, since it depends on a repository that is not vendored with the prototype. It is therefore one of the four tests reported as skipped on the Windows host.

The module reaches 100% statement coverage across its 25 statements, with no uncovered lines. It was additionally subjected to the programmatic conformance audit described in Section 5.6, which checked blob-level access and the absence of any working-tree operation against the specification of Section 4.3, and found the component conformant with RF1, RNF4 and RNF5 as written, requiring no change.

## 5.3 Detection Engine

The Detection Engine is the analytical core of the pipeline and constitutes Stage 2 of the four-stage execution described in Section 5.1.1. It receives the two Dockerfile states delivered by the VCS Connector and determines whether the structural change between them constitutes one or more of the fourteen catalogued refactoring types, as RF2 requires.

The component was implemented with dockerfile-parse 2.0.1, the technology validated in Experiment 5 and selected in Section 4.4. The library reconstructs multi-line RUN instructions joined by backslash continuation into logical single units, which line-level regular expression matching cannot do, and which is the precondition for every rule reasoning about instruction counts.

It is the largest module of the prototype, at 824 statements, and carries 136 of the 302 tests in the suite.

### 5.3.1 Parsing

Two properties of the parsing step were fixed before any rule was written.

The parser is given an in-memory fileobj rather than being constructed with its default behaviour. DockerfileParser writes a Dockerfile to the working directory when content is assigned to it, which would violate the no-filesystem-writes principle the VCS Connector establishes in Section 5.2 and which the whole pipeline maintains.

CRLF is normalised to LF before parsing. Blob content reflects whatever was committed, so a Dockerfile authored on Windows legitimately reaches the engine with CRLF endings, as recorded in Section 5.2.2. Normalising at the entry of the component means no rule has to account for line endings, and a test asserts that CRLF input parses identically to LF input.

RNF4 is satisfied through DockerfileParseError, raised both for non-text input and for any failure inside the parser, so that malformed content never propagates a raw library exception and the orchestrator can distinguish it from a build or daemon failure. It maps to exit code 2 in Table 5.2.

### 5.3.2 Rule registry and the three architectural laws

Each of the fourteen rules is an independent function registered in a central registry through the @rule decorator. Adding a rule is one function and one registry entry; no existing rule and no downstream component is modified. This is the Open-Closed Principle that RNF3 demands, and Section 5.3.4 describes how the criterion was exercised rather than assumed.

Three laws govern the engine and are stated in the module docstring as its contract.

**Directional.** Each rule detects strictly the direction its catalogue definition mandates. The reverse path of a one-directional refactoring introduces technical debt or reintroduces a smell, so it is ignored by design and yields an empty result, an approach aligned with DRMiner. Where the catalogue defines complementary directions as distinct refactorings, as with Extract Stage and Inline Stage, each direction has its own rule and its own identifier. Ambiguous directions were decided explicitly rather than inferred.

**Granular.** Detection operates per instruction subset. Each rule extracts its refactoring from the instructions it concerns and silently ignores changes to other instructions in the same commit, catalogued or not. A commit is not required to contain the refactoring in isolation: a FROM tag update is R02 even when the same commit edits a WORKDIR or an ENV. This replaced an initial whole-file design in which any unrelated change disqualified the match, which would have made the engine useless against real commit histories.

**Non-exclusion.** When a commit matches several rules, all of them are reported. The engine accumulates every identified refactoring into a result list, and the only ordering applied is the registry index, which is decoupled from the quality dimensions. This implements the non-exclusion principle of Section 4.4.2 and replaced an earlier scheme in which a dimension precedence resolved conflicts.

The function detect_refactorings returns a list of typed DetectionResult records naming each refactoring and the instructions involved. Every rule returns None when its conjunction of conditions fails, which the engine renders as absence from the result list, and an empty list is its answer when no rule matches. The engine therefore produces no speculative match and no false positives by construction. The type carries no impact field, and its docstring states why: the identifier is a label naming what was found, never a value consulted to reach the decision, which is the separation RF3 establishes and Section 4.1.1 requires. No rule reads the extended catalogue at any point.

Listing 5.5 shows the shape every rule follows. The decorator takes no arguments and registers the function it receives, so the catalogue identifier lives in the returned record rather than in the registration, and adding a rule is a single function plus a single decorator line.

```
@rule
def detect_inline_run_instructions(
  instructions_a: list, instructions_b: list) -> Optional[DetectionResult]:
  """R01 — Inline RUN Instructions."""
  # rule logic
  return DetectionResult(
    refactoring_id="R01",
    refactoring_name="Inline RUN Instructions",
    instructions_before=tuple(...),
    instructions_after=tuple(...),
  )
```

Listing 5.5 - Shape of a detection rule, shown for R01

The signature is the contract the three architectural laws rest on. Both parameters are instruction lists rather than raw text, so no rule parses and every rule sees the same normalised input; the return type is optional, so failing the conjunction of conditions is expressed as None rather than as a sentinel value the engine would have to interpret; and the record carries the instructions of both states, which is what allows the Report Generator to name what changed without re-deriving it.

### 5.3.3 Rule families

The fourteen rules divide into four families by the structure they analyse. Table 5.5 summarises the mechanical criterion each one applies.

| Family | Rules | Mechanical criterion |
|---|---|---|
| **Base image** | R02, R10, R14 | FROM instructions aligned by stage position |
| **Transfer and declaration** | R03, R04, R08 | Presence or absence of a declared identifier or argument string |
| **RUN body** | R01, R09, R12, R13 | Shell segmentation and command multiset comparison |
| **Stage structure** | R05, R06, R07, R11 | Stage decomposition and body relocation |

Table 5.4 - The four rule families of the Detection Engine and the criterion each applies

**Base image**

The three FROM rules are kept disjoint by what each requires of the image reference, so that no pair can compete for the same change.

R02 reports every FROM satisfying two conditions: entity preservation, meaning identical repository name, digest, platform flags and stage alias, and a differing tag. An omitted tag counts as latest, since Docker resolves it that way and the DL3007 smell is the same. Entity preservation is what separates R02 from R10: a changed repository name, even combined with a tag change, is a full base-image substitution and never R02. The tag separator is the colon after the last slash, so a registry port such as registry:5000/app is not mistaken for a tag. The rule is directional in that the destination tag must be pinned; unpinning to latest reintroduces DL3007 and yields an empty result.

R10 posed a direction problem no static comparison can resolve. The catalogue direction is "towards a lighter and more secure alternative", which is empirical by nature and expressed in ΔSize and ΔCVEs. Deciding it statically would require either a curated list of minimal images, which invents knowledge the prototype does not have, or gating detection on the metrics, which inverts the fixed pipeline order. Both were rejected, and the rule detects any base-image entity change, leaving the direction to be judged downstream from the measured deltas. This is the documented exception to the directional law.

R14 requires the image reference to be untouched, repository name, tag, digest and platform flags all identical, which is the condition keeping it clear of the other two, and then accepts two alias mutations and rejects a third. An alias appearing where none existed is accepted, since it removes the technical debt of numeric stage references such as --from=0; an alias changing between two non-empty values is accepted, being the direct syntactic equivalent of DRMiner's Rename Image behaviour; an alias disappearing is rejected as the reverse path. Scope is the FROM instruction alone: a rename normally forces matching edits to the --from= references reading from the stage, and those are tolerated as noise rather than examined, which is the granular law applied.

All three align FROMs by stage position, and all three stay silent when the two states hold different numbers of FROMs, since positional alignment is then ambiguous and belongs to Extract Stage or Inline Stage territory.

**Transfer and declaration**

R03 detects added environment variables by name, the mechanical criterion separating an addition from a modification: it reports every ENV instruction in the after state declaring at least one name absent from the ENV declarations of the before state. Both the ENV A=1 B=2 and the legacy single-variable syntaxes are handled. R04 applies the same logic to ARG by identifier, the part before the first =, so that ARG NAME without a default declares the identifier just the same; changing only the default value of an existing identifier is a modification and not a detection.

R08 evaluates the ADD and COPY subset granularly. An ADD is reported as replaced when its byte-identical argument string disappears from the ADDs and appears among the COPYs of the after state, since the refactoring swaps the keyword and never the sources or the destination. The comparison is multiset-based, so a pre-existing identical COPY does not create a false positive, and an ADD deleted without a corresponding new COPY is a deletion rather than a refactoring.

R08 also carries a behaviour-preservation guard, added after a correctness defect was found. The rule was firing for swaps where the ADD used a capability COPY lacks: a remote URL source, which ADD downloads and COPY cannot, or an auto-extractable local archive, which ADD unpacks and COPY copies as-is. Such a swap changes the build result, violating both the catalogue clause and Fowler's behaviour-preserving definition of refactoring. Those ADDs are now excluded from the replacement budget, and the rule yields an empty result.

**RUN body**

R01 requires the count of RUN instructions to fall, and splitting one RUN into several is the reverse path and yields nothing. A falling count alone is ambiguous between consolidation and deletion, which is the third challenge recorded in Section 4.1.4, so a content preservation check resolves it: the commands of the disappearing instructions are sought inside the surviving ones by splitting RUN bodies on the shell operators &&, || and ; and comparing the resulting command multisets. Every command accounted for means consolidation; any command missing means deletion, and nothing is reported.

R09 detects a chained shell sequence moved out of a RUN into an external script. That script is a file the pipeline never sees, since the VCS Connector delivers one Dockerfile at two commits, yet the move leaves three facts in the file that make it decidable from the Dockerfile alone. All three must hold. First, shell work left the file: the total number of shell command segments across all RUN instructions falls, recovered by the same segmentation R01 introduced. This condition carries the directional weight, because without it the addition of a script is new functionality rather than an extraction. Second, a script arrived: a transfer in the after state brings in a file whose name ends in .sh or .bash, and no transfer in the before state delivered a script of that file name. Comparing file names rather than whole argument strings is what stops a script that merely changed destination from reading as a new arrival, and it is the guard keeping R09 off the R12 scenario. Third, a RUN executes it at the path the transfer gives it, with a destination ending in / or a transfer with several sources resolving to the destination directory joined with the source file name.

Invocation is positional rather than textual. The script counts as executed only when it stands where a command stands: as a segment's own first token, in its absolute or relative form, or as the file handed to an interpreter in sh, bash, ash, dash, or source. A path passed to another command, as in chmod +x /app/setup.sh, prepares the script without running it and does not satisfy the condition. The reverse path is silent by construction, since inlining a script back makes the transfer disappear rather than appear.

R12 applies the same style of conjunction to a RUN whose sole purpose is to move a file. A disappearing RUN is not evidence on its own, so three facts must hold together: the RUN does nothing but move files, with mkdir tolerated as a companion because it only prepares the directory the move lands in and COPY creates that directory by itself; the origin transfer instruction is identified; and the same sources are shown to land directly on the move's destination in the after state.

R13 detects the reformatting of a RUN, whether by reordering arguments or by splitting the instruction across lines. Comparing normalised strings would have collapsed the two cases into one and lost the distinction the catalogue draws, so the rule compares token sequences: reordering keeps the same tokens in a different sequence, while line splitting keeps the same tokens with different spacing.

**Stage structure**

The stage-level rules share a decomposition helper, _stages(), which converts an instruction list into Stage records carrying the stage index, the image reference, the alias, and the body instructions with the opening FROM excluded. A Stage.identity property returns how a COPY --from names the stage, its alias lower-cased when it has one, otherwise its positional index, so that --from=builder and --from=0 resolve through one code path and named and unnamed stages are served alike. Instructions appearing before the first FROM belong to no stage and are skipped.

R06 detects a stage collapsing into another and R05 is its exact inverse, reusing the same machinery in the opposite direction. A falling stage count is not evidence on its own, and the rule requires the body of the disappeared stage to be shown to have migrated.

R11 detects a stage lifted into a standalone Dockerfile, built separately, with the main file keeping only a reference to the image that build produces. Four conditions are evaluated per stage of the before state: the stage had a body, meaning its instruction list excluding the opening FROM is non-empty; the body is gone, meaning the stage of the same identity in the after state holds no instructions; the base reference changed, since an emptied stage still on its original base is a deletion rather than a move, and this is the condition separating them; and something still consumes it, meaning some transfer in the after state carries --from= naming that identity. The stage count is unchanged throughout, which is what distinguishes R11 from R05 and R06 and stops the three stage rules competing for one pair.

R07 requires two conditions to establish a pure rearrangement: the multiset of instructions is identical, so nothing was added, removed or edited, and the sequence differs. It is directional through a mechanical proxy rather than reporting any reordering, following the catalogue mechanism of postponing frequently changing instructions. Its known limitation is that requiring an identical multiset means a commit that reorders and edits in the same change is not reported.

### 5.3.4 Composite detections

The non-exclusion law produces composite readings on some pairs, and one is worth recording because it appears in the catalogue experiment for R11. Moving a stage out changes the FROM reference of the emptied stage while preserving its alias, which also satisfies R10. Both are reported. The composite reading was accepted rather than suppressed, on the grounds that filtering one in the presence of the other would couple two rules that RNF3 requires to remain independent. The consequence for validation is discussed in Section 5.6.

### 5.3.5 RNF3 exercised

The registry mechanism had been in place since the first rule and all fourteen were added through it, but nothing exercised the acceptance criterion of RNF3 itself. Three tests now do.

The first defines a fictitious rule, R99 — Add HEALTHCHECK, entirely inside the test file, registers it through the public decorator, and asserts three things: that the decorator alone placed it in the registry, that detect_refactorings runs and reports it against a real Dockerfile pair, and that it takes the last registry position so that the reporting order of the catalogue

rules is undisturbed. The second covers the other half of the criterion, measuring the detection of the R01 pair before and after registering the new rule and requiring that the result grows by exactly R99: adding a rule must neither alter nor suppress any existing one. The third verifies that the isolation holds, running without the fixture and requiring that no R99 survives.

A registry_restored fixture snapshots the rule list by value and repairs it in place in its finally clause. Repairing in place rather than rebinding matters, because the @rule decorator closes over that list object and replacing it would break registration for the remainder of the session.

Conformance was verified rather than asserted. Both git diff and git status against src/detection_engine.py are empty, so no line of the production module was touched, which is precisely the Open-Closed Principle the requirement demands. Isolation was confirmed twice, by the third test and by an external check that runs the whole suite and then imports the engine, finding fourteen registered rules and no residual R99. The suite also passes with random ordering disabled, ruling out order dependence.

### 5.3.6 Testing

The component carries 119 tests in tests/test_detection_engine.py: 111 distributed across the fourteen rules and 8 cross-cutting, of which three exercise RNF3 and five cover parsing. Table 5.6 gives the distribution per rule.

| Rule | Tests | Rule | Tests |
|---|---|---|---|
| R01 | 9 | R08 | 10 |
| R02 | 16 | R09 | 10 |
| R03 | 9 | R10 | 7 |
| R04 | 6 | R11 | 6 |
| R05 | 5 | R12 | 7 |
| R06 | 6 | R13 | 7 |
| R07 | 6 | R14 | 7 |

Table 5.5 - Distribution of the 111 rule tests across the fourteen catalogue types

Every rule follows the same test pattern. A mock Dockerfile pair mirrors the corresponding catalogue experiment, so that the unit test exercises the same transformation the empirical measurement used; a CRLF variant confirms the rule is insensitive to line endings; positive cases with unrelated instruction noise confirm the granularity law; and negative cases confirm the directional law, in particular that the reverse transformation yields an empty result. R02

carries the largest suite because its entity-preservation and pinning conditions admit the most edge cases, among them the registry-port lookalike, the stage-alias change, and the simultaneous name-and-tag change that belongs to R10.

The module reaches 97% statement coverage, the lowest of the prototype and a consequence of its size: 13 of its 422 statements are uncovered, distributed across defensive branches in individual rules rather than concentrated in any one of them.

## 5.4 Performance Analyzer and Data Extractor

The Performance Analyzer and the Data Extractor are the two measurement components of the prototype and together constitute the measurement step of Stage 3, described in Section 5.1.1. They produce the four delta metrics on which the empirical contribution of this dissertation rests: ΔSize, ΔCVEs, ΔWarnings and ΔInstr.

Neither component builds the images it measures. Both consume a pair of images produced beforehand by a preparation step that the orchestrator owns, and the design of both follows from that arrangement.

### 5.4.1 Performance Analyzer

The Performance Analyzer measures the impact of the detected refactoring on the Performance dimension. It receives the references to the two images the preparation step built and returns the signed size delta between them, satisfying RF4 and RNF1.

#### 5.4.1.1 The preparation step

The component depends on src/image_builder.py, which builds the two Docker images before the measurement begins. To natively support advanced Dockerfile syntax such as the --chmod directive via BuildKit, the build execution is performed via subprocess invoking docker buildx build with the --load flag, ensuring the compiled images are correctly loaded into the local Docker daemon. The build is a preparation step rather than a responsibility of either measurement component, and the reason is the concurrency of Stage 3: the Performance Analyzer reads the size of each built image while the Data Extractor scans those same images with Trivy, so both need the same resource. Leaving the build inside either component would make the other wait on it, turning a concurrent step into a sequential one; giving each component its own build would duplicate the most expensive operation in the pipeline. Lifting it into a preparation step resolves both, and is why image_builder.py is not a component in the sense of Section 4.1.6, as established in Section 5.1.1.

The step was implemented with the Docker SDK for Python 7.1.0, validated in Experiment 4. Five decisions shaped what it hands to the Performance Analyzer.

**References, not objects.** BuiltImagePair carries image identifiers as strings rather than SDK objects. The Performance Analyzer opens its own Docker connection and looks the images up by identifier, so it shares no client, no connection and no object graph with the preparation step or with the Data Extractor. The only thing crossing the boundary is an identifier,

consumed read-only on both sides, which makes the independence of the two components structural rather than a matter of convention.

**In-memory build context.** The build context is assembled as a tar archive in io.BytesIO and handed to the daemon as a byte stream, never as a directory path. Analysing a repository therefore writes nothing to the caller's working directory, and the Dockerfile is injected into the archive rather than read from disk, so the content the VCS Connector delivered is built exactly as it was committed. A file already named Dockerfile at the root of the context is skipped, since the state under analysis supersedes it. The files accompanying each build come from the historical commit contexts described in Section 5.1.2, with .git, __pycache__, .venv and node_modules excluded.

**No re-pull.** The base image is not re-pulled, so both states build against the same locally cached base. This is what makes the two sizes comparable at all: a tag that moved upstream between the two builds would otherwise contaminate ΔSize with a change that is not the refactoring's.

**Guaranteed cleanup.** Both images are removed when the preparation context closes, in a finally block covering all three failure shapes: a second build failing after the first succeeded, an exception raised by either measurement component inside the block, and an ordinary return. Without this a catalogue-wide run would leave dozens of images behind and exhaust the daemon's storage. Removal errors are swallowed deliberately, since an exception raised during cleanup would mask the failure that triggered it.

**Engine version capture.** RNF5 requires the report to record the version of every external tool invoked, including the Docker Engine. The preparation step is where the Docker environment is established, so it is where that version is read and handed to the Report Generator.

Listing 5.6 shows the build and cleanup contract.

```
client = _connect()
    built: List[Image] = []
    try:
        image_before = _build_image(client, dockerfile_before, resolved_context, "before")
        built.append(image_before)
        image_after = _build_image(client, dockerfile_after, resolved_context, "after")
        built.append(image_after)


        yield BuiltImagePair(image_before=image_before.id, image_after=image_after.id)
    finally:
        _remove_images(client, built)
```

Listing 5.6 - Construction of the image pair, with removal guaranteed by the finally clause

Preparation failures raise ImageBuildError rather than the exception of the measurement component, mapping to exit code 3 in Table 5.2 while the Performance Analyzer maps to 4. RNF4 asks for distinct informative exceptions, and the two events are genuinely different: a

build that does not complete aborts the analysis before any measurement is attempted, whereas a size that cannot be inspected fails one metric of a step already running.

The step is covered by 18 tests in tests/test_image_builder.py, running against a fake Docker client so the suite needs no daemon, plus one end-to-end test that builds two real Alpine images and is skipped when Docker is unreachable. They cover the build itself, the four cleanup shapes, the RNF4 contract across four failure modes, and context assembly, including the assertion that building writes nothing to the working directory.

#### 5.4.1.2 Measurement

Size is read from image.attrs["Size"], the VirtualSize, which is the total uncompressed byte count of all image layers, returned as a 64-bit integer through the same Docker SDK version. The Docker CLI was rejected at design time because it applies decimal rounding and human-readable truncation by design, documented in Docker issue #31298, which would collapse the micro-variations the catalogue produces, the +417 bytes of R09 — to zero and violate RNF1. The SDK queries the daemon API endpoint directly and returns the raw integer.

Two properties define the component. It measures and nothing else: it does not build, does not remove, and never reads a value produced by the Data Extractor. ΔSize is computed from the two images alone, just as ΔCVEs, ΔWarnings and ΔInstr are computed without ΔSize, and that mutual independence is what allows the two components to run concurrently. The module holds no state and opens its own Docker connection rather than receiving one, so it shares no mutable object with its sibling.

The delta is expressed in bytes alone. SizeMetric carries size_before, size_after and delta_size, all in bytes, under the sign convention of this study where a negative delta is a size reduction. Measuring in bytes alone is what keeps the four metrics uniform: ΔCVEs, ΔWarnings and ΔInstr are absolute signed counts, so a relative figure on this one would hand the Report Generator a metric in a different form from its three siblings. A percentage remains derivable from the two absolute sizes by whoever needs it, without the component committing the pipeline to it.

Listing 5.7 shows the measurement, with the connection released whatever the outcome.

```
client = _connect()
    try:
        return SizeMetric.from_sizes(
            _image_size(client, image_before, "before"),
            _image_size(client, image_after, "after"),
        )
    finally:
        with contextlib.suppress(Exception):
            client.close()
```

Listing 5.7 - Computation of the size delta from the two prepared images

**Image size is not bit-for-bit reproducible**

Measuring the R09 catalogue pair repeatedly returned deltas of +414, +416, +417, +418 and +419 bytes across runs, against the +417 recorded in the corresponding experiment directory. The cause is not the component. Four consecutive builds through the Docker CLI, the experiment's own command, with no code of this prototype involved, produced the values in Table 5.6.

| Run | Before (bytes) | After (bytes) | Delta |
|---|---|---|---|
| **1** | **3,633,776** | **3,634,193** | **+417** |
| **2** | **3,633,775** | **3,634,191** | **+416** |
| **3** | **3,633,776** | **3,634,192** | **+416** |
| **4** | **3,633,774** | **3,634,193** | **+419** |
| **Recorded** | **3,633,775** | **3,634,192** | **+417** |

Table 5.6 - Four consecutive CLI builds of the R09 pair, against the values recorded in the experiment

Each absolute size varies within a band of about two bytes and the delta within about three, and the recorded values sit inside those bands. Docker Engine 29.1.3 on WSL2 therefore does not produce a byte-identical image from byte-identical inputs. A trivially simple Dockerfile does reproduce exactly; the variation appears with builds that copy from a context and write files in a RUN layer.

Three consequences follow, none of which the component can remove. First, the statement in Section 4.5.1 that image size is a deterministic function of the Dockerfile content and the base image state, producing the same byte count for the same inputs every time, overstates the guarantee at byte resolution in this environment. Second, the byte-level precision RNF1 demands is a property of the *retrieval* method, and it holds: the SDK returns exact integers where the CLI would round. What does not hold is byte-level *reproducibility* of the measured object, and the two are distinct claims of which only the first belongs to this component. Third, deltas whose magnitude falls within the noise band cannot be distinguished from it. The +417 bytes of R09 is two orders of magnitude above the band and stands unaffected; the +1 byte recorded for R08 does not exceed it, and while Section 4.2 already classifies that result as neutral and far below the relevance threshold, the figure should be read as indistinguishable from zero rather than as a measured increase.

No experiment file was altered. The recorded values remain valid single-run observations of the quantity they measure.

#### 5.4.1.3 Testing

The component is covered by 15 tests in tests/test_performance_analyzer.py, running against a fake Docker client so the suite needs no daemon, plus one end-to-end test that builds two real images through the preparation step and is skipped when Docker is unreachable.

The RF4 and RNF1 group covers the PoC-3 reference delta of −26,318,550 bytes, the +417 byte micro-variation of R09 surviving intact, the signed convention in all three directions, a zero before size, and the metric carrying byte fields only. The boundary group asserts that exactly the two given images are inspected, that the fake client raises on any build call so the component provably never builds, and that both images are left in place for the Data Extractor running alongside. The RNF4 group covers an unreachable daemon, an unknown image naming the offending state, an API error, a missing Size attribute, and a non-integer size, which is what the rejected CLI path would deliver.

The module reaches 100% statement coverage across its 38 statements.

### 5.4.2 Data Extractor

The Data Extractor is the second measurement component of Stage 3 and the one carrying the greatest share of the empirical load. It receives the two Dockerfile states and the references to the two images the preparation phase built, and returns three of the four signed deltas: ΔCVEs, ΔWarnings and ΔInstr. It satisfies RF5, RF6, RNF2, RNF4 and RNF5.

Three instruments produce those three metrics: Trivy scans the built images for vulnerabilities, Hadolint analyses the Dockerfile text for rule violations, and dockerfile-parse 2.0.1 counts logical instructions in-process. Trivy and Hadolint are invoked as containers, aquasec/trivy and hadolint/hadolint, both validated in Experiment 2, in JSON mode, and their heterogeneous schemas are normalised here into a single metric representation.

#### 5.4.2.1 The preparation phase

The component consumes the same image pair the Performance Analyzer measures, produced by the preparation phase described in Section 5.4.1. What it takes from that phase is narrower: Trivy scans the two images, while Hadolint and the instruction count operate on the Dockerfile text the VCS Connector delivered and never touch an image at all. Of the five decisions that shaped the phase, two matter directly here. The images arrive as identifiers rather than SDK objects, so this component opens its own Docker connection and shares no client with its sibling, which is what makes the concurrency of Stage 3 structural rather than conventional. And the images are consumed strictly read-only: the component does not build and does not remove, both being the responsibility of the phase, and tests assert the boundary directly by checking that no emitted command contains build, rmi or remove.

The component never reads a value produced by the Performance Analyzer. Its three metrics are computed without ΔSize, just as ΔSize is computed without them, which is the mutual independence Section 5.1.1 requires of the two Stage 3 components.

#### 5.4.2.2 Invocation through the Docker CLI

This is the one place where the prototype shells out to the Docker CLI. The choice deserves stating, because Section 5.4.1 rejects the CLI for image size on the grounds that it rounds.

The two cases are not the same. Nothing is read from the CLI here beyond a container's standard output, so no rounding can occur, and the tools are invoked through the same

commands the extended-catalogue experiments use. Matching the validated experiment protocol byte for byte is worth more than uniformity with the SDK used elsewhere, because it is what makes the values this component produces directly comparable with the recorded ones.

The tool images are deliberately left untagged and resolve to latest, for the same reason: a pinned tag would measure with a different tool build than the one that produced the recorded values. The cost of that choice is that the tool version can move between runs, and what compensates for it is tool_provenance() recording the version actually used in every report, as RNF5 requires.

#### 5.4.2.3 Security metrics

RF5 requires the count of distinct vulnerability identifiers rather than occurrences, so deduplication spans the whole Trivy report rather than each result block: the same CVE reported for two packages contributes one identifier. The first severity seen for an identifier is the one kept, so a CVE listed twice cannot inflate a tier. A vulnerability whose severity is absent or falls outside the known tiers still counts towards the total, so the tiers never exceed it but may sum to less.

RF5 also requires the delta by severity tier, so delta_cves is accompanied by delta_cves_by_severity across CRITICAL, HIGH, MEDIUM, LOW and UNKNOWN. The single total remains the headline figure the Report Generator aggregates.

A shared database cache. Trivy keeps its vulnerability database in a cache directory that a --rm container loses on exit. Without a persistent volume every scan re-downloads the database and, more consequentially for RNF5, trivy version reports no database metadata at all, the first implementation returned the database version as "unknown" for exactly this reason. A named volume provides a pre-warmed database from which thread-isolated temporary cache directories are initialized for each parallel scan, allowing the database to be downloaded once while preventing file-lock contention between concurrent executions. Verified end to end, tool_provenance() reports a database timestamp rather than "unknown". The volume is a deliberate persistent resource, unlike the images, which are removed when the preparation phase closes.

#### 5.4.2.4 Maintainability metrics

The 47-rule screen of Appendix C is held in MAINTAINABILITY_SCREEN as a declarative list, inspectable and revisable without touching the parsing logic. Its contents were verified programmatically to be set-identical to the filter of the catalogue verification scripts. Every finding of a screened rule counts, including repeated violations of the same rule on different lines: the metric is the global sum of maintainability violations, not the number of distinct rules violated.

Hadolint reads the Dockerfile from standard input rather than from a file, so no temporary Dockerfile appears in the working directory and the text analysed is exactly the one the VCS Connector delivered. CRLF endings are normalised first, since blob content reflects whatever was committed, as established in Section 5.2.2.

Failure is detected by output rather than by exit code. Hadolint exits non-zero whenever it reports findings, which is the normal case here, so the exit code alone cannot signal failure. An empty standard output is what distinguishes a tool that ran from one that did not.

Independent parsing. count_logical_instructions uses dockerfile-parse directly and imports nothing from the Detection Engine. The two have different obligations, the engine needs each instruction's keyword and argument string to decide a refactoring, this component needs only how many there are, and coupling them would let a change to detection move a maintainability metric. A test reads this module's own source and fails if the string detection_engine appears in it, so the coupling cannot reappear by oversight.

#### 5.4.2.5 Internal concurrency

The four external invocations , two Trivy scans and two Hadolint analyses, are dispatched together, since none depends on another's result and the scans dominate elapsed time. The instruction count runs in-process and needs no container.

Listing 5.8 shows the dispatch. Every future is resolved before the first failure is raised, so no container is left running behind an early return.

```
with concurrent.futures.ThreadPoolExecutor(max_workers=4) as pool:
    futures = {
        "scan_before": pool.submit(_scan_image, image_before, "before"),
        "scan_after": pool.submit(_scan_image, image_after, "after"),
        "lint_before": pool.submit(_lint_dockerfile, dockerfile_a, "before"),
        "lint_after": pool.submit(_lint_dockerfile, dockerfile_b, "after"),
    }
    outputs, failure = {}, None
    for name, future in futures.items():
        try:
            outputs[name] = future.result()
        except DataExtractorError as exc:
            failure = failure or exc
    if failure is not None:
        raise failure
```

Listing 5.8 - Concurrent dispatch of the four external tool invocations

This is concurrency nested inside concurrency: the component is itself one of the two branches Stage 3 runs in parallel, and it fans out into four invocations of its own. The nesting is safe because neither level shares mutable state, and because both resolve every future before propagating a failure.

#### 5.4.2.6 Retention of source artifacts

RNF5 requires reproducibility to rest not only on recorded tool versions but on the capability to retain the source analysis artifacts the external tools generate. The conformance audit found this the one acceptance criterion unmet in code: the component counted the findings and let the output go.

ExtractionResult now carries raw_hadolint and raw_trivy, each a mapping from state to the tool's output text, as shown in Listing 5.9.

```
    raw_hadolint={
        "before": outputs["lint_before"].decode("utf-8", errors="replace"),
        "after": outputs["lint_after"].decode("utf-8", errors="replace"),
    },
    raw_trivy={
        "before": outputs["scan_before"].decode("utf-8", errors="replace"),
        "after": outputs["scan_after"].decode("utf-8", errors="replace"),
    },
```

Listing 5.9 - Retention of the raw tool output at the component boundary

The text is decoded from the bytes the tool wrote and kept verbatim. It is never re-serialised from the parsed object, which would silently normalise key order and whitespace and stop it being the artifact. Retention costs nothing at extraction time, since the invocation helper already returns the bytes and the parsed object was built from them. Verified against the real tools on the R09 catalogue pair, the component retained 24,227 and 24,470 bytes of Trivy output for the two states, which the Report Generator writes under raw_data/ as recorded in Section 5.1.4.

#### 5.4.2.7 Validation against the catalogue

The component was run over all fifteen controlled experiments in experiments/extended_catalog/, building each pair through the preparation phase. Four of them carry a recorded metrics file against which the produced values can be compared directly. Table 5.7 gives the comparison.

| Experiment | Warnings | Instr | CVEs |
|---|---|---|---|
| **R06 — Inline Stage** | +0 = +0 | −3 = −3 | +0 = +0 |
| **R09 — Extract RUN Instructions** | +0 = +0 | +1 = +1 | +0 = +0 |
| **R12 — Remove RUN (mv command)** | +0 = +0 | −2 = −2 | +0 = +0 |
| **R11 — Move Stage** | +0 = +0 | −3 vs +1 | +0 = +0 |

Table 5.7 - Values produced by the Data Extractor against those recorded in the four catalogue experiments carrying a metrics file

Three reproduce the recorded values exactly. The fourth diverges on one metric, and the cause is the scope of the prototype rather than a defect.

The R11 divergence. Move Stage extracts a build stage into a separate Dockerfile, and the catalogue experiment performs project-level accounting across both the original file and the newly extracted one, yielding +1. The prototype is bound by RF1 to a single file path and audits the primary Dockerfile alone, yielding −3: the three instructions that left the file are counted as removed, and the FROM the extracted file gains is never seen.

The divergence affects ΔInstr exclusively. ΔWarnings and ΔCVEs matched the recorded baseline precisely, since neither depends on instructions the prototype cannot see. The comparison with R09 is instructive: that refactoring shows no divergence because it offloads logic to a shell script, which is not a Dockerfile instruction and therefore falls outside the count by definition, whereas R11 creates a second Dockerfile whose instructions do count in the catalogue's accounting.

The single-file scope was kept as a deliberate architectural choice, on the grounds that automatic aggregation of satellite files would require heuristics fragile enough to compromise the determinism of the measurement pipeline. The behaviour is consciously assumed as a structural property of the prototype and declared in Section 4.9. No code change was made.

#### 5.4.2.8 Testing

The component carries **41 test functions comprising 43 test cases** in tests/test_data_extractor.py, the second largest suite of the prototype. The external tools are replaced by a fake subprocess.run serving recorded Trivy and Hadolint JSON, so the suite runs without a daemon and without pulling either tool image; one end-to-end test invokes the real containers against two real images and is skipped when Docker is unreachable.

The screen group verifies exactly 47 distinct rules, the exclusion of the default-disabled DL3049–DL3058, that only screened rules are counted, that repeated violations of one rule all count, and the PoC-2 criterion of −1 for an eliminated DL3007 that satisfies RNF2. The ΔCVEs group covers distinct identifiers counted once, per-tier reporting, an unknown severity counting in the total but in no tier, and a clean image yielding zero rather than an error when Trivy omits the vulnerability list entirely. The ΔInstr group covers comments excluded, a backslash-continued instruction counting as one, CRLF tolerated, a single added instruction moving the delta by one, and the source proven free of any Detection Engine import.

The boundary group asserts that no build or removal command is emitted, that both images are scanned read-only, that the two scans share the database cache volume, and that Hadolint reads from standard input rather than a path. The RNF4 group covers eight failure modes: a missing docker command, a tool timeout, empty output, malformed JSON, an unexpected Trivy shape, an unexpected Hadolint shape, non-string Dockerfile content, and parsing failing before any image is scanned. Three retention tests confirm that both states are present for both tools, that a deliberately quirky payload with unusual spacing and reversed key order survives byte for byte while still parsing to the right count, and that the retained text accounts for exactly the identifiers counted. Six further tests cover tool_provenance(), including an unpopulated cache reported as unavailable rather than invented.

Two audit findings shaped the suite. The traversal of the Trivy report assumed every nested member was an object, so a truncated report raised a raw AttributeError from the middle of the walk instead of the DataExtractorError that RNF4 requires; the traversal is now wrapped and three parametrised regression tests cover a non-object result, a non-object vulnerability, and an integer where a vulnerability was expected. And tool_provenance() had no test at all,

which is the worst place in the module for a silent regression: a change to the trivy version schema would return the database version as unavailable without anything failing, exactly as happened once during development.

The module reaches 89% statement coverage across its 144 statements, with 16 uncovered statements. These include the exception handler around dockerfile-parse, which the audit established is unreachable in practice because the parser does not raise on malformed content.

## 5.5 Report Generator

The Report Generator is the terminal component of the pipeline and constitutes Stage 4 of the execution described in Section 5.1.1. It receives the detection identifiers produced in Stage 2 and the metric sets produced in Stage 3, and writes the artifacts through which every result of this dissertation is expressed. It satisfies RF7 and closes the provenance requirement of RNF5.

It is the only component that depends on no external technology. Aggregation, serialisation and formatting are internal logic, which is why Table 5.3 records no library against it.

### 5.5.1 Output artifacts

generate_report() writes four artifacts into a caller-supplied directory, creating it when absent. impact_report.json carries the structured output RF7 requires: the before and after value of every metric, the four deltas, the per-severity CVE vector, the reproducibility block, and pointers to the retained artifacts.

human_readable_summary.txt presents the same content arranged for direct review by a practitioner.

terminal_output.log records the detailed console execution log and diagnostic output printed to the terminal during the pipeline's run.

Under raw_data/, the files trivy_raw.json and hadolint_raw.json hold the source analysis artifacts the Data Extractor retained, each keyed by state so that the before and after evidence stay distinguishable.

The separation of the evidence from the report is deliberate. The retained Trivy output runs to tens of kilobytes, 50,648 bytes for the R09 pair, so embedding it would bury the structured output it accompanies. The JSON references the files by name and they live one directory away, which keeps the report readable while the evidence stays with it. Each raw file is a single JSON document keyed by state, with each state's text embedded as a parsed value rather than as a quoted string, so the artifact remains machine-readable end to end. Output that does not parse is kept as a raw string rather than discarded, on the principle that a tool emitting something unexpected must not cost the evidence.

Every artifact is written as UTF-8 with LF endings, whatever the host platform. A report produced on Windows is therefore byte-identical to one produced on Linux, and two runs can be compared with diff, which is what makes the reproducibility RNF5 demands operational rather than nominal.

### 5.5.2 Aggregation, not interpretation

The component pairs the catalogue identifier of each detection with the four measured deltas exactly as obtained, adding no expected direction, no dimensional label and no judgement about whether the result matches the catalogue. This is the last place in the pipeline where the separation established in Section 4.1.1 could be broken, and it is enforced negatively: a test asserts that the serialised report contains none of the words *performance*, *security*, *maintainability*, *improvement*, *degradation*, *expected*, *better* or *worse*, outside the note explaining the sign convention.

Two further properties follow from the same principle.

**Deltas are carried, not recomputed.** Each delta arrives from the component that measured it and is serialised unchanged. Two independent computations of one quantity could disagree, and the measuring component is the authority on its own metric. The Report Generator is therefore the only component that produces no number of its own.

**One derivation, two formats.** to_text reads the report's own fields and recomputes nothing, so the JSON and the human-readable summary cannot drift apart. A test tampers with a field of the report object and asserts that the tampered value reaches the summary, which proves the summary is derived rather than independently assembled.

**An empty detection list is a valid report.** Where no supported refactoring is recognised, the detection section is empty and the four metrics are still measured and reported. This mirrors the behaviour of the Detection Engine described in Section 5.3.2: the absence of a detection is a finding, not a failure, and the report says so rather than aborting.

### 5.5.3 Provenance

To satisfy reproducibility requirements (RNF5), every generated report embeds an environment provenance block. This block records the exact active versions of the Docker Engine, Hadolint, Trivy, the Trivy vulnerability database timestamp, and helper Python modules during that precise execution. Rather than serving as a static installation checklist, this block acts as an immutable audit trail of the test run, capturing the exact runtime context and database state that participated in that specific evaluation.

RNF5 requires every report to record the analysis timestamp, the version of each external tool invoked, and the Trivy vulnerability database version. The Report Generator assembles that block from values the earlier stages hand it: the Docker Engine version read by the preparation phase, as described in Section 5.4.1, and the Hadolint, Trivy, database and dockerfile-parse versions returned by tool_provenance(), as described in Section 5.4.2.

The ordering matters and is the orchestrator's responsibility rather than this component's. Provenance is gathered after the measurements complete, because Trivy populates its database cache during the first scan and a version query made beforehand would report the database as absent. Collecting it afterwards makes the recorded database version the one the scans actually used, as set out in Section 5.1.1.

### 5.5.4 A naming correction

The conformance verification described in Section 5.6 found one discrepancy affecting this component, and it was worth correcting rather than tolerating.

The Data Extractor exposed its instruction delta as delta_instructions, while RF7 names the field delta_logical_instructions. The name was corrected across the component and its tests, so that it is now identical from the point of measurement to the point of serialisation. A tolerated mismatch would have obliged the Report Generator to translate a key, and a translation layer is exactly where a metric silently becomes a different metric.

The four keys the report emits are delta_size, delta_warnings, delta_cves and delta_logical_instructions, which are the names RF7 specifies. A test asserts the set of delta_* keys in the serialised report against that list, so the nomenclature cannot drift from the design chapter without a failure.

### 5.5.5 Testing and end-to-end verification

The component carries 35 tests in tests/test_report_generator.py and reaches 100% statement coverage.

Beyond the metric content and the RNF5 block, the suite covers the four artifacts written to the expected paths, the output directory created when absent, one raw file per tool, one raw file per tool, both states distinguishable inside each file, the retained findings accounting for the values counted, non-JSON output preserved rather than discarded, the JSON pointing at the raw files, the summary naming them, and LF endings in both text artifacts. Two tests enforce the properties of Section 5.5.2 directly: the vocabulary exclusion and the tampered-field propagation.

The component was also the occasion for the first full end-to-end run of the prototype, executed against the R09 catalogue pair under WSL with a live daemon: detection, historical export and build, both Stage 3 components dispatched concurrently, and the report. Table 5.8 records what it produced.

| **Item** | **Value** |
|---|---|
| **Detection** | R09 — Extract RUN Instructions |
| **ΔSize** | +416 bytes |
| **ΔCVEs** | +0 |
| **ΔWarnings** | +0 |
| **ΔInstr** | +1 |
| **Docker Engine** | 29.1.3 |

| Item | Value |
|---|---|
| **Hadolint** | 2.14.0 |
| **Trivy** | 0.72.0 |
| **Trivy database** | 2026-08-11T19:13:09Z |
| **dockerfile-parse** | 2.0.1 |
| **Retained evidence** | 50,648 bytes (Trivy), 34 bytes (Hadolint) |

Table 5.8 - Output of the first full end-to-end run, against the R09 catalogue pair

The three deltas that do not depend on the build reproduce the recorded catalogue values exactly. ΔSize differs from the recorded +417 by one byte, which falls inside the environmental variation band documented in Section 5.4.1 and is not attributable to any component of the pipeline. The volume of retained Trivy evidence also differs from the figures reported in Section 5.4.2, for the same class of reason: the vulnerability database is versioned and grows between runs, so the size of a scan report is a property of the database state at the moment of the scan rather than of the image alone.

The run also exercised the failure path against a real fault rather than a simulated one. A transient Trivy database download failure during the first attempt surfaced as DataExtractorError naming the failing scan and exited with code 5, which is the RNF4 contract and the mapping of Table 5.2 behaving exactly as specified.

## 5.6 Conformance Verification and Results

The preceding sections describe each component in isolation. This section reports the two verifications performed over the prototype as a whole: the test suite that exercises it, and the programmatic audit that checks it against the design chapter requirement by requirement.

### 5.6.1 Testing strategy

Verification was structured on two levels. Dedicated unit tests validate the isolated behaviour of each individual component, complemented by integration tests verifying the correct interaction and data flow between modules. This structured approach guarantees the internal integrity of the execution pipeline and ensures that every functional and non-functional requirement is exercised rather than assumed.

Three principles govern the suite.

External dependencies are replaced by fakes. The Docker daemon, Trivy, Hadolint and Git repositories are all substituted at the boundary of the component under test: a fake Docker client for the components that build or inspect images, a fake subprocess.run serving recorded tool JSON for the Data Extractor, and a temporary repository built in the fixture for the VCS Connector. The suite therefore runs without a daemon, without network access, and without pulling a tool image.

Every fake is paired with a real invocation. Each component that speaks to an external tool also carries one end-to-end test exercising the real thing, guarded by a skipif and running when the dependency is available. A fake proves the component handles a shape correctly; only the real invocation proves the shape is the one the tool actually emits.

Requirements are tested by name. Tests are grouped by the requirement they satisfy rather than by the function they call, so that a requirement without coverage is visible as an absent group rather than hidden among passing tests. This is what made the two coverage gaps of Section 5.6.3 findable.

### 5.6.2 Test inventory

The suite holds 302 collected tests across eight files, comprising 302 passing tests, 0 skipped, and 0 failures across 296 test functions. Two files hold more test cases than test functions, because one function in each is parametrised: the Data Extractor carries 42 functions comprising 44 cases, one running over three malformed-report shapes, and the orchestrator carries 34 functions comprising 38 cases, one running over five exit-code mappings. When executed within the native WSL2/Ubuntu environment with a reachable Docker daemon and the external support repository configured, all tests execute successfully without environment-dependent skips. Detailed, step-by-step instructions for setting up this environment, cloning the support repository (docker/getting-started), configuring the GETTING_STARTED_REPO environment variable, and reproducing the test suite are documented in the project's README.md repository file. Table 5.9 gives the test distribution.

| **Module or component** | **Test file** | **Tests** | **Requirements exercised** |
|---|---|---|---|
| VCS Connector | test_vcs_connector.py | 8 | RF1, RNF4 |
| VCS Connector (integration) | test_vcs_connector_getting_started.py | 1 | Experiment 3 replication |
| Detection Engine | test_detection_engine.py | 136 | RF2, RF3, RNF3 |

| Module or component | Test file | Tests | Requirements exercised |
|---|---|---|---|
| Preparation phase | test_image_builder.py | 25 | RNF4 |
| Performance Analyzer | test_performance_analyzer.py | 15 | RF4, RNF1, RNF4 |
| Data Extractor | test_data_extractor.py | 44 | RF5, RF6, RNF2, RNF4, RNF5 |
| Report Generator | test_report_generator.py | 35 | RF7, RNF5 |
| Pipeline orchestrator and CLI | test_pipeline.py | 38 | Stage sequencing, resource lifetime, exit codes |
| **Total** | — | **302** | **302 passing, 0 skipped** |

Table 5.9 - Distribution of the test suite across the modules of the prototype

### 5.6.3 Coverage audit

Table 5.10 gives the per-module statement coverage figures across the complete prototype package when executed in the target WSL2 environment with full external tool availability.

| Module | Statements | Uncovered | Coverage |
|---|---|---|---|
| vcs_connector.py | 25 | 0 | 100% |
| detection_engine.py | 824 | 45 | 93% |
| image_builder.py | 83 | 2 | 98% |

| Module | Statements | Uncovered | Coverage |
|---|---|---|---|
| performance_analyzer.py | 38 | 0 | 100% |
| data_extractor.py | 144 | 16 | 96% |
| report_generator.py | 94 | 0 | 100% |
| pipeline.py | 77 | 6 | 92% |
| **Package total** | 1285 | 72 | **94%** |

Table 5.10 - Statement coverage per module across the complete prototype package

Classifying the 72 uncovered statements (as reported in Table 5.10) produced the finding the audit existed to look for. Four are unreachable in practice: two in the Data Extractor and two in the Detection Engine, both the same except guarding the dockerfile-parse call, which cannot fire because the parser accepts arbitrary text and treats the first word of each line as an instruction rather than raising. Two are reachable only through an external fault, in the handler for a daemon that answers but reports no version. Nine are defensive guard branches inside individual rules, for which no claim of unreachability is made, only that no test reaches them. The remaining six sit in the orchestrator (pipeline.py), which is the least covered module of the package at 92% and the only one written after the audit was first performed.

Two of the 72 were the finding. Both sat in the Detection Engine, both were reachable, and both implemented behaviour the engine relies on rather than defensive code: digest parsing in _parse_from_value, which is what allows R02 to fire correctly on a base image pinned by digest, and the relative invocation form in _invokes_script, which is what allows R09 to detect a script called as RUN ./setup.sh rather than by absolute path. A regression in either would have passed the suite silently.

Three tests were added to close the gap, with no production change: one exercising R02 on a FROM carrying both a digest and a tag, its negative counterpart asserting that a changed digest is a different entity and therefore not R02, and one exercising R09 on the relative invocation form.

Notably, statement coverage in data_extractor.py reaches 96% (with only 6 uncovered lines), directly benefiting from the execution of real end-to-end integration tests against Hadolint and Trivy containers under the WSL2 runtime.

### 5.6.4 System-wide conformance verification

The components were systematically verified against the architectural specifications established in Chapter 4, comparing the implemented interfaces, metric names, and behavioural guarantees with those defined during the design phase. This alignment ensures that the implemented code strictly adheres to the planned system structure and maintains consistency across the development lifecycle.

Table 5.11 maps each component to the requirements it carries and records the outcome.

| **Component** | **Requirements** | **What was checked** | **Result** |
|---|---|---|---|
| VCS Connector | RF1, RNF4, RNF5 | Blob-level access through data_stream; no working-tree operation anywhere in the module; three distinct exception types; idempotence across repeated calls | Conformant |
| Detection Engine | RF2, RF3, RNF3 | Fourteen rules in a declarative registry; DetectionResult carrying no impact field; no read of the extended catalogue; registration through the decorator alone | Conformant |
| Performance Analyzer | RF4, RNF1, RNF4 | image.attrs["Size"] read through the SDK; no subprocess import in the module, so the rejected CLI path is unreachable; five failure modes distinguished | Conformant |
| Data Extractor | RF5, RF6, RNF2, RNF4, RNF5 | Distinct identifiers deduplicated globally; per-severity vector; 47-rule screen set-identical to the catalogue filter; no import of the Detection Engine; retention of raw_trivy and raw_hadolint as Section 4.6 requires | Conformant |
| Report Generator | RF7, RNF5 | Structured output carrying before and after values, the four deltas and the provenance | Conformant after one correction |

| Component | Requirements | What was checked | Result |
|---|---|---|---|
| | | block; artifacts written and referenced | |
| All components | RF1 (single-file scope) | No Dockerfile discovery mechanism anywhere in the package | Conformant |

Table 5.11 - Conformance of each component against the requirements of the design chapter

Three of these checks deserve comment, because each is verified by a property of the code rather than by an assertion about its output.

The Performance Analyzer cannot use the CLI. RNF1 demands byte-level precision and Section 4.5.1 rejects the Docker CLI on rounding grounds. The audit verified this by the absence of any subprocess import in the module, which makes the rejected path unreachable rather than merely unused. A requirement enforced by what a module cannot do is stronger than one enforced by what it happens to do.

The Data Extractor cannot couple to the Detection Engine. Both use dockerfile-parse, and a shared helper would have let a change to detection move a maintainability metric. The audit verified the separation by reading the module's own source and failing if the string detection_engine appears in it, so the coupling cannot reappear by oversight.

No component can discover a Dockerfile. RF1 binds the prototype to a single file path supplied by the caller. The audit verified that no module performs directory traversal or filename matching, so a second Dockerfile is unreachable rather than merely unused, which is what makes the scope limitation of Section 5.1.5 a structural property rather than a convention.

One correction was applied. The Data Extractor exposed its instruction delta as delta_instructions while RF7 names the field delta_logical_instructions. The name was corrected across the component and its tests, so that it is identical from the point of measurement to the point of serialisation, and a test now asserts the set of delta_* keys in the serialised report against the four names RF7 specifies. This was the only discrepancy the audit found in code.

One divergence was resolved in the manuscript rather than in the code. The audit also reported that an RF5 acceptance criterion requiring ΔWarnings of −1 for DL3020 could not be satisfied, because DL3020 is absent from the 47-rule screen. The code was found correct: Appendix C derives the screen by dimensional exclusion, removing rules that target security surfaces so that the maintainability metric does not duplicate what ΔCVEs measures, and DL3020 is excluded by exactly that criterion. The requirement text carried a criterion predating the screen, and the reference was removed from the manuscript. This is the one case in the implementation where a conformance finding was resolved by correcting the specification rather than the code, and it is recorded here rather than silently, because a specification that changes to match an implementation deserves the same scrutiny as the reverse.

One naming choice was reconciled rather than changed. The Performance Analyzer names its output structure SizeMetric instead of reusing MetricSet, which the Data Extractor already applies to a structure with entirely different fields. Two distinct names for two distinct shapes keep the Report Generator's imports unambiguous, and since the design chapter does not name the Performance Analyzer's output type, nothing constrains the choice. The audit recorded the divergence as deliberate rather than treating it as a defect.

### 5.6.5 Summary of results

The prototype implements the five components of Section 4.1.6 across the four stages of Table 5.1, executes end to end from the command line, and produces the four delta metrics on real repositories. Every functional requirement from RF1 to RF7 and every non-functional requirement from RNF1 to RNF5 is exercised by a named group of tests and confirmed by the programmatic audit. The suite stands at 302 collected tests with 302 passing, 0 skipped, and 0 failures when executed in the native WSL2 environment maintaining a robust package coverage of 94% across 1285 statements (with only 72 uncovered statements).

Two limitations are recorded as structural rather than incidental. The single-file scope of RF1 produces the $\Delta\text{Instr}$ divergence on R11 reported in Section 5.4.2, where the prototype measures −3 against the +1 of the catalogue's project-level accounting, because the extracted Dockerfile lies outside what the prototype reads; the choice was kept deliberately, on the grounds that automatic aggregation of satellite files would require heuristics fragile enough to compromise the determinism of the measurement pipeline. And image size is not bit-for-bit reproducible in the execution environment, as established in Section 5.4.1, which bounds from below the size deltas that can be distinguished from environmental variation.

Neither limitation was discovered by the static code audit. Both emerged organically during the iterative development and practical testing of the prototype. The single-file scope restriction (RF1) and its resulting $\Delta\text{Instr}$ divergence on R11 became apparent when evaluating how refactorings manifest across real-world repositories, where multi-file splits follow unpredictable patterns with arbitrary naming conventions (e.g., Dockerfile.builder) and varying subfolder contexts. Attempting to track these files automatically would necessitate complex, fragile guessing heuristics that compromise pipeline determinism and statistical consistency, making multi-file tracking a deliberate candidate for future work. Similarly, the non-reproducibility of image sizes was uncovered empirically when repeated executions of identical rule pairs yielded slight byte-level variations due to external Docker environment volatility, mirror updates, and daemon factors, establishing a baseline environmental noise floor against which micro-variations must be interpreted.

# 6 Experimentation and Evaluation

Design Science Research prescribes that an artefact be assessed against the objectives that motivated its construction, and that the assessment be conducted with criteria established independently of the artefact itself. This chapter reports the Evaluation activity of the methodology adopted in this dissertation, taking as its object the prototype whose design and development were presented in the preceding chapter.

The artefact under evaluation is an automated four-phase pipeline. It receives a version-controlled repository, a pair of commits delimiting a change, and the Dockerfile under analysis, and returns a structured report of the refactorings identified and of their measured impact. The first phase collects data, retrieving the two Dockerfile states from the version history. The second identifies which of the catalogue refactorings are present between them. The third constructs the corresponding pair of images and measures the four quality metrics over them. The fourth aggregates identifications with measurements and emits the result. From this structure follow two distinct capabilities: the artefact identifies refactorings between commit pairs, a determination made from the structure of the two states, and it measures their downstream empirical impact, a determination that depends on constructing images and on external analysis.

The independence that the methodology demands is satisfied by three properties of the evaluation design rather than by assertion. First, the expected behaviour was specified in advance, since the criteria against which identification is judged were written before the artefact was executed over the evaluation material, and state what the artefact must do rather than what it was observed to do. Second, the artefact has no access to those criteria: identification is decided from the structure of the two Dockerfile states alone, and no phase of the pipeline consults the expected result at any point during execution. Third, execution and adjudication are separate procedures, since the artefact produces and records its output without reference to the criteria and the comparison against them is performed afterwards, by a distinct procedure. An execution therefore cannot be shaped by the answer against which it is subsequently measured.

The evaluation addresses two questions, which are answered by different means and are kept apart throughout. The first concerns identification, namely whether the artefact recognises a refactoring when it is present and abstains when it is not. The second concerns impact measurement, namely whether the four metrics respond to the transformation they are intended to capture, and whether the magnitude of that response can be attributed to the refactoring rather than to the environment in which it was obtained.

The two questions admit different standards of evidence because they rest on different foundations. Identification is determined by the detection logic alone, operating on the two Dockerfile states and depending on no external component, so it is deterministic in that identical inputs yield identical identifications, and it is exactly reproducible. Measurement depends on external analysers and on a build environment that the artefact does not control, so its results are conditional, holding under an environment that must be recorded alongside them.

This separation is structural rather than presentational. The measurement phase does not receive the result of the identification phase: it operates on the Dockerfile pair and on the constructed images, without knowledge of which refactoring was identified. An identification

therefore cannot bias a measurement, nor can a measurement be used to infer an identification. The independence established at design time is preserved at evaluation time, and each question can be answered without contaminating the other.

Three conditions bound the reproducibility of measurement, and they frame the reading of every value reported in this chapter. The versions of the external analysers are not fixed by the artefact, so an analyser version is a property of when an analysis was performed rather than of the code that performed it, and each report consequently carries a provenance record of the toolchain that produced it. Vulnerability findings depend on an evolving knowledge base, so reproducing a vulnerability count requires the specific state of that knowledge base, which the provenance record identifies. Image construction is not byte-reproducible in this environment, so size deltas below a threshold of environmental variation cannot be distinguished from it.

This asymmetry between the two questions is a finding of the evaluation and not a weakness to be minimised. Identification is deterministic; measurement is conditional upon a recorded environment. Establishing the distinction at the outset is what motivates the provenance record and the retention of unmodified analyser output, and it governs how every reported value is to be read.

Four metrics are evaluated. Image size, expressed in bytes, addresses the performance dimension. The count of distinct vulnerability identifiers, stratified by severity, addresses security. The count of linter findings over the screened rule set and the count of logical Dockerfile instructions both address maintainability, the former capturing declared quality defects and the latter the structural weight of the file. All four are expressed as the difference between the state after the change and the state before it. For image size, vulnerabilities and linter findings the direction of improvement is fixed, since a negative difference is an improvement in every case. The count of logical instructions admits no such fixed direction, and this exception requires statement at the outset because it governs the reading of every such value in the chapter.

The sign of that metric is determined by the class of refactoring applied rather than by whether the outcome is desirable, and both directions are correct outcomes for the classes that produce them. Refactorings of consolidation and removal, such as inlining RUN instructions, collapsing a build stage or eliminating a relocation instruction, reduce the count, and the reduction is their intended effect. Refactorings of extraction and addition, such as introducing a build stage, declaring an environment variable or adding a build argument, increase it, and the increase is equally their intended effect. The metric must therefore never be read in isolation as a quality signal, since it is interpretable only jointly with the identity of the refactoring that produced it. An increase following an extraction is the expected outcome and not a regression, whereas a decrease following an addition would constitute an anomaly. Deferring this convention would leave the first reported value open to misinterpretation.

The chapter opens in Section 6.1 by recording the technical environment and the execution mechanics common to every experiment reported in it, so that any figure presented subsequently can be located in the configuration that produced it. Section 6.2 then evaluates detection accuracy across the fourteen catalogue refactorings, answering the question of identification. Section 6.3 follows, evaluating the measurement of impact across the quality dimensions under controlled scenarios: performance through image size, security through the vulnerability count and its stratification, and maintainability through the linter warning and logical instruction counts. Finally, Section 6.4 provides a discussion of the results, and Section

6.5 addresses Threats to Validity, consolidating structural limitations, risk and bias factors, and methodological mitigations that bound the interpretation of what precedes them.

This sequence is not arbitrary. Detection accuracy is treated before the measurement sections because a measurement is meaningful only once the refactoring to which it is attributed has been correctly identified, so identification is logically prior and its evaluation conditions the interpretation of everything that follows. The measurement sections proceed by quality dimension, matching the classification of the extended catalogue, so that each metric is evaluated against a scenario in which a refactoring of the corresponding dimension is applied in isolation. Section 6.5 then presents Threats to Validity, encompassing structural limitations, risk and bias factors, and methodological mitigations, treated together rather than distributed across the sections they affect, because the principal ones bear on more than one of them, allowing their combined effect on the rigour and validity of the evaluation to be assessed as a whole.

## 6.1 Experimental Setup

Every experiment reported in this chapter was executed within a single configuration, recorded here so that any figure presented subsequently can be located in the environment that produced it.

### 6.1.1 Hardware and operating system

The measurements were performed on a machine with an AMD Ryzen 7 7735HS processor and 15.2 GiB of RAM. All measurements and pipeline executions were conducted inside WSL2 running Ubuntu 26.04 LTS. Operating within this Linux environment is a strict precondition for the evaluation, ensuring that file paths, execution mechanics, and interactions with the Docker daemon remain consistent, standardized, and isolated from host-level variations.

### 6.1.2 Technology stack

The technology stack is recorded in three groups, distinguished by how each component is obtained and by what fixes its version, since these determine how reproducible each part of the environment is.

**Group A — Python packages:** GitPython 3.1.50, dockerfile-parse 2.0.1 and docker 7.1.0, among others. These are declared dependencies of the prototype and are pinned by version.

**Group B — Pre-installed system components:** Python 3.14.4, Git 2.53.0, Docker Engine 29.1.3 and Docker Buildx 0.36.1.

One property of this group requires explicit statement, because it determines which component is responsible for every image constructed in this chapter: the builds are performed by Buildx (BuildKit) and not by the Docker SDK. A build reported here is therefore a BuildKit build, and its behaviour is a property of Buildx 0.36.1 rather than of the Python package through which the pipeline addresses the daemon.

Group C — Containerised analysers: Trivy 0.72.0 and Hadolint 2.14.0.

These differ from the two preceding groups in a way that bears directly on reproducibility. Both are invoked without an explicit tag, which resolves to the locally cached image in the Docker store rather than fetching live at every execution. Consequently, an internet connection is required only when an analyser image is absent from the local store rather than at every scan invocation. The versions recorded above reflect the state of the local cache at the time of the analysis campaign, reinforcing the necessity of the provenance record.

## 6.2 Detection Accuracy

The detection engine is the analytical core of the prototype: every impact measurement reported downstream is attributed to the refactoring the engine identifies, so an error in identification propagates to every figure that depends on it. This section reports the inputs supplied to that component, the specification against which its output is judged, the procedures by which it was executed and audited, and the values obtained.

### 6.2.1 Evaluation corpus

Public Dockerfile histories were rejected as the primary evaluation instrument due to both practical and methodological barriers. Practically, real-world repositories present severe operational hurdles: initial base commits are often broken or corrupted at origin; they depend on missing local files, uncommitted assets, or unstable network dependencies; and they frequently rely on private internal images, confidential credentials, or environment secrets tied to specific production infrastructures. Methodologically, their coverage of the catalogue is incidental, since some refactorings occur frequently in the wild while others appear rarely or not at all, so an accuracy figure computed over them measures the distribution of practice rather than the capability of the engine. Furthermore, real commits mix refactorings with unrelated edits in ways that cannot be separated without prior interpretation, which would make the ground truth itself a matter of judgement.

A dedicated repository was therefore constructed, in which each commit is authored to exercise a defined portion of the catalogue (publicly accessible at https://github.com/DiogoVieira123/docker-refactoring-corpus). The corpus comprises 49 commit pairs distributed across three independent services, each with its own Dockerfile and its own evolution history:

- an API service, contributing 18 pairs;
- a queue worker, contributing 17 pairs;
- an authentication service, contributing 14 pairs.

The three differ in base image family, in build strategy, opposing single-stage to multi-stage, and in the vocabulary of their shell commands, so that the engine is not evaluated against a single stylistic idiom.

Coverage is systematic rather than opportunistic. Table 6.1 records the distribution across the fourteen catalogue rules and the four case classes.

| Rule | Canonical | False-negative | Mixed | False-positive | Total |
|---|---|---|---|---|---|
| **R01 — Inline RUN Instructions** | 1 | 1 | 2 | 1 | 5 |
| **R02 — Update Base Image TAG** | 1 | 1 | 1 | 1 | 4 |
| **R03 — Add ENV Variable** | 1 | 1 | 1 | 1 | 4 |
| **R04 — Add ARG Instruction** | 2 | 1 | 1 | 1 | 5 |
| **R05 — Extract Stage** | 0 | 1 | 2 | 1 | 4 |
| **R06 — Inline Stage** | 1 | 1 | 1 | 1 | 4 |
| **R07 — Sort Instructions** | 1 | 1 | 1 | 1 | 4 |
| **R08 — Replace ADD with COPY** | 1 | 1 | 1 | 1 | 4 |
| **R09 — Extract RUN Instructions** | 1 | 1 | 1 | 1 | 4 |
| **R10 — Update Base Image** | 1 | 1 | 2 | 1 | 5 |
| **R11 — Move Stage** | 1 | 1 | 0 | 1 | 3 |
| **R12 — Remove RUN Instruction** | 1 | 1 | 1 | 1 | 4 |
| **R13 — Update RUN Instruction** | 1 | 1 | 3 | 1 | 6 |

| Rule | Canonical | False-negative | Mixed | False-positive | Total |
|---|---|---|---|---|---|
| **R14 — Rename Image** | 1 | 1 | 3 | 1 | 6 |

Table 6.1 - Distribution of the 49 commit pairs across the fourteen catalogue rules and the four case classes. A pair performing several refactorings is counted once under each rule it exercises, which is why the row totals exceed the number of pairs.

Each pair belongs to one of four classes, defined so that the corpus measures sensitivity and specificity independently rather than conflating them into a single aggregate.

Canonical cases, 14 pairs. The refactoring in the form the catalogue describes, applied in isolation.

False-negative tests, 14 pairs. The same refactoring expressed through a syntactic variant that a naive implementation would miss: the exec form of RUN rather than the shell form, a heredoc rather than backslash continuations, an image tag resolved through a build argument rather than written literally, a path carrying a trailing separator.

Mixed cases, 7 pairs. Commits performing several refactorings at once. Under the non-exclusion principle adopted in the design, every rule whose criteria are satisfied must be reported.

False-positive tests, 14 pairs. Changes that resemble a refactoring without satisfying it, one for each rule. A URL argument of an ADD instruction changes version while remaining an ADD, which is not a replacement by COPY; a LABEL is split across continuation lines, which is not the reformatting of a RUN; a chmod is folded into a transfer flag, which is not the elimination of a relocation; a registry prefix is made explicit, which does not substitute the image it qualifies; a build argument is redeclared to bring it into a stage's scope, which does not introduce a new argument. These pairs carry the identifier of the rule that must not fire, inverting the usual reading of the specification.

### 6.2.2 External specification

The expected behaviour of the engine on each pair is recorded in a human-readable specification document, mapping.md, kept under experiment and validation/detetion/ in the project repository. Each entry defines, for one pair, the artefact analysed, the two commit identifiers delimiting the change, the case class, the rules that apply, and, where present, a prose statement of the expected detection.

Two properties of this document bear on the validity of the evaluation. First, it is a specification rather than a log, since it was authored alongside the corpus commits and states what the tool must do, not what it was observed to do. Second, it is external to the tool: the detection engine has no access to it at any point in its execution, and detection is decided from the structure of the two Dockerfile states alone, in accordance with the requirement

that the engine remain autonomous of the catalogue. The specification is consulted only by the audit procedure, after detection has completed and its results have been written to disk.

### 6.2.3 Tool execution

Execution and adjudication are separate procedures, so that an execution cannot be shaped by the criteria against which it is subsequently measured. Prior to running the analysis pipeline, the repository's provided Makefile is executed to build and cache local custom base images and dependencies within the Docker daemon.

The evaluation was conducted by means of a runner script that invoked the prototype systematically across the corpus. For each of the 49 pairs, the script executed the tool through its command-line interface with the corresponding repository path, commit identifiers, and Dockerfile location, replicating the exact invocation a human operator would perform, but applied uniformly across all pairs. Each execution wrote its report and its raw tool output to a designated directory (pair_0 through pair_48), accompanied by a log recording the exact command issued, output streams, and process exit code. This stage computed no metric and compared nothing on the fly, recording strictly what occurred.

### 6.2.4 Automated audit

The audit is performed by a program, audit_corpus.py, written so that the correspondence between the specification and the tool output is established mechanically rather than by inspection.

For each pair the program reads the report produced by the tool, extracts the set of refactoring identifiers reported, and applies the check the pair's case class requires. For canonical, false-negative and mixed cases, every rule listed in the specification must appear in the report, and its absence is recorded as an error. For false-positive cases, the listed rule must be absent, and its presence is recorded as an error. Any unlisted detection reported on a positive case is likewise treated as an error.

The output of the audit was itself reviewed manually, pair by pair, to verify that the program had not failed.

A single strict conformance rate is computed, representing the proportion of pairs on which the set of rules reported matches the specification exactly, with zero false negatives, zero false positives, and any unlisted detection treated as an error.

### 6.2.5 Results

The prototype was executed over the complete corpus. All 49 pairs were analysed successfully, each producing a report, a human-readable summary and the raw output of the external analysis tools.

The engine recorded a strict conformance rate of 100% (49/49), with zero false negatives, zero false positives, and zero unlisted detections. Table 6.2 records the conformance by case class.

| Case class | Pairs | Conforming | Expectation |
|---|---|---|---|
| **Canonical** | 14 | 14 | every listed rule reported |
| **False-negative test** | 14 | 14 | every listed rule reported |
| **Mixed** | 7 | 7 | every listed rule reported |
| **False-positive test** | 14 | 14 | no listed rule reported |
| **Total** | 49 | 49 | — |

Table 6.2 - Conformance of the detection engine with the reference specification, by case class

## 6.3 Evaluation of Metric Impact (Impact Measurement Reliability)

### 6.3.1 Performance Impact (ΔSize)

The performance dimension is measured through image size, expressed as the difference in bytes between the image built from the state after the change and the image built from the state before it. This subsection reports the controlled scenario constructed to exercise that measurement, the invocation performed, and the values obtained.

#### 6.3.1.1 Rationale for an isolated scenario

The measurement was first exercised against the corpus used for detection, where a commit pair substituting a heavier base image for a lighter one was analysed. The prototype reported a reduction, and the direction of the effect was captured, but the magnitude was 11 bytes. In that pair the substitution occurs in a build stage whose base image does not reach the final image; the delta therefore reflects only the residual difference in the artifacts copied from it rather than a direct reduction of the final image.

A dedicated scenario was consequently constructed. It consists of a single-stage Dockerfile carrying one instruction, so that the size delta reflects the base-image substitution alone and no additional instruction interferes with it.

The repository built for this purpose is local and was not published, unlike the corpus used for the detection evaluation. The script that reproduces the procedure, validate_r10_isolated.sh, is available in the author's GitHub repository, so the scenario can be reconstructed and the steps below repeated in full.

#### 6.3.1.2 Repository preparation and commit pair

The scenario applies Update Base Image (R10), substituting debian:12-slim for alpine:3.19. The repository was initialised as follows.

```
mkdir -p /var/tmp/test-r10-isolated && cd /var/tmp/test-r10-isolated && git init
```

Listing 6.1 - Initialization of the isolated local repository for performance validation under /var/tmp

The before state declares the heavier base image:

```
echo "FROM debian:12-slim" > Dockerfile
git add Dockerfile && git commit -m "debian base"
export HASH_BEFORE=$(git rev-parse HEAD)
```

Listing 6.2 - Creation of the initial commit and capture of the before-state commit hash for the performance validation scenario.

The after state declares the lighter one:

```
echo "FROM alpine:3.19" > Dockerfile
git add Dockerfile && git commit -m "alpine base"
export HASH_AFTER=$(git rev-parse HEAD)
```

Listing 6.3 - Creation of the after-state commit by switching the base image to Alpine and capturing its commit hash for performance validation.

The two Dockerfiles contain a single FROM instruction each, and differ in nothing else.

#### 6.3.1.3 Tool invocation

The prototype was invoked through its command-line interface, receiving the repository path, the two commit identifiers, and the output directory:

```
python3 main.py /var/tmp/test-r10-isolated $HASH_BEFORE $HASH_AFTER \
 -o /var/tmp/test-r10-isolated/output
```

Listing 6.4 - Invocation of the prototype over the performance validation commit pair, specifying the repository path, commit hashes, and the output directory

The result was read from the human-readable summary the tool emits:

```
cat /var/tmp/test-r10-isolated/output/human_readable_summary.txt
```

Listing 6.5 - Inspection of the human-readable summary report generated by the prototype for the performance validation scenario

#### 6.3.1.4 Values obtained

| Metric | Before | After | Delta |
|---|---|---|---|
| **Image size (bytes)** | 28,234,415 | 3,421,731 | −24,812,684 |

Table 6.3 - Image size measured over the isolated R10 commit pair, as reported by the prototype

### 6.3.2 Security Impact (ΔCVEs)

The security dimension is measured through the count of distinct vulnerability identifiers, expressed as the difference between the state after the change and the state before it. This subsection reports the scenario constructed to exercise that measurement, the repository preparation, the tool invocation, and the values obtained.

#### 6.3.2.1 Repository preparation and commit pair

The scenario applies Update Base Image TAG (R02), substituting `node:18.16.0-alpine` for `node:18.16.0`. The repository was initialised as follows:

```
mkdir -p /var/tmp/demo_test_env && cd /var/tmp/demo_test_env && git init -q
git config user.email "security-validation@local"
git config user.name "Security validation"
```

Listing 6.6 - Initialization of the isolated local repository and Git configuration for the security validation scenario.

The before state declares the heavier Debian-based image:

```
printf 'console.log("App running");\n' > app.js
cat > Dockerfile <<DOCKERFILE
FROM node:18.16.0
WORKDIR /app
COPY . .
CMD ["node", "app.js"]
```

```
DOCKERFILE
git add . && git commit -q -m "chore: initial state on the Debian base"
export HASH_BEFORE=$(git rev-parse HEAD)
```

Listing 6.7 - Creation of the initial state application file (app.js) and Dockerfile using an older Node.js base image for the security validation scenario.

The after state declares the lighter Alpine variant:

```
cat > Dockerfile <<DOCKERFILE
FROM node:18.16.0-alpine
WORKDIR /app
COPY . .
CMD ["node", "app.js"]
DOCKERFILE
git add . && git commit -q -m "refactor(dockerfile): move to the Alpine base"
export HASH_AFTER=$(git rev-parse HEAD)
```

Listing 6.8 - Creation of the after-state Dockerfile shifting to the Alpine-based image variant, committing the change, and capturing the commit hash for security validation.

The two Dockerfiles differ in the `FROM` tag and in nothing else. The script that recreates this local repository and executes the prototype over it is available in the author's repository under `experiment and validation/measurement impact/security/security_validation.sh`.

#### 6.3.2.2 Tool invocation

The prototype was invoked through its command-line interface, receiving the repository path, the two commit identifiers, and the output directory:

```
python3 main.py /var/tmp/demo_test_env $HASH_BEFORE $HASH_AFTER \
 -o security_validation_report
```

Listing 6.9 - Invocation ofței prototype over the security validation commit pair, specifying the repository path, commit hashes, and the output report directory.

#### 6.3.2.3 Inspection of the artefacts

Once the prototype had been executed, the consolidated raw report (trivy_raw.json) and the structured report (impact_report.json) were opened and inspected. The before state, corresponding to the Debian-based image, was examined and specific CVE identifiers of

CRITICAL and HIGH level were located within it. The after state, corresponding to the Alpine variant, was then examined, and the identifiers located in the preceding state were absent from it.

The packages no longer present in the after state include compilation toolchains such as `gcc-12@12.2.0-14` and `g++-12`, together with pre-installed system and multimedia utilities such as `imagemagick@8:6.9.11.60+dfsg-1.6` and `curl@7.88.1-10`, and libraries such as `krb5-multidev`. These constitute a sample of the inventory of packages and vulnerabilities recorded for the before state.

#### 6.3.2.4 Values obtained

| Severity | Before | After | Delta |
|---|---|---|---|
| **CRITICAL** | 56 | 2 | −54 |
| **HIGH** | 1,894 | 21 | −1,873 |
| **MEDIUM** | 5,991 | 15 | −5,976 |
| **LOW** | 954 | 11 | −943 |
| **UNKNOWN** | 45 | 0 | −45 |
| **Total** | **8,940** | **49** | **−8,891** |

Table 6.4 - Vulnerability counts by severity over the isolated R02 commit pair, as reported by the prototype.

#### 6.3.2.5 Deduplication of identifiers

The processed report, `impact_report.json`, applies a deduplication filter, so that each unique CVE identifier contributes a single unit to the count regardless of how many packages carry it. The raw file, `trivy_raw.json`, retains the complete output for both states exactly as Trivy emits it, repeating the same identifier once for each affected package.

The filter follows from what a vulnerability identifier designates. A CVE points to a problem of origin rather than to a location within the scanned inventory, and correcting that problem resolves every occurrence of the identifier at once. Counting the same identifier repeatedly, once per affected package, therefore multiplies the entry without adding information to the final count, and obscures the number of distinct problems the state actually carries.

### 6.3.3 Maintainability Impact (ΔWarnings and ΔInstr)

The maintainability dimension is addressed by two metrics: the count of linter findings over the screened rule set, and the count of logical Dockerfile instructions. Both are expressed as the difference between the state after the change and the state before it. This subsection reports the scenario constructed to exercise them and the values obtained.

#### 6.3.3.1 Experimental input

A controlled local repository was built holding a single commit pair, in which the before state carries two maintainability defects at once and the after state applies the corresponding refactorings together, so that both metrics are exercised in one run.

The before state raises two warnings. DL3007 is raised against the mutable image tag, and belongs to the 47-rule maintainability screen derived from Appendix C, under the Reproducibility subcategory. DL3059 is raised against the two consecutive RUN instructions producing separate layers.

The commit applies two catalogue refactorings jointly. Update Base Image TAG (R02) substitutes ubuntu:latest for the pinned ubuntu:22.04, removing the condition that raises DL3007, which is the screened rule and therefore the one that moves the filtered warning count. Inline RUN Instructions (R01) chains the two separate RUN instructions into a single one, removing the condition that raises DL3059 and reducing the logical instruction count in the same operation.

The scratch repository is created under /var/tmp rather than /tmp, since the latter is cleared when the WSL2 virtual machine shuts down, and is kept off a Windows mount so that Git does not refuse it under its dubious-ownership guard.

```
REPO=/var/tmp/docker_validation
rm -rf "$REPO"
mkdir -p "$REPO"
cd "$REPO"
git init -q
git config user.email "maintenance-validation@local"
git config user.name  "Maintenance validation"
```

Listing 6.10 - Initialization and configuration of the scratch repository for maintainability validation under /var/tmp.

The before state declares the mutable tag and the two separate RUN instructions:

```
cat > Dockerfile <<DOCKERFILE
FROM ubuntu:latest
RUN apt-get update
```

```
RUN apt-get install -y curl
DOCKERFILE


git add Dockerfile
git commit -q -m "chore: initial state with a mutable tag and two RUN layers"
HASH_BEFORE="$(git rev-parse HEAD)"
```

Listing 6.11 - Creation of the before-state Dockerfile with a mutable base image tag and separate RUN instructions for maintainability validation, along with commit hash capture.

The after state pins the tag and consolidates the two instructions into one:

```
cat > Dockerfile <<DOCKERFILE
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y curl
DOCKERFILE


git add Dockerfile
git commit -q -m "refactor(dockerfile): pin the base image tag and inline the RUN instructions"
HASH_AFTER="$(git rev-parse HEAD)"
```

Listing 6.12 - Creation of the after-state Dockerfile pinning the image tag and inlining the RUN instructions for maintainability validation, along with the subsequent commit and hash capture.

#### 6.3.3.2 Tool invocation

The prototype was invoked over the commit pair, with both output streams captured to a log:

```
REPORTS="/var/tmp/maintainability_validation_report"
mkdir -p "$REPORTS"
python3 main.py "$REPO" "$HASH_BEFORE" "$HASH_AFTER" -o "$REPORTS" \
  2>&1 | tee "$REPORTS/terminal_output.log"
```

Listing 6.13 - Invocation of the prototype over the maintainability validation commit pair, specifying the output directory and capturing both output streams to a log file.

The procedure is packaged as a script, run_maintenance_validation.sh, which recreates the repository, runs the prototype, and verifies that the two artefacts to be inspected,

impact_report.json and raw_data/hadolint_raw.json, are present and non-empty. The artefacts are available in the author's GitHub repository, under experiment and validation/measurement impact/manutenibility/, covering both the logical instruction counter and the use of the external Hadolint tool for counting warnings.

#### 6.3.3.3 Values obtained

The structured report was inspected and cross-referenced against the raw Hadolint output.

| Metric | Before | After | Delta |
|---|---|---|---|
| Warnings (filtered) | 2 | 1 | −1 |

Table 6.5 - Filtered warning counts over the maintainability commit pair, as recorded in the metrics.warnings block of the structured report.

These figures correspond to the values registered in the warnings block of the JSON report, and represent the filtered count resulting from the 47-rule catalogue screen. The count of 2 in the before state comprises DL3007 and DL3008; the remaining screened warning, DL3008, is present in both states and therefore does not contribute to the delta. The delta of −1 corresponds specifically to the elimination of DL3007 by R02, that rule being the one of the two within the screen that was refactored.

| Metric | Before | After | Delta |
|---|---|---|---|
| Logical instructions | 3 | 2 | −1 |

Table 6.6 - Logical instruction counts over the same commit pair, recorded under metrics.logical_instructions.delta_logical_instructions

The expected instruction counts for both states were established by counting them manually, the repository having been authored so that the resulting count was known in advance. The values reported by the prototype were then read from impact_report.json and compared against that manual count.

The raw Hadolint output was examined for both states. The DL3007 entry raised against the mutable tag appears in the before state and is absent from the after, so the warning delta traces to the specific rule that produced it. The subtraction recorded in impact_report.json was checked against those raw counts.

#### 6.3.3.4 Treatment of duplicate warnings

The prototype applies no deduplication to Hadolint warnings. The raw artefacts hold the output exactly as the tool emits it, never re-serialised, and the reported values and delta in human_readable_summary.txt and terminal_output.log are produced without any deduplication step.

The rationale rests on a property of the linter. Hadolint is positional: it reports the exact line at which each violation occurs. Suppressing warnings that share an identifier would conceal the fact that the same problem may occur at several points of the Dockerfile, at line 30 and again at line 40. Retaining the duplication ensures that every exact occurrence of each infraction remains visible to an auditor, and that the spatial relation between the finding and the code is not lost.

This treatment is specific to the maintainability dimension and is not the one applied elsewhere in the prototype. In the security dimension the raw artefacts likewise hold the scanner output as emitted, but the reported values and delta are produced with deduplication of identical CVE identifiers applied, for the reasons given in Subsection 6.3.2.

## 6.4 Discussion of Results

### 6.4.1 Detetion Engine

The detection engine recorded a strict conformance rate of 100% (49/49), with zero false negatives, zero false positives, and zero unlisted detections. Because the conformance evaluation requires an exact match between the expected specification and the tool's output, the result is unambiguous: every rule the specification requires was reported, no rule the specification forbids was reported, and no pair produced a detection outside the specification.

The significance of this outcome follows from the position the component occupies. The detection engine is the analytical core of the prototype, and every impact measurement reported downstream is attributed to the refactoring the engine identifies, so an error in identification propagates to every figure that depends on it. An accuracy result for this component therefore conditions the interpretation of every measurement reported elsewhere in this chapter.

Three aspects of the result merit comment, and a fourth concerns the mechanism by which it was established.

#### 6.4.1.1 Sensitivity to syntactic variation

The fourteen false-negative pairs were all detected, which indicates that the rules are formulated over the structure of the instruction rather than over its text. The engine recognised a consolidation expressed in the exec form of RUN, a command block rewritten as a heredoc, an image tag substituted through the default value of a build argument while the FROM line itself remained unchanged, and an instruction relocated and reformatted within the same commit. Each of these is a variant that a naive implementation would miss, and detecting all fourteen answers the question the class was designed to isolate: whether the engine recognises the operation or merely a spelling of it.

#### 6.4.1.2 Handling of co-detection

The seven mixed pairs were reported in full, with no omission and no spurious addition. These include commits combining a consolidation with a keyword replacement and a reformatting; a stage extraction with a base-image substitution and a stage rename; and a relocation absorbed into a transfer together with a command extraction and an instruction reordering. The result confirms that the rules operate independently. Each judged its own subset of instructions without being disabled by concurrent changes to instructions belonging to other rules, which is the behaviour the non-exclusion principle requires and the property that a single-refactoring corpus would leave untested.

#### 6.4.1.3 Specificity

The fourteen control pairs produced no detection whatsoever, not merely an absence of the forbidden rule, but an empty result set in every case. This is a stronger outcome than the specification demands, and indicates that the discriminating conditions of each rule reject near-miss changes rather than being satisfied by superficial resemblance. These pairs resemble a refactoring without satisfying it: a URL argument of an ADD instruction changing version while remaining an ADD, which is not a replacement by COPY; a LABEL split across continuation lines, which is not the reformatting of a RUN; a chmod folded into a transfer flag, which is not the elimination of a relocation; a registry prefix made explicit, which does not substitute the image it qualifies; a build argument redeclared to bring it into a stage's scope, which does not introduce a new argument. Each carries the identifier of the rule that must not fire, inverting the usual reading of the specification, and it is that inversion which makes the absence of false alarms a measured property rather than an assumption.

#### 6.4.1.4 Independence of the adjudication

Two properties of the reference specification bear on the validity of these figures. First, mapping.md is a specification rather than a log: it was authored alongside the corpus commits and states what the tool must do, not what it was observed to do. Second, it is external to the tool. The detection engine has no access to it at any point in its execution, and detection is decided from the structure of the two Dockerfile states alone, in accordance with the requirement that the engine remain autonomous of the catalogue. The specification is consulted only after detection has completed and its results have been written to disk.

The separation between execution and adjudication ensures that the tool cannot influence the interpretation of its own results. Furthermore, potential defects in the automated comparison were guarded against through a rigorous manual review of every pair, confirming that the outcomes reflect true conformity rather than an artifact of the audit script.

### 6.4.2 Performance Impact (ΔSize)

The measurement of the performance dimension returned, over the isolated scenario, a before size of 28,234,415 bytes, an after size of 3,421,731 bytes and a delta of −24,812,684 bytes. The discussion below examines what that figure establishes, and what the provenance of the values contributes to it.

#### 6.4.2.1 Magnitude of the Effect

The isolated scenario evaluates a single-stage transition from a heavier base image (debian:12-slim) to an ultra-light one (alpine:3.19). The measurement returned a before size of 28,234,415 bytes, an after size of 3,421,731 bytes, and a delta of −24,812,684 bytes. This unattenuated drop reflects the direct impact of the base-image substitution without interference from unrelated instructions.

#### 6.4.2.2 Validation under a known direction

The scenario was designed so that the expected direction of the effect was known in advance: the before state declares Debian (a comparatively heavier image), and the after state declares Alpine (an ultra-light image). That the prototype reported a pronounced decrease over such a pair confirms that it successfully captures the expected direction of the effect required by the controlled construction of the scenario.

#### 6.4.2.3 Provenance of the reported values

The figures are raw. No conversion from megabytes to bytes was performed at any point between retrieval and reporting, and the values are unrounded.

Rather than relying on textual command-line parsing (such as querying docker images via terminal), these values originate directly from the official Docker SDK. The data are fidedignos because they stem natively from the SDK's internal attributes (e.g., retrieving image.attrs["Size"]), preserving full byte-level precision.

```
def _image_size(client: docker.DockerClient, image_reference: str, label: str) -> int:
    """Read the exact uncompressed byte size of one built image."""
    image = client.images.get(image_reference)
    size = image.attrs["Size"]
    if not isinstance(size, int):
        raise PerformanceAnalyzerError(
            f"expected an integer byte count, got {type(size).__name__}."
        )
    return size
```

Listing 6.14 - Abbreviated excerpt from src/performance_analyzer.py (lines 95–135); exception handling blocks omitted.

### 6.4.3 Security Impact (ΔCVEs)

The measurement of the security dimension returned a transition from 56 to 2 CRITICAL vulnerabilities, a delta of −54, and from 1,894 to 21 HIGH vulnerabilities, a delta of −1,873. The discussion below examines what the figures establish about the integration of the external scanner, the counting policy applied to them, and the traceability of the conditions under which they were obtained.

#### 6.4.3.1 Fidelity of extraction

The inspection of the artefacts was directed at a question prior to the counts themselves: whether the scanner transfers its output to the prototype intact, and whether the prototype reads that output without loss.

The raw consolidated report was examined for both states. In the before state, corresponding to the Debian-based image, specific CVE identifiers of CRITICAL and HIGH level were located. In the after state, corresponding to the Alpine variant, the risk vectors identified in the preceding state were not present in the inventory. Those identifiers are a small sample of the massive inventory of packages and vulnerabilities recorded in the raw file for the before state, which was a general-purpose operating system carrying a wide attack surface.

Two families of package account for much of what disappears. The transition from Debian to Alpine removes the C and C++ compilation ecosystem and its corresponding development libraries, the *-dev packages, whose absence from the minimalist distribution prevents the accumulation of the thousands of CVEs associated with those support packages; gcc-12@12.2.0-14 and g++-12 cease to form part of the base image. System and multimedia utilities pre-installed in the Debian image, such as imagemagick@8:6.9.11.60+dfsg-1.6 and curl@7.88.1-10, together with libraries such as krb5-multidev, likewise cease to appear in the inventory audited by the scanner.

Cross-referencing the raw data against the structured report validated the exactness of the counts: the CRITICAL counters transitioned rigorously from 56 to 2 and the HIGH counters from 1,894 to 21, confirming that the subtraction was executed by the engine without deviation.

The evidence supporting this point is that the scanner succeeded in passing its information to the prototype and that the prototype succeeded in reading that data, which demonstrates that the prototype consumes the raw scanner document without data loss and without parsing errors, and that Trivy is correctly integrated into the tool.

```
def _scan_image(image_reference: str, label: str) -> bytes:
    return _run(
        [
            "docker", "run", "--rm",
            "-v", _DOCKER_SOCKET,
            "-v", _trivy_cache_mount(label),
            TRIVY_IMAGE, "image", "--quiet", "--format", "json",
            image_reference,
        ],
        f"Trivy scan of the '{label}' image",
    )

# Direct retention of the scanner's raw bytes (RNF5):
raw_trivy={
    "before": outputs["scan_before"].decode("utf-8", errors="replace"),
    "after": outputs["scan_after"].decode("utf-8", errors="replace"),
}
```

Listing 6.15 - Invocation of the Trivy scanner over the built image and raw byte retention (_scan_image and raw_trivy

#### 6.4.3.2 Magnitude of the reduction

The drastic difference in the number of vulnerabilities follows from the transition between two classes of base image. The before state is built on Debian, a complete and robust operating system carrying thousands of pre-installed system packages, network utilities, glibc-ecosystem libraries and support tooling, each of which the scanner analyses, finding close to nine thousand vulnerabilities accumulated across the global ecosystem of the operating system. The after state is built on Alpine Linux, designed from the outset for containers, ultra-minimalist, using musl libc rather than glibc, and stripped of everything superfluous, including heavyweight utilities and package managers such as apt.

The reduction therefore expresses the passage from a general-purpose base operating system to an ultra-minimalist distribution, which massively reduces the attack surface exposed to the scanner.

#### 6.4.3.3 Deduplication of identifiers

The counter in the structured report applies a filter to identical CVE identifiers: where the same identifier is seen twice, it contributes a single unit. The raw file retains the complete

data for both states exactly as the scanner emits them. This counting policy is what RF5 governs, the requirement of the Data Extractor that prescribes the count of distinct identifiers.

The rationale follows from the granularity at which the scanner reports. The scanner does name the package in which each finding occurs, so where two packages of the Dockerfile carry the same vulnerability it raises an alert for each of them. What it does not provide is the line at which each package is declared, and without that the two alerts cannot be distinguished from one another: they present as two identical CVEs. Counting both would therefore add an entry without adding information, since the identifier designates the origin of the problem and the consequences it produces rather than a location within the file.

Counting the identifier once matches the unit at which the finding is acted upon. When a CVE is raised, the identifier is examined, the problem it reports is reviewed, the location of the fault is traced, and one or two lines of code, or two or three packages, are corrected. Recording three packages carrying three identical CVEs in place of a single entry produces confusion rather than information.

#### 6.4.3.4 Traceability of the execution conditions

The provenance section of the structured report was examined, and the prototype was confirmed to register automatically the exact version of the scanner in use, 0.72.0, the timestamp corresponding to the vulnerability database employed at the moment of the analysis, and the environment of the tool that participated in the run.

Two consequences follow, and together they constitute the application of RNF5, the reproducibility requirement dedicated to these records, since the tool registers automatically in the impact report both the timestamp and the exact version of the scanner database used during that specific analysis.

Within a single analysis, the database version is the same for both commits, held in separate caches, which permits the CVE delta to be correlated directly with the refactoring rather than with a change in the scanner's knowledge of the packages. Across different days, the tool preserves full traceability through the recorded provenance, which permits any difference in counts to be attributed either to a change in the code or to an update of the scanner's external database.


```
{
 "environment": {
  "docker_engine": "29.1.3",
  "hadolint": "Haskell Dockerfile Linter 2.14.0",
  "trivy": "0.72.0",
  "trivy_vulnerability_db": "2026-09-02T20:01:19.708956607Z",
  "dockerfile_parse": "2.0.1",
  "gitpython": "3.1.50",
```

```
    "docker_sdk": "7.1.0",
    "python": "3.14.4"
  }
}
```

Listing 6.16 - Environment provenance block within the structured report (impact_report.json

#### 6.4.3.5 Severity stratification and ordering

The severity block of the structured report was analysed to confirm how the prototype stratifies results across CRITICAL, HIGH, MEDIUM, LOW and UNKNOWN, validating the coherence of the mathematical deltas.

The mathematical coherence of the deltas is validated by the sum of the individual severity tiers extracted from the raw scan data and processed into the structured report: the before-state tiers sum exactly to the reported total ($56 + 1,894 + 5,991 + 954 + 45 = 8,940$), and the after-state tiers sum correspondingly ($2 + 21 + 15 + 11 + 0 = 49$), confirming that the engine processes all categories without omission.

The engine organizes the vulnerability output in descending order of severity, following the standard classification hierarchy defined in the static configuration, SEVERITY_TIERS.

```
# Static hierarchy declared from most to least severe:
SEVERITY_TIERS: Sequence[str] = ("CRITICAL", "HIGH", "MEDIUM", "LOW", "UNKNOWN")


# Processing and aggregation indexed by severity tier:
counts = {tier: 0 for tier in SEVERITY_TIERS}
for severity in severity_of.values():
    if severity in counts:
        counts[severity] += 1
```

Listing 6.17 - Static declaration of the severity hierarchy (SEVERITY_TIERS) and categorical vulnerability aggregation logic

### 6.4.4 Maintainability Impact (ΔWarnings and ΔInstr)

The maintainability dimension returned a filtered warning count of 2 before and 1 after, a delta of −1, and a logical instruction count of 3 before and 2 after, a delta of −1. The two metrics are discussed separately, since each validates a distinct mechanism of the prototype:

the first the integration of an external linter and the screen applied to its output, the second an internal counting module.

#### 6.4.4.1 Composition of the warning delta

The delta of −1 requires explanation, since two refactorings were applied and multiple warnings were emitted. During the analysis of the *before* state, the raw linter output from Hadolint emitted five distinct rule violation codes (DL3007, DL3008, DL3009, DL3015, and DL3059). However, when processed through the declarative maintainability screen comprising the 47 rules derived from Appendix C, only two of these violations (DL3007 and DL3008) met the screening criteria, establishing an initial filtered count of 2. In the *after* state, three codes were emitted (DL3008, DL3009, and DL3015), of which only DL3008 passed the screen, resulting in a final count of 1 and a delta of −1.

Only one of the two refactorings eliminated a warning belonging to the maintainability screen, namely the substitution of the base image tag (ubuntu:latest for ubuntu:22.04), which removed DL3007. DL3008 remained unaltered in the code, so the filtered count fell from 2 to 1.

The persistence of DL3008 is itself explicable from the code rather than from the tool. The warning states that versions should be pinned in apt-get install, recommending apt-get install <package>=<version> in place of apt-get install <package>. Since the command in the Dockerfile remained apt-get install -y curl, with no specific version fixed for curl, the linter continued to raise the same infraction in the *after* state.

#### 6.4.4.2 Correctness of the 47-rule screen

The behaviour of DL3059 under the screen provides direct evidence that the filter is correctly applied.

DL3059, the multiple-layers warning, left the raw after output as a consequence of the Inline RUN Instructions refactoring, which consolidated the two separate RUN instructions of the before state into a single chained instruction. That elimination is visible in the raw artefact. It was nonetheless not counted in the structured report, because DL3059 does not belong to the screen, which shows that the filter works: a warning genuinely eliminated by a refactoring does not move the filtered metric when the rule producing it lies outside the screened set.

The screen isolates only the warnings relevant to maintainability for the computation of ΔWarnings, and this case demonstrates the isolation operating in the direction that would otherwise go unnoticed.

```
MAINTAINABILITY_SCREEN: Sequence[str] = (
  # Reproducibility (16)
  "DL3005", "DL3006", "DL3007", "DL3008", "DL3013", "DL3016", "DL3017",
  "DL3018", "DL3028", "DL3031", "DL3033", "DL3035", "DL3037", "DL3039",
```

```
    "DL3041", "DL3062",
    # Structural correctness (15)
    "DL3000", "DL3003", "DL3011", "DL3012", "DL3021", "DL3022", "DL3023",
    "DL3024", "DL3043", "DL3044", "DL3045", "DL3061", "DL3063", "DL4003",
    "DL4004",
    # Usage and notation (12)
    "DL3001", "DL3010", "DL3014", "DL3025", "DL3027", "DL3029", "DL3030",
    "DL3034", "DL3038", "DL4001", "DL4005", "DL4006",
    # Metadata (1)
    "DL4000",
    # Shell correctness, ShellCheck (2)
    "SC2046", "SC2086",
    # Build logs (1)
    "DL3047",
)

_SCREEN = frozenset(MAINTAINABILITY_SCREEN)

def _count_screened_warnings(findings) -> int:
    if not isinstance(findings, list):
        raise DataExtractorError(f"Unexpected shape: {type(findings).__name__}")
    return sum(1 for finding in findings if _rule_code(finding) in _SCREEN)
```

Listing 6.18- Declarative 47-rule maintainability screen definition and screening logic

#### 6.4.4.3 Fidelity of the linter integration

The validation of the integration rests on direct numerical correspondence rather than abstract assertion. Comparing the raw data of the before and after states against the structured report (impact_report.json) confirms that the reported figures are fully synchronized with what the linter emitted. Specifically, the five raw codes emitted in the before state (DL3007, DL3008, DL3009, DL3015, and DL3059) are filtered down to 2 valid maintainability warnings, and the three raw codes in the after state yield 1 filtered warning, matching the structured metrics exactly. The mathematical subtraction recorded in the report (−1) directly reflects this filtered transition, demonstrating sound integration of Hadolint with the prototype with zero data loss.

```
def _lint_dockerfile(dockerfile_content: str, label: str) -> bytes:
    payload = dockerfile_content.replace("\r\n", "\n").encode("utf-8")
    return _run(
        [
            "docker", "run", "--rm", "-i",
            HADOLINT_IMAGE, "hadolint", "--format", "json", "-",
        ],
        f"Hadolint analysis of the '{label}' Dockerfile",
        stdin=payload,
    )
```

Listing 6.19 - Invocation of Hadolint via stdin streaming and raw output capture

```
{
  "warnings": {
    "before": 2,
    "after": 1,
    "delta_warnings": -1
  }
}
```

Listing 6.20 - Structured report block (impact_report.json) recording warning counts and delta

#### 6.4.4.4 Treatment of duplicate warnings

The prototype deliberately applies no deduplication of warnings sharing an identifier, which is the opposite of the policy applied to the CVEs of the security dimension.

The reason is that Hadolint is positional: it states exactly where the error occurs, generating distinct records for different lines (such as a warning identified at line 30 and another at line 40). Eliminating duplicate alerts would conceal this spatial granularity. Retaining the duplication in the raw output guarantees that an auditor can see every exact occurrence of each infraction without losing the spatial relation to the code. The policy applies at both levels of the maintainability artefacts. The raw artefacts hold the information as the tool emits it, and the report presenting the values and the delta in impact_report.json is produced with no deduplication applied at any point. In the security dimension the raw artefacts likewise hold the scanner output as emitted, but the corresponding report is produced with deduplication applied, for the reasons given in that subsection.

#### 6.4.4.5 The logical instruction module

The second metric validates an internal counting module operating over the syntax trees provided by the third-party dockerfile-parse library, rather than an external linter integration, and the validation was performed by manual cross-checking.

The expected number of logical instructions for the before and after states was counted manually. The values produced by the prototype were then read from the structured report (impact_report.json), which recorded a before count of 3, an after count of 2, and a delta of −1, matching the manual count exactly.

This confirms that the internal counting module successfully measures structural complexity for the evaluated test scenario, aligning the prototype's automated output with the manual ground truth.

```
{
  "logical_instructions": {
    "before": 3,
    "after": 2,
    "delta_logical_instructions": -1
  }
}
```

Listing 6.21 - Structured report block (impact_report.json) recording logical instruction counts and delta

#### 6.4.4.6 Availability of the artefacts

The information validating both the logical instruction counter and the use of the external Hadolint tool for counting warnings is available in the author's GitHub repository, under experiment and validation/measurement impact/manutenibility/

## 6.5 Threats to Validity

### 6.5.1 Detection

**Structural limitations:** The evaluation corpus is purpose-built, restricting automated inspection to a single Dockerfile per execution across three conventional services, given that refactorings such as R11 (Move Stage) operate across multiple Dockerfiles rather than a single file. It validates the catalogue across its spanned syntactic range but does not measure real-world frequency or unexercised constructs such as ONBUILD.

**Risk and bias factors:** The specification, the corpus, and the prototype were authored by the same researcher, introducing potential author bias.

**Methodological mitigations:** Author bias is contained by a specification authored independently of prototype observation, blind engine execution without access to the specification, and an automated audit program (`audit_corpus.py`) distinct from the tool producing the results.

### 6.5.2 Performance

**Structural limitations:** Image sizes are not strictly byte-reproducible in this environment, with repeated builds fluctuating by approximately two bytes in absolute terms and three bytes in delta.

**Risk and bias factors:** Controlled scenarios were designed with the expected direction of the effect known in advance of measurement.

**Methodological mitigations:** Reproducibility is supported by publishing the test scenario script (`validate_r10_isolated.py`) in the author's GitHub repository. Data integrity is maintained by retrieving image sizes as raw integers directly from the Docker SDK without unit conversions or rounding.

### 6.5.3 Security

**Structural limitations:** CVE deduplication in the structured report is an architectural design choice governed by RF5, counting distinct vulnerability identifiers rather than raw package occurrences, since the scanner names the affected package but not the line at which it is declared, so two alerts sharing an identifier cannot be distinguished from one another.

**Risk and bias factors:** The external vulnerability database evolves independently, meaning counts can change due to external updates rather than code modifications.

**Methodological mitigations:** Provenance tracking (`RNF5`) automatically records the exact scanner version (0.72.0) and database timestamp, raw scan files are retained untouched, and arithmetic consistency is verified by summing severity tiers.

### 6.5.4 Maintainability

**Structural limitations:** Metrics are bounded by a declarative screen of 47 rules, meaning warnings genuinely eliminated by a refactoring (such as DL3059 after RUN instructions are consolidated) do not move the metric if the rule lies outside the screen.

**Risk and bias factors:** Manual baseline counts were performed by the primary researcher.

**Methodological mitigations:** Warning deduplication is intentionally omitted as an architectural design choice to preserve positional line-level granularity (retaining Hadolint's exact line numbers), and validation data supporting the metrics is made publicly available in the author's GitHub repository under `experiment and validation/manutenibility/`.

# 7 Conclusion

This dissertation set out from a gap between two bodies of knowledge that had developed independently of one another. Refactoring theory, established for application source code since Fowler, offers a disciplined vocabulary of behaviour-preserving transformations, yet its application to Dockerfiles remained largely unexplored. Existing catalogues of Dockerfile refactorings enumerated transformation types without classifying them by the quality dimensions they affect, and the tools available for their detection identified structural change without quantifying its downstream consequences. A developer consolidating instructions or substituting a base image therefore acted on expectation rather than on evidence, since neither the literature nor the available instruments established what such a transformation does to image size, to vulnerability exposure, or to the maintainability of the file.

The problem this addresses is not merely one of missing tooling. Technical debt in Dockerfiles survives for extended periods, quality defects persist across revisions, and the transformations that would remove them carry effects across several dimensions at once, some of which oppose each other. Without a means of measuring those effects, the trade-offs a refactoring introduces remain invisible at the moment the decision is taken.

## 7.1 Fulfilment of the research objectives

Three research questions guided the work, and each was addressed through a distinct artefact and a distinct form of evidence.

**RQ1** concerned the state of the art in the detection, classification and impact assessment of Dockerfile refactorings. A systematic literature review following the PRISMA protocol established that the existing body of work divides between studies characterising the occurrence and persistence of Dockerfile smells and tools performing detection or automated repair, and that no existing instrument links a detected refactoring to a measured effect across quality dimensions. That finding defined the gap the remaining two questions address.

**RQ2** concerned the extension of a refactoring catalogue by quality dimension. The extended catalogue classifies fourteen refactoring types across performance, security and maintainability, evaluating each type independently on all three axes rather than subordinating one to another. The classification rests on evidence drawn from the literature and from controlled experiments, and records regressions alongside improvements, so that a transformation whose benefit on one axis carries a cost on another is represented as such rather than reduced to a single verdict.

**RQ3** concerned the effectiveness of a prototype in detecting refactorings automatically and assessing their impact. Effectiveness was defined in the introduction through two complementary dimensions, detection accuracy and impact measurement reliability, and both were evaluated empirically.

The prototype was implemented as five components executing across four sequential stages, built on the technologies validated during the rapid experimentation phase. It receives a repository and two commit identifiers, retrieves the corresponding Dockerfile states at blob level, determines which catalogue refactorings separate them, builds the corresponding pair of images within historical commit contexts, measures four quality metrics over them, and emits a structured report pairing the detected identifiers with the measured deltas. The separation between detection and measurement established at design time is preserved in the code as built: no measurement component receives the detection result, so the deltas are obtained without any expectation of the outcome encoded in the software.

## 7.2 Principal findings

Detection accuracy. Evaluated over a purpose-built corpus of forty-nine commit pairs distributed across three independent services, the detection engine recorded a strict conformance rate of 100% (49/49), with zero false negatives, zero false positives and zero unlisted detections. The corpus was designed so that sensitivity and specificity are measured independently rather than conflated: canonical cases establish that each rule fires on the operation it is defined to detect; false-negative cases express the same refactoring through syntactic variants a naive implementation would miss; mixed cases verify that concurrent refactorings are all reported; and false-positive cases, carrying the identifier of the rule that must not fire, make the absence of false alarms a measured property rather than an assumption. Those fourteen control pairs produced an empty result set in every case, which is a stronger outcome than the specification demands.

Performance. Measured over an isolated single-stage scenario substituting debian:12-slim for alpine:3.19, the prototype reported a before size of 28,234,415 bytes, an after size of 3,421,731 bytes and a delta of −24,812,684 bytes. The values are retrieved as raw integers through the Docker SDK, with no unit conversion and no rounding applied between retrieval and reporting, which is what gives the measurement the byte-level precision that the Docker command-line interface, applying decimal rounding by design, could not provide.

Security. Measured over a commit pair applying Update Base Image TAG, the prototype reported a transition from 56 to 2 CRITICAL vulnerabilities and from 1,894 to 21 HIGH vulnerabilities, deltas of −54 and −1,873 respectively. The counting policy governed by RF5 records distinct vulnerability identifiers rather than raw package occurrences, and the arithmetic coherence of the stratification is verifiable directly: the before-state tiers sum exactly to the reported total, 56 + 1,894 + 5,991 + 954 + 45 = 8,940, and the after-state tiers sum correspondingly, 2 + 21 + 15 + 11 + 0 = 49. Under RNF5 the prototype records automatically the exact scanner version, 0.72.0, together with the timestamp of the vulnerability database employed, so that any difference in counts between analyses can be attributed either to a change in the code or to an update of the external database.

Maintainability. The dimension is addressed by two metrics, and the evaluation of the first demonstrated the correctness of the screen rather than merely its output. The raw linter emitted five distinct rule violations in the before state, of which the declarative screen of forty-seven rules derived from Appendix C retained two, DL3007 and DL3008. The filtered count fell from 2 to 1, the delta corresponding specifically to the elimination of DL3007 by the tag substitution, while DL3008 persisted because the command retained an unpinned package version. The behaviour of DL3059 is the more instructive case: genuinely eliminated by the

consolidation of the RUN instructions, it did not move the filtered metric because the rule lies outside the screen, which demonstrates the isolation operating in the direction that would otherwise pass unnoticed. The second metric, the logical instruction count, returned a before count of 3, an after count of 2 and a delta of −1, matching an independently performed manual count exactly.

Taken together, the results establish an asymmetry that is itself a finding of the work rather than a weakness of it. Detection is deterministic: it operates on the two Dockerfile states alone, depends on no external component, and is exactly reproducible. Measurement is conditional: it depends on external analysers and on a build environment the prototype does not control, and its results hold under an environment that must be recorded alongside them. Establishing that distinction is what motivates the provenance record and the retention of unmodified analyser output, and it governs how every reported value is to be read.

## 7.3 Limitations

Three limitations bound the interpretation of these results and are stated here as structural properties rather than as incidental shortcomings.

The evaluation corpus is purpose-built, and the specification against which the engine is judged was authored by the same researcher who developed the prototype. The design contains this bias without eliminating it, since the specification states the expected behaviour independently of any observation of the tool, the engine has no access to it during execution, and the audit is performed by a program distinct from the one producing the results. The figures nonetheless establish faithful implementation of the catalogue across the syntactic range the corpus spans, not the frequency with which those refactorings occur in practice.

Image size is not strictly byte-reproducible in the execution environment, with repeated builds of the same pair fluctuating by approximately two bytes in absolute terms and three bytes in delta. Deltas whose magnitude falls within that band cannot be distinguished from environmental variation, which bounds from below the effects the performance metric can resolve.

The analysis is confined to a single Dockerfile per execution. This is the limitation with the widest consequence, and it is the point from which the future work below proceeds.

## 7.4 Future work

### 7.4.1 Multi-Dockerfile support

The single-file scope defined by RF1 binds the prototype to one artefact supplied by the caller, and no component performs directory traversal or filename matching, so a second Dockerfile

is unreachable rather than merely unused. The constraint was adopted deliberately, since automatic aggregation of satellite files would require heuristics fragile enough to compromise the determinism of the measurement pipeline, and most of the catalogue operates over the internal structure of one file.

Its cost is nonetheless concrete and identifiable. Move Stage (R11) extracts a build stage into a separate Dockerfile built independently, so the transformation is by definition distributed across two artefacts. Under the current scope the prototype reasons from what remains in the primary file rather than from what the extraction produced, and the instruction count reflects the three instructions that left the file without seeing the FROM that the extracted file gains. The refactoring is detected correctly, but its full effect on the project is not measurable from the single artefact the prototype reads.

Extending the architecture to repositories holding multiple Dockerfiles would provide native coverage of inter-file refactoring operations, and would allow the project-level accounting that R11 requires to be performed by the tool rather than reconstructed by the reader. The extension bears principally on the retrieval and measurement stages, since the detection rules already operate over instruction lists and would receive several such lists rather than one.

### 7.4.2 Input mapping mechanisms

Multi-file support introduces an ambiguity the single-file scope does not have: determining which artefacts belong to the same build, and which file in the after state corresponds to which in the before state. A stage extracted into a new Dockerfile has no predecessor to be compared against, a Dockerfile relocated within the repository has a predecessor under a different path, and a repository holding several unrelated services has files that must not be compared at all.

Explicit input mapping mechanisms would resolve these structural and path ambiguities by making the correspondence between artefacts an input to the analysis rather than something inferred from it. Supplying that mapping explicitly preserves the property that motivated the original constraint, since the pipeline would continue to operate on a declared set of artefacts rather than on a heuristic guess about which files belong together, and the determinism of the measurement would be retained while the scope of what can be measured widens.

# Reference


[1] M. Fowler, Refactoring: Improving the Design of Existing Code, 2nd ed., Addison-Wesley Professional, Boston, 2018. https://martinfowler.com/books/refactoring.html.

[2] E. Ksontini, M. Kessentini, T. do N. Ferreira, F. Hassan, Refactorings and Technical Debt in Docker Projects: An Empirical Study, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2021: pp. 781–791. https://doi.org/10.1109/ASE51524.2021.9678585.

[3] Stack Overflow Developer Survey 2025, Stack Overflow (2025). https://survey.stackoverflow.co/2025/technology#most-popular-technologies-platform-platform (accessed January 8, 2026).

[4] Cloud Native 2024: Approaching a Decade of Code, Cloud, and Change, CNCF (2025). https://www.cncf.io/reports/cncf-annual-survey-2024/ (accessed January 10, 2026).

[5] J. Cito, G. Schermann, J.E. Wittern, P. Leitner, S. Zumberi, H.C. Gall, An Empirical Analysis of the Docker Container Ecosystem on GitHub, in: 2017 IEEE/ACM 14th International

Conference on Mining Software Repositories (MSR), 2017: pp. 323–333. https://doi.org/10.1109/MSR.2017.67.
[6] G. Bavota, B. Russo, A large-scale empirical study on self-admitted technical debt, in: Proceedings of the 13th International Conference on Mining Software Repositories, Association for Computing Machinery, New York, NY, USA, 2016: pp. 315–326. https://doi.org/10.1145/2901739.2901742.
[7] Docker, Best practices, Docker Documentation (2026). https://docs.docker.com/build/building/best-practices/ (accessed January 20, 2026).
[8] Y. Wu, Y. Zhang, T. Wang, H. Wang, Characterizing the Occurrence of Dockerfile Smells in Open-Source Software: An Empirical Study, IEEE Access 8 (2020) 34127–34139. https://doi.org/10.1109/ACCESS.2020.2973750.
[9] E. Ksontini, A. Abid, R. Khalsi, M. Kessentini, DRMiner: A Tool For Identifying And Analyzing Refactorings In Dockerfile, in: 2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR), 2024: pp. 584–594.
[10] Hadolint, Hadolint: Haskell Dockerfile Linter, (2026). https://github.com/hadolint/hadolint.
[11] J. vom Brocke, A. Hevner, A. Maedche, Introduction to Design Science Research, in: 2020: pp. 1–13. https://doi.org/10.1007/978-3-030-46781-4_1.
[12] Código de Conduta do IPP, (2018). https://www.ipp.pt/sobre/transparencia-integridade-anticorrupcao/codigo-de-conduta (accessed April 14, 2026).
[13] IEEE Code of Ethics | IEEE, (2020). https://www.ieee.org/about/corporate/governance/p7-8 (accessed April 14, 2026).
[14] I. Kumara, M. Garriga, A.U. Romeu, D. Di Nucci, F. Palomba, D.A. Tamburri, W.-J. van den Heuvel, The do's and don'ts of infrastructure code: A systematic gray literature review, Information and Software Technology 137 (2021) 106593. https://doi.org/10.1016/j.infsof.2021.106593.
[15] C. Boettiger, An introduction to Docker for reproducible research, SIGOPS Oper. Syst. Rev. 49 (2015) 71–79. https://doi.org/10.1145/2723872.2723882.
[16] D. Nüst, V. Sochat, B. Marwick, S.J. Eglen, T. Head, T. Hirst, B.D. Evans, Ten simple rules for writing Dockerfiles for reproducible data science, PLOS Computational Biology 16 (2020) e1008316. https://doi.org/10.1371/journal.pcbi.1008316.
[17] G. Rosa, F. Zappone, S. Scalabrino, R. Oliveto, Fixing Dockerfile smells: an empirical study, Empir Software Eng 29 (2024) 108. https://doi.org/10.1007/s10664-024-10471-7.
[18] CVE: Common Vulnerabilities and Exposures, (2026). https://www.cve.org/ (accessed January 28, 2026).
[19] Jean Rémy Falleri, Florèal Morandat, Xavier Blanc, Matias Martinez, Martin Monperrus, Fine-grained and accurate source code differencing | Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering, (2014). https://dl.acm.org/doi/10.1145/2642937.2642982.
[20] M. Vidoni, A systematic process for Mining Software Repositories: Results from a systematic literature review, Information and Software Technology 144 (2022) 106791. https://doi.org/10.1016/j.infsof.2021.106791.
[21] Multi-stage, Docker Documentation (2026). https://docs.docker.com/build/building/multi-stage/ (accessed February 15, 2026).
[22] T. Durieux, Empirical Study of the Docker Smells Impact on the Image Size, (2024). https://doi.org/10.1145/3597503.3639143.
[23] J. Henkel, C. Bird, S.K. Lahiri, T. Reps, Learning from, Understanding, and Supporting DevOps Artifacts for Docker, in: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, 2020: pp. 38–49. https://doi.org/10.1145/3377811.3380406.

[24] Software Size Measurement: A Framework for Counting Source Statements | CMU Software Engineering Institute, (1992). https://www.sei.cmu.edu/library/software-size-measurement-a-framework-for-counting-source-statements/ (accessed July 16, 2026).
[25] ISO/IEC 25010:2011(en), Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models, (n.d.). https://www.iso.org/obp/ui/#iso:std:iso-iec:25010:ed-1:v1:en (accessed July 17, 2026).
[26] R.C. Martin, Agile software development: principles, patterns, and practices, Prentice Hall/Pearson Education, Upper Saddle River, NJ, 2003.
[27] A. Zerouali, T. Mens, G. Robles, J.M. Gonzalez-Barahona, On the Relation between Outdated Docker Containers, Severity Vulnerabilities, and Bugs, in: 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER), 2019: pp. 491–501. https://doi.org/10.1109/SANER.2019.8668013.
[28] Q.-C. Bui, M. Laukötter, R. Scandariato, DockerCleaner: Automatic Repair of Security Smells in Dockerfiles, in: 2023 IEEE International Conference on Software Maintenance and Evolution (ICSME), 2023: pp. 160–170. https://doi.org/10.1109/ICSME58846.2023.00026.
[29] A. Rahman, L. Williams, Different Kind of Smells: Security Smells in Infrastructure as Code Scripts, IEEE Security & Privacy 19 (2021) 33–41. https://doi.org/10.1109/MSEC.2021.3065190.
[30] Z. Zhu, T. Chen, C. Liu, H. Liu, Q. Song, Z. Xu, Y. Liu, Doctor: Optimizing Container Rebuild Efficiency by Instruction Re-Orchestration (preprint, arXiv:2504.01742), (2025). https://doi.org/10.48550/arXiv.2504.01742.
[31] M.U. Haque, M.A. Babar, Well Begun is Half Done: An Empirical Study of Exploitability & Impact of Base-Image Vulnerabilities, (2021). https://doi.org/10.48550/arXiv.2112.12597.
[32] E. Ksontini, M. Mastouri, R. Khalsi, W. Kessentini, Refactoring for Dockerfile Quality: A Dive into Developer Practices and Automation Potential (preprint, arXiv:2501.14131), (2025). https://doi.org/10.48550/arXiv.2501.14131.
[33] D. Spadini, M. Aniche, A. Bacchelli, PyDriller: Python framework for mining software repositories | Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ACM Conferences (2018). https://dl.acm.org/doi/10.1145/3236024.3264598.
[34] Docker Engine API, Docker Documentation (2026). https://docs.docker.com/reference/api/engine/.
[35] moby, The Size reported using `docker inspect` and `docker images` does not match · Issue #31298 · moby/moby, GitHub (2017). https://github.com/moby/moby/issues/31298 (accessed February 12, 2026).
[36] aquasecurity/trivy, (2026). https://github.com/aquasecurity/trivy (accessed July 11, 2026).
[37] L. Bass, P. Clements, R. Kazman, Software architecture in practice, Fourth edition, Addison-Wesley, Boston, 2022.
[38] T. Bray, The JavaScript Object Notation (JSON) Data Interchange Format, RFC 8259, Internet Engineering Task Force, 2017. https://doi.org/10.17487/RFC8259.
[39] A.S. Horowitz, Covering project skill gaps | PMI, (2001). https://www.pmi.org/learning/library/covering-project-skill-gaps-5007 (accessed February 3, 2026).

[40] M.E. Berger, Contribution of individual project participant competencies to project success, (2010). https://www.pmi.org/learning/library/individual-project-competencies-project-success-6457 (accessed January 15, 2026).

# Appendix A — Skills Management

This section addresses skill management, which is essential for the successful completion of this project. The development and application of relevant skills are crucial for achieving the objectives related to the detection and evaluation of refactorings in Infrastructure as Code. Skill management encompasses technical, analytical, and interpersonal competencies required for studying software repositories, developing analysis tools, and empirically evaluating quality metrics.

## Required Skills and Current Proficiency

The successful delivery of this project requires competencies across three categories, namely technical skills, analytical skills, and interpersonal skills.

Technical skills encompass knowledge of containerization technologies, specifically Docker internals and best practices, static analysis techniques including Abstract Syntax Tree (AST) parsing and regular expressions, Mining Software Repositories (MSR) methodologies, and programming proficiency in languages such as Python or Java for tool prototyping.

Analytical skills involve the ability to evaluate complex datasets, perform statistical analysis to assess quality impacts such as image size, and interpret security vulnerabilities within Dockerfiles.

Interpersonal skills include project management, time management, and effective collaboration and communication with the supervisor throughout the research phases.

Table A.1 presents the assessment of current proficiency levels against the required proficiency for each skill. The gap column identifies critical areas requiring development. Skills rated with a High gap demand immediate attention, while those with Medium gap require progressive improvement throughout the project lifecycle.

| Skill | Required Proficiency (1-5) | Current Proficiency (1-5) | Gap | Comments |
|---|---|---|---|---|
| **Docker Internals & IaC** | 5 | 4 | Low | Deep understanding of build caching, layer filesystem, and instruction ordering is required to correctly classify refactorings. |
| **Static Analysis & Parsing** | 4 | 0 | High | Crucial for developing the detection tool; requires learning how to construct and traverse Abstract Syntax Trees (AST). |
| **Algorithm Design** | 4 | 1 | High | Necessary to design efficient logic that compares Dockerfile versions without generating false positives/negatives. |
| **Mining Software Repositories** | 4 | 2 | Medium | Ability to automate the extraction of large datasets (commit history, file diffs) from platforms like GitHub. |
| **Tool Development (Python/Java)** | 4 | 2 | Medium | Proficiency in a high-level language to implement the prototype and handle file I/O. |
| **Statistical Analysis** | 3 | 1 | Medium | Essential for interpreting empirical results (image sizes) to determine statistical significance. |

| Skill | Required Proficiency (1-5) | Current Proficiency (1-5) | Gap | Comments |
|---|---|---|---|---|
| **Academic Writing** | 4 | 3 | Low | Ability to synthesize complex literature and report technical findings clearly in the dissertation. |
| **Time Management** | 4 | 3 | Low | Experience in managing smaller projects; needs to build on strategies for larger, collaborative projects (like this thesis timeline). |
| **Communication** | 4 | 2 | Medium | Effective in written communication but seeking improvement in presenting complex technical information to non-specialists |
| **Critical Thinking** | 4 | 2 | Medium | Contributes to analytical tasks in initial exposure to professional settings; needs improvement in critical assessment of situations (data interpretation). |
| **Problem-Solving** | 4 | 3 | Low | Needs enhancement in the ability to identify, analyze, and solve problems (e.g., debugging the |

| Skill | Required Proficiency (1-5) | Current Proficiency (1-5) | Gap | Comments |
|---|---|---|---|---|
| | | | | prototype) in dynamic environments. |
| **Continuous Learning** | 4 | 3 | Low | Demonstrates the rudiments of how important it is to adapt and learn new skills (e.g., new parsing libraries). |
| **Decision-Making** | 4 | 3 | Low | Requires improvement in assessing risks and benefits to make informed and timely decisions (e.g., methodological choices). |

Table A.1 - Assessment of current skills

As indicated in Table A.1, two skills present a High gap requiring significant development. Static Analysis & Parsing is crucial for developing the detection tool and requires learning how to construct and traverse Abstract Syntax Trees. Algorithm Design is necessary to design efficient logic that compares Dockerfile versions without generating false positives or negatives. Skills with Medium gap include Mining Software Repositories, Tool Development, Statistical Analysis, Communication, and Critical Thinking.

## Development Strategy for Identified Gaps

The skills identified with High and Medium gaps require targeted development strategies. Given that Static Analysis & Parsing and Algorithm Design present the most critical gaps, these areas receive priority attention through a combination of formal education and practical application.

Formal Education and Documentation Study addresses the theoretical foundation required for high-gap skills. For Static and Syntactic Analysis, this means putting into practice the concepts of Abstract Syntax Tree through compiler design courses, taking a look at the documentation of tools like Hadolint and DRMiner and reading articles on empirical software engineering. The process started with simpler Dockerfiles before progressing to more complex refactoring cases. For Algorithm Design, the study involves analyzing pattern detection algorithms and version comparison methods mentioned in the MSR literature.

The main strategy to turn theoretical knowledge into practical skills is learning by doing, which in this context means building the prototype progressively and learning from the challenges that come up during implementation. This involves building the prototype tool in a progressive way, learning from the challenges of implementation and correcting errors through continuous adjustments. Facing the difficulties of the Dockerfile's complex syntax directly provides a deeper understanding than theoretical study could offer, since practical exposure reveals edge cases and inconsistencies that are not always evident when reading the specification. Similarly, for algorithm design, the practical implementation of detection rules, together with tests against known refactoring patterns, allows these skills to be developed continuously throughout the project, reinforcing the connection between theoretical knowledge and its concrete application. Supervision and mentorship work at a different level, since regular guidance from the academic supervisor corrects the course of methodological decisions before their consequences propagate through the work: feedback on implementation approaches arrives while an approach can still be changed, and targeted advice on academic writing arrives while a chapter is still being written rather than after it is finished.

This process contributes to the development of medium-level skills such as communication and critical analysis, since frequent progress meetings require that results be presented clearly and that strategy choices be properly justified, and a choice that cannot be justified out loud is usually one that was not fully reasoned through in the first place.

Autonomous study and group analysis facilitate the development of skills in repositories for the mining and evaluation of statistical data. This includes automating the collection of data from platforms such as GitHub, working with data analysis tools and consulting public repositories to understand how developers structure Infrastructure as Code in real projects.

## Monitoring and Impact on Success

Managing skills is not something that happens only at the beginning of the project, it needs to be monitored continuously throughout the whole project so that any gaps that appear can be addressed in time. Regular reviews are important to make sure that what the project requires at each stage is still aligned with the skills and resources that are available at that point. Similarly, as the project progresses to the evaluation phase, proficiency in statistical analysis tools must be strengthened to properly interpret empirical results. Continuous identification of emerging skill gaps mitigates the risk of producing invalid results or delaying thesis submission [39].

The success of this dissertation depends directly on the competencies applied throughout the project, from tool design to final analysis of results [40]. Proficiency in IaC concepts and algorithm design is critical because errors in parsing logic compromise the validity of collected data, undermining the entire study. Effective project management is vital to coordinate time-intensive tasks such as mining repositories and building images, ensuring the schedule is met without bottlenecks. Analytical thinking enables statistical correlation of metrics with refactoring patterns to determine their true impact. Domain knowledge in Docker internals and security is essential for correctly classifying refactorings by quality dimension. Finally, communication skills determine the effectiveness of conveying complex technical concepts in the written dissertation and synthesizing findings for the scientific community.

Summary for Appendix A. This appendix established the technical and operational foundation for skill management throughout the project. The required competencies span technical skills (Docker, IaC, Static Analysis, Algorithm Design, MSR, Programming), analytical skills (statistical analysis, data interpretation), and interpersonal skills (communication, project management). The proficiency assessment identified two High-gap areas requiring immediate attention, namely Static Analysis & Parsing and Algorithm Design, along with five Medium-gap areas requiring progressive development. The development strategy combines formal education for theoretical grounding with learning-by-doing for practical consolidation, supported by supervision and self-directed study. Continuous monitoring ensures that competencies evolve alongside project requirements, while the direct relationship between skill proficiency and project success underscores the importance of addressing identified gaps systematically. This preparation is what allows the prototype tool to be developed in a structured way and the Dockerfile refactorings to be properly evaluated throughout the project.

# Appendix B — Project Planning

This section addresses project planning, providing elements of the project charter such as stakeholders, benefits, constraints, and assumptions. Other topics covered include the project scope, the different phases, and the expected deliverables. This includes the WBS and a project schedule shown as a Gantt chart, where the scope, milestones and costs are highlighted, as well as how the project is monitored and controlled. Besides that, the chapter also covers how risks are identified and managed so that the project can be completed successfully.

Project planning is one of the most essential stages in the project management process. It establishes the groundwork for the whole project and strongly influences the success or failure of the final outcome. The project planning phase provides a structured framework for implementing the detection of refactorings in Dockerfiles and evaluating their effects on performance, maintainability, and security. By precisely defining what is to be delivered, establishing a detailed schedule, and implementing robust monitoring and risk management practices, this plan provides the foundation upon which a successful project will be delivered within the established constraints and scope.

## Scope and Objectives

The stakeholders of the project include the supervisor, Isabel Azevedo, who is responsible for academic guidance and project oversight. The end-users would include development and DevOps teams who write and maintain Dockerfiles in professional environments, as well as organizations that rely on containerized environments to ensure efficiency, security, and cost control. If industrial partnerships exist, these would also be considered relevant stakeholders, providing feedback on the practical applicability of the tool and the catalogue being developed.

The project brings several significant benefits. First, it contributes to an improved understanding of Dockerfile refactoring practices through the extension of an existing catalogue, classifying refactorings across the quality dimensions of performance, security, and maintainability. Additionally, a prototype tool is developed to automate the detection of refactorings and empirically evaluate their impact on four quality metrics. The project provides practical insights into how refactorings influence containerised systems, supported by empirical evidence of their measured effects.

The project must remain in compliance with the deliverables scheduled for completion by September 20th, 2026. The scope is strictly confined to Dockerfile refactorings, excluding any other Infrastructure as Code technologies. The tool's development is restricted to a prototype phase, and a fully mature, production-ready solution is not expected. The analysis is confined to a single Dockerfile per execution, so refactorings distributed across several artefacts of a repository lie outside the delivered scope.

The key assumptions of the project are as follows. It is assumed that a purpose-built corpus of version-controlled repositories can be authored to exercise every catalogue refactoring under known conditions, since public repository histories cover the catalogue only incidentally and mix refactorings with unrelated edits in ways that cannot be separated without prior interpretation. It is assumed that the four impact metrics, image size, vulnerability count, linter warnings, and logical instruction count, can be measured automatically through established external tooling. The existing literature is expected to provide a sufficient basis for the extension of the refactoring catalogue, and adequate tools and frameworks are assumed to be available for prototype development. Finally, it is assumed that effective communication with the supervisor will be maintained regarding the establishment of an agreed timeline for the main deliverables.

The project is organized into seven phases aligned with the research methodology. Planning and Management defines the objectives, scope, schedule, and risk strategy. The Literature Review establishes the theoretical basis and identifies the research gap. Extended Catalogue Construction produces the classification of fourteen refactoring types across three quality dimensions, supported by controlled experiments. Prototype Design specifies the requirements and the architecture. Prototype Implementation and Verification builds the five pipeline components and establishes their correctness through testing and conformance auditing. Evaluation Corpus and Validation constructs the artefacts against which the prototype is evaluated and carries out that evaluation. Closure delivers the final documentation, the public artefacts, and the defence materials.

The requirements analysis and work structuring include a systematic review of the literature, the extension of the catalogue based on solid principles and empirical evidence, the design and implementation of the prototype, the construction of a purpose-built evaluation corpus and of dedicated impact validation repositories, experimentation with rigorous metric collection, and detailed documentation of the entire process. Expected results include the extended refactoring catalogue organized by quality dimensions, the prototype source code

with technical documentation, the evaluation corpus with its reference specification, the empirical validation results for the four metrics, and the final dissertation document delivered in two phases according to the academic calendar.

## Work Breakdown Structure

A Work Breakdown Structure is a project management tool used to break down the scope of a project into smaller, more manageable elements. This hierarchical breakdown organizes the work from the overall objective down to the specific tasks, making the planning, execution, and control of the project easier. The WBS has a hierarchical structure in which higher levels represent the overall project and lower levels progressively decompose it into components and tasks. Each element is associated with a specific deliverable, and there is no overlap between tasks. By defining well what is inside the area, the WBS helps to identify all the work that needs to be done and reduces the risk of letting important activities pass.

The project is structured in seven main phases, namely Project Planning and Management, Literature Review, Extended Catalogue Construction, Prototype Design, Prototype Implementation and Verification, Evaluation Corpus and Validation, and Closure. Each phase consists of the tasks that detail the activities necessary to achieve the project's objectives.

To ensure methodological rigor, these WBS phases were explicitly aligned with the six activities of the Design Science Research (DSR) methodology [11]. This mapping ensures that every technical task contributes directly to the research objectives. Table B.1 summarizes how the WBS phases correspond to the DSR process and the respective deliverables produced.

| **DSR Activity** | **WBS Phase(s)** | **Deliverables** |
|---|---|---|
| **Problem identification and motivation** | Project Planning and Management | Project charter, risk management plan, Gantt chart |
| **Definition of objectives** | Project Planning and Management, Literature Review | Research questions (RQ1, RQ2, RQ3), gap identification report, systematic literature review |
| **Design and development** | Extended Catalogue Construction, Prototype Design, Prototype Implementation and Verification | Extended refactoring catalogue (14 types classified across three quality dimensions), prototype tool covering version-control retrieval, refactoring detection, performance analysis, data extraction, and report generation |
| **Demonstration** | Evaluation Corpus and Validation | Purpose-built evaluation corpus, reference specification, impact |

| DSR Activity | WBS Phase(s) | Deliverables |
|---|---|---|
| | | validation repositories, prototype execution results |
| **Evaluation** | Evaluation Corpus and Validation | Detection conformance results, impact measurement results for the four metrics, threats to validity analysis |
| **Communication** | Closure | Dissertation document, public artefacts and reproduction scripts on GitHub |

Table B.1 - Mapping of DSR Activities to WBS Phases and Deliverables

Figure B.1 presents the complete Work Breakdown Structure for the project:

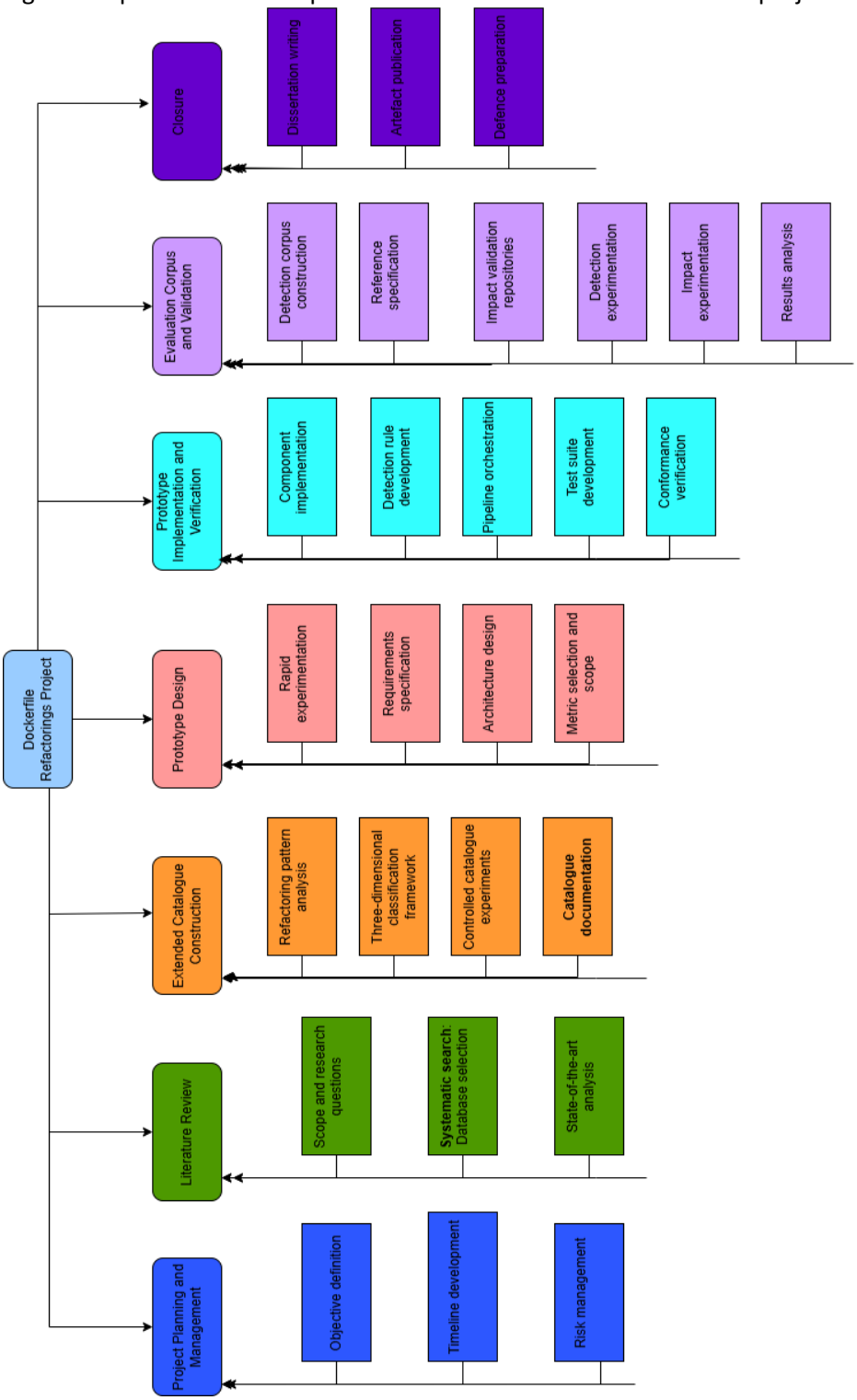



Figure B.1 - Work Breakdown Structure (WBS) for the Docker refactorings project

The project planning and management phase establishes the operational basis of the work. Objective definition delimits the research problem and fixes the criteria by which the outcome is judged. Timeline development produces the schedule against which progress is measured, allocating time to each phase according to its dependencies. Risk management identifies the threats to the project, assesses their severity, and defines a response for each. Figure B.2 details the tasks within this phase.

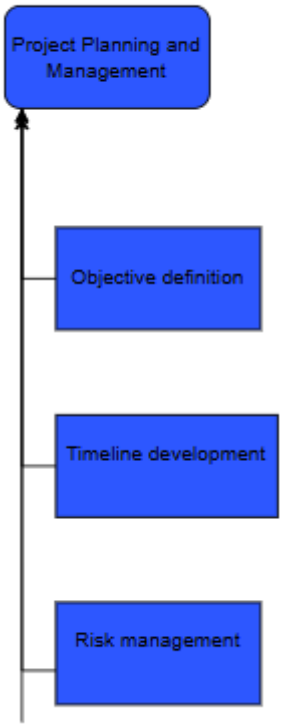



Figure B.2 - Tasks of the Project Planning and Management phase

The literature review phase establishes the theoretical basis of the work and locates the gap it addresses. Scope and research questions delimits the enquiry and fixes the criteria governing which literature is admitted. Systematic search applies those criteria to the body of published work, following a protocol that makes the selection reproducible. State-of-the-art analysis examines what the retained material establishes about Dockerfile quality and about the tools available for its analysis, and identifies what none of them provides. Figure B.3 shows the breakdown of this phase.

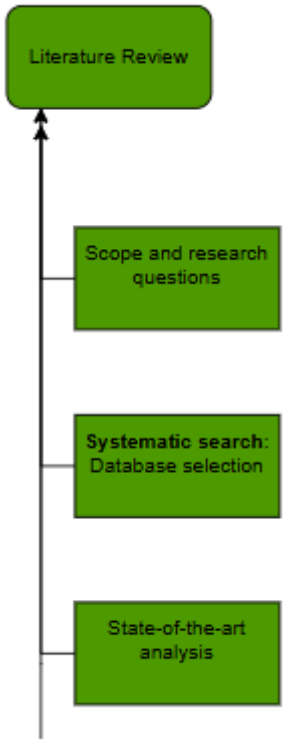



Figure B.3 - Tasks of the Literature Review phase

The extended catalogue construction phase produces the principal conceptual contribution of the work. Refactoring pattern analysis consolidates what the literature and the official documentation establish about each transformation. Three-dimensional classification framework defines how the effect of a refactoring is expressed across the quality dimensions, and on what basis a direction is assigned. Controlled catalogue experiments supply the empirical evidence for the classifications that cannot be derived from the literature alone. Catalogue documentation consolidates the result into the fourteen entries and their justifications. Figure B.4 shows the tasks involved.

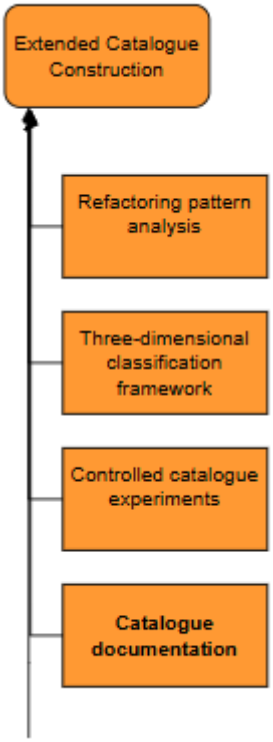



Figure B.4 - Tasks of the Extended Catalogue Construction phase

The prototype design phase transforms the conceptual contribution into a technical specification. Rapid experimentation establishes the feasibility of each candidate technology against the operation it is to perform, so that the specification rests on demonstrated capability rather than on expectation. Requirements specification defines what the system must do and the conditions under which it must do it, with an acceptance criterion for each requirement. Architecture design decomposes the system into components and fixes the boundaries between them. Metric selection and scope determines what is measured and what is deliberately left unmeasured. Figure B.5 illustrates this phase.

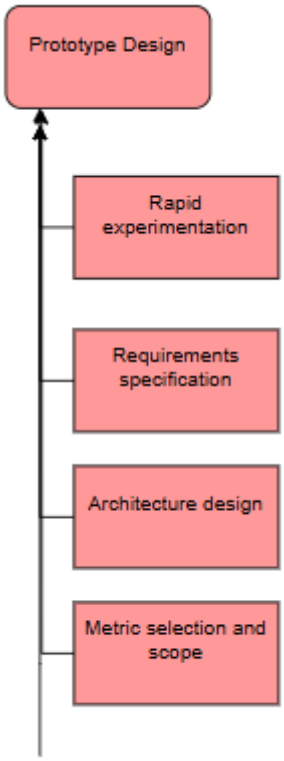



Figure B.5 - Tasks of the Prototype Design phase

The prototype implementation and verification phase realises the design in code and establishes its correctness. Component implementation produces the pipeline components as independent modules. Detection rule development produces the rules through which each catalogued refactoring is recognised, governed by the principles that determine how a detection is reported. Pipeline orchestration establishes the execution sequence and the management of the resources it consumes. Test suite development exercises each component in isolation and verifies the interaction between them. Conformance verification checks the implementation against its specification programmatically rather than by inspection. Figure B.6 illustrates this phase.

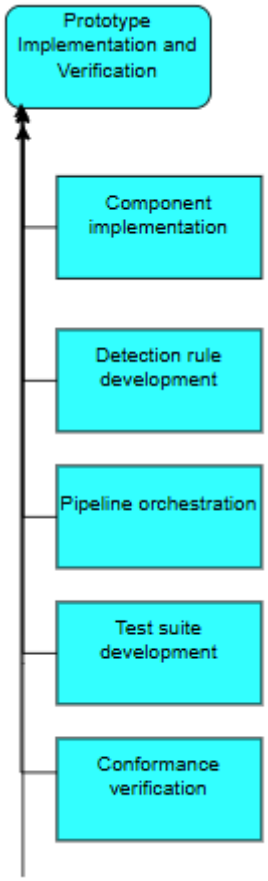



Figure B.6 - Tasks of the Prototype Implementation and Verification phase

The evaluation corpus and validation phase constructs the artefacts against which the prototype is evaluated and carries out that evaluation. It comprises two construction efforts of distinct natures, followed by the experimentation each one supports.

Detection corpus construction produces a purpose-built repository in which every commit is authored to exercise a defined portion of the catalogue, designed so that sensitivity and specificity are measured independently rather than conflated. Reference specification records the expected outcome for each case, authored alongside the corpus and kept external to the tool, which has no access to it during execution. Impact validation repositories address a different requirement: where the detection corpus maximises syntactic breadth, each of these isolates a single effect, so that the corresponding metric can be read without interference from unrelated instructions.

Detection experimentation executes the prototype over the corpus and adjudicates the outcome against the specification. Impact experimentation measures the four quality metrics over the validation repositories. Results analysis closes the phase, cross-referencing the raw tool artefacts against the structured reports and establishing the threats to validity that bound the interpretation of the findings. Figure B.7 details this phase.

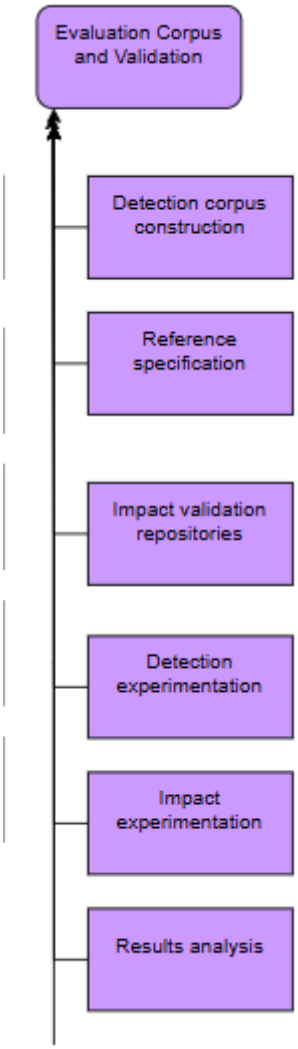



Figure B.7 - Tasks of the Evaluation Corpus and Validation phase

The closure phase consolidates the work and prepares the final deliverables. Dissertation writing turns the drafts accumulated throughout the project into a coherent document, organising the findings, relating them to the existing literature, and identifying both the original contributions and the limitations of what was performed. Artefact publication makes the code, the validation data and the reproduction scripts available, so that the study can be replicated. Defence preparation produces the materials for the public defence and verifies compliance with the formal submission requirements. Figure B.8 shows the Closure phase tasks.

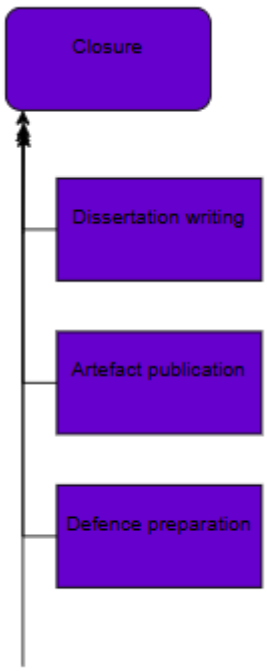



Figure B.8 - Tasks of the Closure phase

## Integrated Project Schedule

Key project milestones have been established to ensure that the project timeline remains on track. A Gantt chart is a graphical representation of a timeline that includes critical milestones, start and end dates for each activity. This chart provides a roadmap for the project, allowing visualization of dependencies, identification of critical paths, and monitoring of progress over time. Figure B.9 presents the complete project schedule as a Gantt chart.

| | Task Mode | Task Name | Duration | Start | Finish | Predecessors |
|---|---|---|---|---|---|---|
| 1 | | Dockerfile refactoring: detection and effects | 234 days? | Wed 29/10/25 | Sun 20/09/26 | |
| 2 | | Project Planning & Management | 15 days | Wed 29/10/25 | Tue 18/11/25 | |
| 6 | | Literature Review | 30 days? | Wed 19/11/2 | Tue 30/12/25 | |
| 10 | | Extended Catalogue Construction | 32 days | Wed 31/12/25 | Thu 12/02/26 | |
| 15 | | Prototype Design | 37 days | Wed 11/02/2 | Thu 02/04/26 | |
| 20 | | Prototype Implementation and Verification | 56 days | Fri 03/04/26 | Fri 19/06/26 | |
| 26 | | Evaluation Corpus and Validation | 50 days | Mon 22/06/26 | Fri 28/08/26 | |
| 33 | | Closure | 16 days | Mon 31/08/2 | Sun 20/09/26 | |
| 37 | | Incremental dissertation drafting and documentation | 234 days | Wed 29/10/25 | Sun 20/09/26 | |
| 38 | | 1º meeting | 1 day | Thu 18/12/25 | Thu 18/12/25 | |
| 39 | | 2º meeting | 1 day | Fri 20/03/26 | Fri 20/03/26 | |
| 40 | | 3º meeting | 1 day | Mon 15/06/2 | Mon 15/06/2 | |
| 41 | | 4º meeting | 1 day | Thu 10/09/26 | Thu 10/09/26 | |



Figure B.9 - Project schedule showing the seven dissertation phases, their durations, dependencies, and key milestones from October 2025 to September 2026

A hybrid Agile approach is adopted for this project, since it supports iterative development and continuous adaptability, aligning well with the explorative nature of detecting Dockerfile refactorings and evaluating their effects. The method combines Scrum and Kanban, Scrum being responsible for structuring the development cycles into fixed iterations, while Kanban is used alongside it to keep the workflow visible and moving continuously, so that tasks do not accumulate or get blocked between iterations. Exploratory work of this kind regularly surfaces syntactic variants and detection limitations that no requirement written at the outset could have anticipated, and the short development cycles make it possible to incorporate each such finding into the catalogue and the detection rules as it appears.

It also encourages delivering small working components frequently, such as partial versions of the refactoring catalogue, a manual proof of concept to test feasibility, or basic scripts for metric collection. Besides that, this iterative way of working makes it easier to keep writing the dissertation chapters at the same time as the technical work is being done, which keeps the supervisor informed and ensures that progress never stops.

Frequent meetings with the supervisor help validate methodological choices early, reducing the risk of pursuing incorrect assumptions. Continuous integration and the testing of detection algorithms reduce the accumulation of errors in the final phases. The Agile approach focused on testing and experimentation is compatible with the iterative validation of the detection rules and the metric collection modules.

Waterfall is far too structured for an exploratory research project that requires predefined requirements at the outset, which is unrealistic when the understanding of refactorings and of their measured behaviour keeps evolving throughout the study. On top of that, Scrum on its own does not quite fit the research context, and Kanban lacks proper milestones, which can easily hold up tasks that depend on each other. The combined hybrid approach allows the project to adapt to new research findings while guaranteeing the delivery of high-quality, empirically validated components of the detection tool and catalogue.

The project execution phase refers to the actual implementation of the activities planned and specified in the WBS and schedule. It is expected that most tasks will be completed within the defined deadlines. However, it is recognized that technical limitations, the availability of computational resources, or the need to refine requirements during research may introduce risks of delay. These challenges are anticipated to be most likely during the implementation phase, due to the complexity of the detection rules, and during the evaluation phase, where the construction of the corpus and the execution of real builds both consume substantial time. To mitigate potential delays caused by the complexity of detecting refactorings or syntactic variations in Dockerfiles, an iterative approach is adopted, first developing the simplest rules and progressively refining them. If integration challenges arise, component requirements are promptly reassessed.

The primary objective is to complete all tasks by the deadline of 20 September 2026, which is what makes planning and risk management operational concerns rather than documentation exercises. Regular communication with the supervisor is what allows adjustments to be made proactively rather than after a deadline has already slipped: a scope change discussed at a progress meeting can be absorbed by reallocating time between phases, whereas the same change noticed at the end of a phase cannot. The constraint is therefore not the schedule alone but the schedule read against the original scope, since finishing on time by narrowing what is delivered would satisfy the first and defeat the second.

The existing control mechanisms ensure compliance with the schedule and the quality of the results, with performance evaluation focused on conformity with the plan and the achievement of the research objectives. Tools such as Jira or Trello are used to monitor and track the progress of the tasks, while GitHub is used for version control of the code, the evaluation corpus, and the validation artefacts, ensuring traceability throughout. Regular meetings with the supervisor act as the main checkpoint for validating methodological decisions and addressing technical challenges. Reviewing the key milestones and getting feedback on the intermediate deliverables, such as the catalogue and the prototype, helps to make sure the project stays on track and that any conceptual errors are caught early, so that the quality of the final delivery is not affected. Lastly, the project concludes with a Closure phase, specifically dedicated to final dissertation writing, which focuses on refining, perfecting, and consolidating all iterative drafts and research findings into a polished final document for submission by 20 September 2026.

# Risk Management

Some risks identified in the project include delays in the timeline due to challenges in detecting refactorings, unavailability of necessary tools or technologies, and limitations in the evaluation material available for experimentation. These are mitigated through regular reviews, clear communication with the supervisor, and appropriate contingency planning. During the execution of the project, several potential risks were identified that could negatively impact progress and expected results.

**Risk 1: Difficulty in Automatic Refactoring Detection**

- Description: Some refactorings may be difficult to detect automatically due to syntactic or contextual variations in Dockerfile writing styles.
- Cause: Complexity and diversity of Dockerfile writing practices in the community, with no standardized way to express the same functionality.
- Effect: Reduced precision of the developed tool and need for manual validation, significantly increasing required effort.
- Risk Owner: Principal Investigator (Student).
- Probability (1-5): 3
- Impact (1-5): 4
- PI Score: 12
- Expected Result, No Action: Tool with low detection rate and unreliable results that compromise study validity.
- Risk Response Type: Mitigate
- Response Description: Formulate each detection rule over the structure of the instruction rather than over its text; author a dedicated class of test cases expressing each refactoring through syntactic variants a naive implementation would miss; iterate development based on continuous supervisor feedback and preliminary results.

**Risk 2: Author Bias in a Purpose-Built Evaluation Corpus**

- Description: The evaluation corpus and the reference specification are authored by the same researcher who develops the prototype, so the transformations against which the tool is judged originate from the same source as the implementation.
- Cause: Public repository histories cover the catalogue only incidentally and mix refactorings with unrelated edits, which makes a purpose-built corpus the only practical instrument for systematic coverage.
- Effect: Accuracy figures that establish faithful implementation of the catalogue but do not establish behaviour on refactorings performed by unrelated developers.
- Risk Owner: Principal Investigator (Student) and Supervisor.
- Probability (1-5): 4
- Impact (1-5): 3
- PI Score: 12
- Expected Result, No Action: Results perceived as self-confirming, with generalisation to real-world practice unsupported.
- Risk Response Type: Mitigate and Accept
- Response Description: Author the reference specification independently of any observation of the tool, stating what the tool must do rather than what it was observed to do; keep the specification external to the engine, which has no access to it during execution; perform adjudication through a program distinct from the one producing the results; document the residual limitation explicitly as a threat to validity and identify independent replication as future work.

**Risk 3: Incompatibility with Existing Analysis Tools**

- Description: Tools like DRMiner may not be publicly available or function as expected per documentation.
- Cause: Tools are often not widely available or poorly documented, hindering replication.
- Effect: Need to develop components from scratch, resulting in significant schedule delays.
- Risk Owner: Principal Investigator (Student).
- Probability (1-5): 2
- Impact (1-5): 3
- PI Score: 6
- Expected Result, No Action: Tool development slower than planned.
- Risk Response Type: Mitigate
- Response Description: Study alternative tools such as static analysis tools and available Dockerfile parsers; plan from the start to develop critical components in-house rather than depending exclusively on third-party tools.

**Risk 4: Ambiguity in Refactoring Classification**

- Description: Different refactorings may affect several quality dimensions simultaneously, and a classification forcing each one into a single category would misrepresent that behaviour.
- Cause: Multifaceted nature of refactorings.
- Effect: A catalogue that conceals trade-offs, hindering result interpretation and reducing the practical utility of the work.
- Risk Owner: Principal Investigator (Student) and Supervisor.
- Probability (1-5): 2
- Impact (1-5): 3
- PI Score: 6
- Expected Result, No Action: Inconsistent classification and low catalogue utility.
- Risk Response Type: Mitigate
- Response Description: Evaluate each refactoring independently across all three quality axes, recording improvements, regressions and neutral outcomes on each rather than assigning a dominant dimension; discuss ambiguous cases with the supervisor to reach consensus; validate the classification against the literature and against controlled experiments.

**Risk 5: Delays in Metrics Collection**

- Description: Executing actual Docker builds to measure image sizes can be time-consuming, particularly where each measurement requires building and scanning a pair of images.
- Cause: Computational limitations and the need to execute real builds.
- Effect: Delay in the evaluation phase, possibly compromising the overall schedule of the final project.
- Risk Owner: Principal Investigator (Student).
- Probability (1-5): 3
- Impact (1-5): 3
- PI Score: 9
- Expected Result, No Action: Experimentation could be incomplete or rushed at project end.
- Risk Response Type: Mitigate
- Response Description: Automate metric collection through scripts wherever possible; separate the detection evaluation, which requires no image builds, from the impact evaluation, which does, so that the former can proceed independently of build cost.

**Risk 6: Non-Reproducibility of Environmental Measurements**

- Description: Image size may not be byte-reproducible across builds of identical inputs, and vulnerability counts depend on an external database that evolves independently of the code under analysis.
- Cause: Build-time variation introduced by the container engine, and the daily update cycle of the vulnerability database.
- Effect: Measured deltas of small magnitude indistinguishable from environmental variation, and counts that differ between analyses performed on different dates for reasons unrelated to the refactoring.
- Risk Owner: Principal Investigator (Student).
- Probability (1-5): 4
- Impact (1-5): 3
- PI Score: 12
- Expected Result, No Action: Measurements reported as exact when they are conditional, leading to unsupported conclusions about small effects.
- Risk Response Type: Mitigate and Accept
- Response Description: Record the version of every external tool and the timestamp of the vulnerability database in every report, so that any difference can be attributed either to the code or to the environment; perform both states of a comparison within a single analysis session; quantify the variation band empirically and state it explicitly, so that deltas falling within it are read as indistinguishable from zero rather than as measured effects.

**Risk 7: Scope Creep**

- Description: Temptation to expand scope beyond planned, for example adding more refactoring types or analyzing other Infrastructure as Code files beyond Dockerfiles.
- Cause: Natural academic interest and interesting discoveries during research that may suggest attractive extensions.
- Effect: Project completion delay and focus dispersion compromising analysis depth.
- Risk Owner: Principal Investigator (Student) and Supervisor.
- Probability (1-5): 3
- Impact (1-5): 4
- PI Score: 12
- Expected Result, No Action: Project might not be completed on time.
- Risk Response Type: Accept and Monitor
- Response Description: Define scope clearly at the start and commit to respecting it; document interesting ideas for future work instead of incorporating them into the current project; consult the supervisor before any scope expansion to assess if it is justifiable and viable.

**Risk 8: Environmental Exposure During Image Construction**

- Description: Building images and running containerised analysers pulls base images and tool images from public registries, which introduces external content into the execution environment.
- Cause: Dependence on publicly hosted base images and on analyser containers resolved at execution time.
- Effect: Compromise of the development environment, or measurements taken against an image whose content differs from the one intended.
- Risk Owner: Principal Investigator (Student).
- Probability (1-5): 2
- Impact (1-5): 4
- PI Score: 8
- Expected Result, No Action: Environment could be compromised, or measurements contaminated by an unnoticed change in an external image.
- Risk Response Type: Mitigate
- Response Description: Execute all builds and scans inside an isolated virtualised environment; refrain from re-pulling the base image between the two states of a comparison, so that both build against the same locally cached layers; remove built images once the measurement completes.

**Risk 9: Limited Generalisation Beyond the Constructed Corpus**

- Description: The evaluation rests on repositories authored for the purpose, so the results do not establish behaviour on Dockerfiles written by unrelated developers, nor the frequency with which the catalogued refactorings occur in practice.
- Cause: The systematic coverage the evaluation requires cannot be obtained from public repository histories, and confidentiality restrictions preclude access to enterprise codebases.
- Effect: Findings that establish faithful implementation of the catalogue without establishing external validity.
- Risk Owner: Principal Investigator (Student) and Supervisor.
- Probability (1-5): 4
- Impact (1-5): 2
- PI Score: 8
- Expected Result, No Action: Overstated conclusions about the applicability of the results.
- Risk Response Type: Accept

- Response Description: Construct the corpus to span the widest syntactic range practicable, including variants a naive implementation would miss; document the limitation explicitly in the threats to validity; identify evaluation against third-party repositories as future work.

**Risk 10: Inadequate Measurement Tools**

- Description: Tools for measuring image sizes, vulnerability counts, or linter warnings may not be sufficiently accurate or consistent.
- Cause: Technical limitations of available tools.
- Effect: Unreliable metrics leading to wrong conclusions about refactoring impact.
- Risk Owner: Principal Investigator (Student).
- Probability (1-5): 2
- Impact (1-5): 4
- PI Score: 8
- Expected Result, No Action: Results would be questionable.
- Risk Response Type: Mitigate
- Response Description: Validate each measurement against a scenario whose expected outcome is known by construction; retain the unmodified output of every external tool alongside the processed report, so that any reported figure can be traced back to the raw evidence; document the known limitations of the tools used.

**Risk 11: Negative or Inconclusive Experimental Results**

- Description: Experimentation may not reveal significant impacts of refactorings on the quality metrics considered, either because effects are small or because measurement variability masks them.
- Cause: Refactorings may have genuinely small effects under certain conditions, or environmental variability masks existing effects.
- Effect: Difficulty drawing strong conclusions and perception of project failure by the investigator.
- Risk Owner: Principal Investigator (Student) and Supervisor.
- Probability (1-5): 2
- Impact (1-5): 3
- PI Score: 6
- Expected Result, No Action: Dissertation could have weak conclusions.
- Risk Response Type: Accept
- Response Description: Accept that negative results also contribute to science. Being honest about what was actually found, discussing the limitations of the study, and

explaining why certain expected effects may not have appeared are all part of good research practice. These gaps can then guide future work.

**Risk 12: Lack of Time for Adequate Dissertation Documentation**

- Description: Time pressure may compromise dissertation writing quality, especially if previous phases are delayed.
- Cause: Underestimating the time needed for quality academic writing, or accumulated delays in other project phases.
- Effect: Incomplete or low-quality dissertation that does not do justice to the work performed.
- Risk Owner: Principal Investigator (Student).
- Probability (1-5): 3
- Impact (1-5): 5
- PI Score: 15
- Expected Result, No Action: Delivery could be low quality or even result in failure.
- Risk Response Type: Mitigate
- Response Description: Set aside time for writing from the beginning of the project, documenting the process throughout instead of leaving everything for the end, and reviewing the chapters with the supervisor regularly to ensure good quality and continuous improvements before the final submission of the document.

This appendix consolidated the scope, the work breakdown structure, the schedule, and the risk management strategy that supported the execution of the work reported in the main chapters of this dissertation.

# Appendix C — Maintainability Screen Rules

The rules comprising the maintainability screen were systematically derived through a strict dimensional exclusion process applied to the default rule sets of Hadolint and ShellCheck. To construct a clean metric that does not overlap with other quality dimensions, the selection followed four criteria. Baseline inclusion: all default-enabled rules related to code structure, notation, metadata, build logs, and shell correctness were initially considered. Performance exclusion: rules primarily targeting image-size optimization or layer-cache manipulation, such as cache cleanup, were deliberately removed, as their effect is already captured directly by ΔSize. Security exclusion: rules targeting security attack surfaces or privilege configurations were excluded to prevent duplication with ΔCVEs. Retained scope: the resulting 47 rules isolate structural correctness, clean notation, and maintainable authoring practices, acting as a proxy for the maintainability dimension alongside ΔInstr.

The screen is applied to every extended-catalogue experiment. It is held as a single declarative rule list in the Data Extractor, so that it can be inspected and revised independently of the parsing logic. Rules that are disabled by default in Hadolint — notably the label-schema rules DL3049–DL3058 and the optional HEALTHCHECK rule DL3057 — are not part of the screen, since they do not fire unless explicitly configured. Table C.1 lists the 47 rules, grouped by the sub-category that motivated their inclusion.

| Category | Rule ID | Source | Description / Objective |
|---|---|---|---|
| **Reproducibility** | DL3005 | Hadolint | Do not use apt-get dist-upgrade. |
| | DL3006 | Hadolint | Always tag the version of an image explicitly. |
| | DL3007 | Hadolint | Using latest is prone to errors if the image will ever update. Pin the version explicitly to a release tag. |
| | DL3008 | Hadolint | Pin versions in apt-get install. |
| | DL3013 | Hadolint | Pin versions in pip. |
| | DL3016 | Hadolint | Pin versions in npm. |
| | DL3017 | Hadolint | Do not use apk upgrade. |
| | DL3018 | Hadolint | Pin versions in apk add. |
| | DL3028 | Hadolint | Pin versions in gem install. |
| | DL3031 | Hadolint | Do not use yum update. |
| | DL3033 | Hadolint | Specify version with yum install -y <package>-<version>. |
| | DL3035 | Hadolint | Do not use zypper update. |
| | DL3037 | Hadolint | Specify version with zypper install -y <package>[=]<version>. |
| | DL3039 | Hadolint | Do not use dnf update. |
| | DL3041 | Hadolint | Specify version with dnf install -y <package>-<version>. |
| | DL3062 | Hadolint | Pin versions in go install. |

| Category | Rule ID | Source | Description / Objective |
|---|---|---|---|
| **Structural correctness** | DL3000 | Hadolint | Use absolute WORKDIR. |
| | DL3003 | Hadolint | Use absolute paths, or use WORKDIR to switch to a directory. |
| | DL3011 | Hadolint | Valid UNIX ports range from 0 to 65535. |
| | DL3012 | Hadolint | Multiple HEALTHCHECK instructions. |
| | DL3021 | Hadolint | COPY with more than 2 arguments requires the last argument to end with /. |
| | DL3022 | Hadolint | COPY --from should reference a previously defined FROM alias. |
| | DL3023 | Hadolint | COPY --from cannot reference its own FROM alias. |
| | DL3024 | Hadolint | FROM aliases (stage names) must be unique. |
| | DL3043 | Hadolint | ONBUILD, FROM or MAINTAINER triggered from within ONBUILD instruction. |
| | DL3044 | Hadolint | Do not refer to an environment variable within the same ENV statement where it is defined. |
| | DL3045 | Hadolint | COPY to a relative destination without WORKDIR set. |
| | DL3061 | Hadolint | Invalid instruction order. Dockerfile must begin with FROM, ARG or comment. |

| Category | Rule ID | Source | Description / Objective |
|---|---|---|---|
| | DL3063 | Hadolint | Stage name should not be a reserved word. |
| | DL4003 | Hadolint | Multiple CMD instructions found. If you list more than one CMD then only the last CMD will take effect. |
| | DL4004 | Hadolint | Multiple ENTRYPOINT instructions found. If you list more than one ENTRYPOINT then only the last ENTRYPOINT will take effect. |
| **Usage and notation** | DL3001 | Hadolint | Command does not make sense in a container. |
| | DL3010 | Hadolint | Use ADD for extracting archives into an image. |
| | DL3014 | Hadolint | Use the -y switch. |
| | DL3025 | Hadolint | Use arguments JSON notation for CMD and ENTRYPOINT arguments. |
| | DL3027 | Hadolint | Do not use apt as it is meant to be an end-user tool, use apt-get or apt-cache instead. |
| | DL3029 | Hadolint | Do not use --platform= with FROM. |
| | DL3030 | Hadolint | Use the -y switch to avoid manual input yum install -y <package>. |
| | DL3034 | Hadolint | Non-interactive switch missing from zypper command: zypper install -y. |
| | DL3038 | Hadolint | Use the -y switch to avoid manual input dnf install -y <package>. |

| Category | Rule ID | Source | Description / Objective |
|---|---|---|---|
| | DL4001 | Hadolint | Either use Wget or Curl but not both. |
| | DL4005 | Hadolint | Use SHELL to change the default shell. |
| | DL4006 | Hadolint | Set the SHELL option -o pipefail before RUN with a pipe in. |
| **Metadata** | DL4000 | Hadolint | MAINTAINER is deprecated. |
| **Shell correctness (ShellCheck)** | SC2046 | ShellCheck | Quote this to prevent word splitting. |
| | SC2086 | ShellCheck | Double quote to prevent globbing and word splitting. |
| **Build logs** | DL3047 | Hadolint | Use wget --progress to avoid excessively bloated build logs. |

Table C.1 - Complete list of the 47 rules comprising the maintainability screen, grouped by their respective sub-categories, including their identifier, source tool, and official description.